RENTO – AN AI-POWERED CAR RENTAL PLATFORM WITH SMART DYNAMIC PRICING AND VEHICLE RECOMMENDATION

ANISHA GHATAN

(LCID: LC00017002320)

FINAL YEAR PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT FOR THE DEGREE OF BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)

FACULTY OF COMPUTER SCIENCE IN MULTIMEDIA

LINCOLN UNIVERSITY COLLEGE

MALAYSIA

[INSERT MONTH, YEAR OF SUBMISSION]

RENTO – AN AI-POWERED CAR RENTAL PLATFORM WITH SMART DYNAMIC PRICING AND VEHICLE RECOMMENDATION

ANISHA GHATAN

(LCID: LC00017002320)

FINAL YEAR PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT FOR THE DEGREE OF BACHELOR OF INFORMATION TECHNOLOGY (HONOURS)

FACULTY OF COMPUTER SCIENCE IN MULTIMEDIA

LINCOLN UNIVERSITY COLLEGE

MALAYSIA

[INSERT MONTH, YEAR OF SUBMISSION]

RECOMMENDATION

[To be received from Resource Center]

CERTIFICATION

This is to certify that the Final Year Project report entitled "Rento – An AI-Powered Car Rental Platform with Smart Dynamic Pricing and Vehicle Recommendation" submitted by Anisha Ghatan (LCID: LC00017002320) has been examined and is recommended for acceptance in partial fulfilment of the requirements for the degree of Bachelor of Information Technology (Honours), Faculty of Computer Science in Multimedia, Lincoln University College, Malaysia.

.....

Mr. Rohit Pandey

Internal Supervisor

Date: .....

.....

Prof. Dr. Govinda Prasad Acharya

Chairperson, RDSC

Date: .....

.....

Dr. Suman Thapaliya

Director / Department Head

Date: .....

.....

Ms. Sarala Karki

Research Learning & Development

Date: .....

DECLARATION OF AUTHENTICITY

I, Anisha Ghatan (LCID: LC00017002320), hereby declare that this Final Year Project report entitled “Rento – An AI-Powered Car Rental Platform with Smart Dynamic Pricing and Vehicle Recommendation” is the result of my own project work, carried out under the supervision of Mr. Rohit Pandey, except where otherwise stated and acknowledged. All sources of information used in this report have been cited and are listed in the References.

The system was developed from an open-source PHP car-rental template (“Car Rental Portal”). The template is identified and cited in Chapter 3, which also sets out which parts of the system were rebuilt or newly added during this project.

I declare that an artificial-intelligence assistant, Claude (Anthropic), was used during this project as a coding and documentation assistant. It helped to write and refactor source code, automated tests and diagrams, and to draft and edit parts of this report. I directed the work, set the requirements and design decisions, reviewed, tested and verified all outputs, and I take full responsibility for the system and for the content of this report. The commits produced with this assistance are marked as co-authored in the project's Git history. This use of AI tools is in line with the policy of Lincoln University College: [INSERT statement on the college's AI policy].

This report has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution.

.....

Anisha Ghatan

LCID: LC00017002320

Date: .....

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my internal supervisor, Mr. Rohit Pandey, for his guidance, patience and constructive feedback throughout this project. His advice on scope, methodology and evaluation shaped both the system and this report.

I am grateful to Dr. Suman Thapaliya, Head of Department, and to the Faculty of Computer Science in Multimedia at Lincoln University College for the knowledge, facilities and support provided during the Bachelor of Information Technology (Honours) programme.

I also acknowledge the open-source communities whose work made this project possible, including the authors of the original car-rental template, PHP, MySQL, Bootstrap, scikit-learn and PyTorch, and the use of the AI assistant Claude (Anthropic) as a coding and documentation aid, as described in the Declaration of Authenticity.

Finally, I thank my family and friends for their encouragement, understanding and support throughout my studies.

Anisha Ghatan

TABLE OF CONTENTS

RECOMMENDATION i

CERTIFICATION ii

DECLARATION OF AUTHENTICITY iii

ACKNOWLEDGEMENTS iv

TABLE OF CONTENTS v

LIST OF TABLES x

LIST OF FIGURES xii

ABBREVIATIONS AND ACRONYMS xiv

EXECUTIVE SUMMARY xvi

CHAPTER 1: INTRODUCTION 1

1.1 Background of the Study 1

1.2 Problem Statement 2

1.3 Objectives 3

1.3.1 General Objective 3

1.3.2 Specific Objectives 3

1.4 Scope of the Study 4

1.5 Significance of the Study 5

1.6 Limitations 5

1.7 Organization of the Report 6

CHAPTER 2: LITERATURE REVIEW 7

2.1 Introduction 7

2.2 Review of Related Works 7

2.2.1 Dynamic Pricing and Revenue Management 7

2.2.2 Demand Forecasting 9

2.2.3 Recommender Systems 11

2.2.4 Existing Rental Platforms 12
2.3 Theoretical Framework 15
2.3.1 Rule-Based Dynamic Pricing 16
2.3.2 Ensemble Learning Methods 17
2.3.3 Regression Evaluation Metrics 17
2.3.4 Content-Based Recommendation Scoring 18
2.4 Research Gap 19
2.5 Summary 20
CHAPTER 3: METHODOLOGY 21
3.1 Introduction 21
3.2 Development Methodology 21
3.2.1 Choice of Methodology 21
3.2.2 Starting Point of the Project 23
3.2.3 Use of Artificial-Intelligence Assistance 25
3.2.4 Version Control and Project Timeline 26
3.3 Data and Requirements Collection 27
3.4 Software Requirements Specification 28
3.4.1 Users of the System 28
3.4.2 Functional Requirements 29
3.4.3 Non-Functional Requirements 30
3.5 Tools and Technologies 31
3.6 Feasibility Study 32
3.6.1 Technical Feasibility 32
3.6.2 Operational Feasibility 33
3.6.3 Economic Feasibility 33
3.6.4 Schedule Feasibility 33
3.6.5 Legal and Ethical Feasibility 33

3.7 System Architecture 34
3.8 Summary 35
CHAPTER 4: SYSTEM DESIGN 37
4.1 Introduction 37
4.2 Use Case Diagram 37
4.3 Data Flow Diagrams 39
4.3.1 Context Diagram 40
4.3.2 Level 1 Data Flow Diagrams 40
4.3.3 Level 2 Data Flow Diagrams 42
4.4 Database Design 44
4.4.1 Entity Relationship Diagrams 44
4.4.2 Relationships and Keys 46
4.4.3 Design of the Main Tables 47
4.5 Class and Module Diagram 48
4.6 Sequence Diagrams 50
4.7 Activity Diagrams 55
4.8 User Interface Design 59
4.8.1 Design Principles 59
4.8.2 Design System 59
4.8.3 Page Design 60
4.9 Summary 63
CHAPTER 5: IMPLEMENTATION 64
5.1 Introduction 64
5.2 Development Environment and Code Structure 64
5.3 Database Implementation 65
5.4 Customer Modules 66
5.4.1 Car Listing and Car Page 66

5.4.2 Booking, My Bookings and Invoice 67
5.4.3 Find My Car and Packages 68
5.5 Administration Modules 68
5.6 JSON Application Programming Interface 70
5.7 Smart Pricing Engine 71
5.8 Forecast Pipeline 73
5.9 Machine-Learning Experiments 74
5.9.1 Purpose and Data 74
5.9.2 Models and Evaluation Method 75
5.9.3 Results: Demand 75
5.9.4 Results: Price 76
5.9.5 Discussion 77
5.10 Recommendation Engine 78
5.10.1 Trip Parser 78
5.10.2 Scoring 78
5.11 Security Implementation 80
5.12 Other Quality Features 81
5.13 Summary 82
CHAPTER 6: TESTING AND EVALUATION 83
6.1 Introduction 83
6.2 Test Strategy and Environment 83
6.3 Unit Testing 84
6.4 Integration Testing 85
6.5 System Testing 85
6.6 Evaluation of the Smart Pricing Engine 86
6.7 Evaluation of the Recommendation Engine 87
6.8 Performance Evaluation 89

6.9 Accessibility, Compatibility and Usability 90
6.10 Requirements Traceability 91
6.11 Summary 92
CHAPTER 7: CONCLUSION AND FUTURE WORK 93
7.1 Introduction 93
7.2 Summary of the Work 93
7.3 Achievement of the Objectives 93
7.4 Conclusions 94
7.5 Limitations 95
7.6 Future Enhancements 96
7.7 Final Remarks 97
REFERENCES 98

LIST OF TABLES

Table 1.1 Scope of the project 4

Table 2.1 Summary of related works 13

Table 2.2 Feature comparison of existing platforms and Rento 15

Table 3.1 Development phases and their main outputs 23

Table 3.2 Comparison of the baseline template and the final system 24

Table 3.3 Project timeline 27

Table 3.4 Functional requirements 29

Table 3.5 Non-functional requirements 31

Table 3.6 Tools and technologies 32

Table 4.1 Use case description: request a booking 39

Table 4.2 Data stores used in the data flow diagrams 39

Table 4.3 Data dictionary of tblbooking 47

Table 4.4 Data dictionary of tbl_price_forecast 48

Table 4.5 Main elements of the design system 60

Table 5.1 Structure of the source code 65

Table 5.2 Database migrations 66

Table 5.3 JSON API endpoints 71

Table 5.4 Pricing factors and their default values 71

Table 5.5 Models used in the experiments 75

Table 5.6 Results of Experiment 1: hourly demand (test set, 3,650 hours) 76

Table 5.7 Results of Experiment 1b: daily demand (test set, 131 days) 76

Table 5.8 Results of Experiment 2: day price (test set, 246 listings) 77

Table 5.9 Criteria extracted by the trip parser 78

Table 5.10 Criteria and weights of the recommendation engine 79

Table 5.11 Security measures 81

Table 6.1 Test plan 83
Table 6.2 Unit test results 84
Table 6.3 Integration test results 85
Table 6.4 System test cases 85
Table 6.5 Checks of the 60-day forecast (27 September 2026) 87
Table 6.6 Price of the Toyota Fortuner (base Rs 10,000) on typical days 87
Table 6.7 Results of the recommendation scenarios 88
Table 6.8 Server response times (30 requests each, local server) 89
Table 6.9 Page weight of the original template and of Rento (desktop, 1366 pixels) 90
Table 6.10 Requirements and test evidence 91
Table 7.1 Achievement of the specific objectives 94

LIST OF FIGURES

Figure 2.1 Conceptual framework of the study 16

Figure 3.1 Phased iterative development methodology 22

Figure 3.2 Gantt chart of the recorded development phases, generated from the Git commit history 27

Figure 3.3 System architecture of Rento 35

Figure 4.1 Use case diagram of Rento 38

Figure 4.2 Context diagram (level 0 data flow diagram) 40

Figure 4.3 Level 1 data flow diagram, part A: customer processes 41

Figure 4.4 Level 1 data flow diagram, part B: administration and background processes 42

Figure 4.5 Level 2 data flow diagram of process 5.0 (book and invoice) 43

Figure 4.6 Level 2 data flow diagram of process 8.0 (forecast prices) 44

Figure 4.7 Entity relationship diagram, part 1: bookings, fleet and smart pricing 45

Figure 4.8 Entity relationship diagram, part 2: accounts, security, packages and content 46

Figure 4.9 Class and module diagram 49

Figure 4.10 Sequence diagram: customer login 51

Figure 4.11 Sequence diagram: search and filter the car listing 52

Figure 4.12 Sequence diagram: price quotation through the JSON API 53

Figure 4.13 Sequence diagram: trip-based recommendation 54

Figure 4.14 Sequence diagram: booking request, confirmation and invoice 55

Figure 4.15 Activity diagram: booking workflow 56

Figure 4.16 Activity diagram: price forecast recomputation 58

Figure 4.17 Home page on desktop (left) and mobile (right) 61

Figure 4.18 Car page with the price calendar on desktop (left) and mobile (right) 61

Figure 4.19 Find My Car recommendations on desktop (left) and mobile (right) 62

Figure 4.20 Dark mode of the home page (left) and the car page on mobile (right) ... 62

Figure 5.1 Code excerpt: the booking is stored with the price calculated on the server (vehical-details.php) ... 67

Figure 5.2 Printable invoice showing the stored price and the price of each day ... 67

Figure 5.3 Smart Pricing page with the model health and a preview of the next days 69

Figure 5.4 AI Insights with the suggested price for each car ... 70

Figure 5.5 Code excerpt: pricing one car for one day (ml/pricing_rules.py) ... 72

Figure 5.6 Smart price of the Toyota Fortuner for the next 60 days (forecast of 27 September 2026) ... 73

Figure 5.7 Code excerpt: replacing the forecast in one transaction (ml/batch_forecast.py) ... 74

Figure 5.8 Actual and predicted daily demand on the test period ... 76

Figure 5.9 Comparison of the price models on the test set ... 77

Figure 5.10 Code excerpt: scoring and explaining the budget and road criteria (includes/recommendation_engine.php) ... 80

Figure 6.1 Data transferred per page by the original template and by Rento (desktop) ... 90

ABBREVIATIONS AND ACRONYMS

AI	Artificial Intelligence
API	Application Programming Interface
APA	American Psychological Association (citation style)
BIT	Bachelor of Information Technology
BS	Bikram Sambat (the official calendar of Nepal)
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CV	Cross-Validation
DB	Database
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
EV	Electric Vehicle
FK	Foreign Key
FR	Functional Requirement
FYP	Final Year Project
GB	Gradient Boosting
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
JS	JavaScript
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LR	Linear Regression
MAE	Mean Absolute Error
MAMP	macOS, Apache, MySQL, PHP (local server package)
ML	Machine Learning
MLP	Multilayer Perceptron
NFR	Non-Functional Requirement
NPR	Nepalese Rupee (shown as “Rs”)
OWASP	Open Worldwide Application Security Project
PDO	PHP Data Objects
PK	Primary Key
R^2	Coefficient of determination
RF	Random Forest
RMSE	Root Mean Squared Error
SEO	Search Engine Optimisation

SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
UML	Unified Modeling Language
URL	Uniform Resource Locator
WCAG	Web Content Accessibility Guidelines
XSS	Cross-Site Scripting

EXECUTIVE SUMMARY

Car rental businesses in Nepal usually quote one fixed daily price, even though demand rises sharply around Dashain, Tihar, weekends and last-minute trips, and most bookings are still arranged by phone or social media. Customers also find it hard to know which vehicle suits a trip to, for example, the mountain roads of Mustang. This project developed Rento, a web-based car rental platform for Nepal with explainable AI-based pricing and trip-based vehicle recommendation.

Rento was built with PHP 8.3, MySQL 8, Bootstrap 5.3 and Python 3.12, using a phased, iterative method on top of an open-source car-rental template that was almost entirely rebuilt. Its intelligent components are a rule-based smart pricer that combines seven demand factors with a −15% / +25% cap and a human-readable reason for every price; a 60-day price forecast recomputed nightly and after every relevant change; a content-based recommendation engine that scores cars against a natural-language trip description; and offline machine-learning experiments that prepare a learned pricer for the time when enough bookings exist.

Evaluation showed that all 480 forecast prices respected the caps and rounding rules, and that the recommendation engine reached a normalised precision@3 of 0.94 and a hit rate of 0.91 on twelve test trips. Among the models compared, a Random Forest explained 49% of daily demand variance on an external rental dataset and 98% of price variance on 1,640 scraped Nepali listings. Fifty-four PHPUnit and forty-six pytest tests passed, and page weight fell by up to 80% compared with the original template. Because Rento has only four real bookings, the live site uses the rule-based pricer; the learned models are ready to be switched on once about 200 bookings exist.

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Vehicle rental is a practical necessity in Nepal. Much of the country is reached only by road, public transport between cities is limited, and domestic travel peaks sharply around the national festivals of Dashain and Tihar, when large numbers of people travel from Kathmandu to their home districts. Visitors, families and businesses therefore rely on hired cars, jeeps and vans for trips to destinations such as Pokhara, Chitwan, Lumbini and Mustang, as well as for airport transfers and daily use within the Kathmandu valley.

A market survey carried out for this project in September 2026 (documented in the project repository) shows how the local rental market currently operates. The largest online marketplace, Sajilo Rental, lists more than a thousand vehicles, almost all rented with a driver, at prices that are nearly flat by vehicle category: about Rs 8,000 per day for hatchbacks, sedans and the Mahindra Scorpio, and about Rs 10,000 for vans (Sajilo Rental, n.d.). Only one operator surveyed, Self Drive Nepal, published a self-drive rate card, with rates of about Rs 5,000 per day for hatchbacks and sedans and Rs 7,000 to Rs 10,000 for sport utility vehicles (Self Drive Nepal, 2025). Most other operators handle bookings through enquiry forms, telephone calls and messaging applications such as WhatsApp and Viber, and few publish fixed prices for tours or packages. Where seasonal surcharges exist, they are usually stated informally; one operator, for example, notes that prices rise by 10–15% in the October–November peak season (Nepal Vehicle Hiring, 2026).

In other industries with fixed capacity and perishable inventory, such as airlines, hotels and international car rental, prices have long been set dynamically according to expected demand. This practice, known as revenue management or yield management, aims to sell the right capacity to the right customer at the right price (Weatherford & Bodily, 1992; Talluri & van Ryzin, 2004). Car rental companies were among its early adopters: National Car Rental reported substantial revenue gains after introducing a demand-based pricing and fleet system (Geraghty & Johnson, 1997), and Hertz applied similar techniques to rental rates and fleet allocation (Carroll & Grimes, 1995). More recently, online platforms have combined such pricing with

machine-learning models of demand, as in the pricing tips offered to Airbnb hosts (Ye et al., 2018).

A second development is the use of recommender systems to help customers choose among many options. Recommender systems filter a catalogue according to a user's stated needs or past behaviour and present the most suitable items with an explanation (Ricci et al., 2015). For car rental, the relevant needs are practical: the number of travellers, the budget, the type of road and the purpose of the trip. A customer planning a family trip to Pokhara, for instance, needs a comfortable car with at least five seats, whereas a trip on the rough mountain roads of Mustang calls for a rugged four-wheel-drive vehicle.

This project, Rento, brings these ideas to the Nepali rental context. Rento is a web-based car rental platform for self-drive cars, cars with a driver and tour packages, with all prices in Nepalese rupees. It sets a price for every car on every day from explainable demand rules, shows customers why a price is higher or lower, and recommends vehicles for a trip described in plain language. The system was developed from an existing open-source PHP car-rental template, which was rebuilt almost entirely during the project; Chapter 3 describes this starting point and the work carried out.

1.2 Problem Statement

Car rental businesses in Nepal usually charge a fixed daily price that does not reflect changes in demand. The same car costs the same during Dashain and Tihar, on weekends and for last-minute bookings as on a quiet weekday, even though demand in these periods is much higher. Businesses either lose revenue at peak times or apply informal surcharges that customers cannot anticipate. Customers, in turn, find prices unclear, because the published price often excludes fuel, driver costs and peak-season charges that are only explained over the phone.

Customers also find it difficult to choose the right vehicle for their trip. Rental websites typically offer a list of cars with basic filters, but they do not help a customer decide whether a compact car is suitable for a family of five, or whether a particular sport utility vehicle can handle the road to Mustang. The choice is left to the customer or to a telephone conversation with the operator.

Finally, bookings are largely handled manually over telephone and social media. This makes availability uncertain, prevents customers from seeing and comparing prices in advance, and leaves no reliable record of the price that was agreed.

The problem addressed by this project is therefore the lack of an online rental platform for Nepal that sets demand-aware prices transparently, recommends suitable vehicles for a given trip, and records bookings at a price calculated and stored by the system. Rento addresses this with an online booking platform that combines explainable, AI-based pricing with trip-based vehicle recommendation.

1.3 Objectives

1.3.1 General Objective

The general objective of this project is to design, develop and evaluate a secure web-based car rental platform for Nepal that prices vehicles dynamically according to expected demand, explains every price to the customer, and recommends suitable vehicles for a trip.

1.3.2 Specific Objectives

The specific objectives are:

1. To develop a responsive web application for customers and administrators with a PHP backend, a MySQL database and a JSON application programming interface (API), covering vehicle listing, booking, invoices, packages and administration.
2. To design and implement a smart pricing engine that adjusts each car's daily price for weekends, festivals and festival travel days, travel seasons, booking lead time, occupancy and recent demand, within configurable limits, and that gives a human-readable reason for every adjustment.
3. To build a forecast pipeline that computes and stores the price of every active car for the next 60 days, recomputed automatically every night and after every relevant change.
4. To implement a recommendation engine that interprets a natural-language trip description and ranks the available cars with a match score and reasons.

5. To train and evaluate machine-learning models for demand and price prediction, and to compare them with baselines using standard regression metrics, in preparation for a learned pricer.
6. To apply recognised web security practices, including password hashing, protection against cross-site request forgery (CSRF) and injection, rate limiting and server-side price calculation.
7. To test the system through unit, integration and system tests, and to evaluate its performance, accessibility and responsiveness on desktop and mobile screens.

1.4 Scope of the Study

The scope of the study covers the features, users and environment summarised in Table 1.1.

Table 1.1 Scope of the project

Area	In scope	Out of scope
Users	Guests, registered customers and one or more administrators	Vehicle owners listing their own cars (marketplace model)
Services	Self-drive cars, cars rented only with a driver, and tour or transfer packages in Nepal	Other countries; buses and goods transport
Customer features	Search and filters, car page with a price calendar, trip-based recommendation, booking request, My Bookings and printable invoice, package enquiries, testimonials, FAQs, contact	Online payment, mobile application, live chat assistant
Administration	Bookings, vehicles, brands, packages and enquiries, festivals, smart-pricing settings, AI Insights, content, email and system checks	Accounting, payroll and fleet maintenance
Intelligence	Rule-based dynamic pricing, 60-day forecast, content-based recommendation, offline evaluation of machine-learning models	A learned pricer in live use (planned once enough bookings exist)
Prices and data	Nepalese rupees; festival calendar for BS 2083 and provisional BS 2084	Currency conversion; automatic import of official holiday lists
Deployment	Local server (MAMP: Apache, MySQL and PHP on macOS) with a deployment guide for real hosting	Public cloud deployment

1.5 Significance of the Study

For customers, Rento makes rental prices transparent. The price of every day is shown in advance with its reasons, such as a weekend or a Dashain travel day, and the total quoted when a booking is requested is stored and appears unchanged on the invoice. The recommendation feature helps customers choose a suitable vehicle without needing detailed knowledge of the fleet.

For rental businesses, the system shows how demand-based pricing can be introduced in a controlled and explainable way. Prices follow the festival calendar and booking patterns automatically, but the adjustments are limited by a configurable maximum discount and surcharge, and the administrator can review suggestions, apply them or set manual prices. Online booking requests with stored prices reduce the manual work of telephone bookings and the risk of disputes over the agreed price.

Academically, the project applies revenue-management and recommender-system concepts to a small business in a developing market, where historical booking data is scarce. It demonstrates a practical approach to this "cold start": a transparent rule-based pricer is used from the first day, while machine-learning models are trained and evaluated on external data so that they can replace or complement the rules once enough bookings exist.

1.6 Limitations

The study has the following limitations:

- **Limited booking data.** The system holds only four real bookings, two of them confirmed. The machine-learning models could not be trained on Rento's own data and were evaluated on external datasets; the occupancy and demand pricing factors remain switched off until 50 confirmed bookings exist, and the learned pricer until about 200.
- **External datasets.** The demand dataset is an external hourly rental dataset and not Nepali booking data, and the price dataset consists of publicly listed prices collected by scraping one marketplace. Results from these datasets indicate how the models behave but cannot be transferred directly to Rento.
- **Market benchmarks.** Only one published self-drive rate card was found, so the market price used for comparison is based on a single source.

- **Festival calendar.** Nepali festivals follow the lunar calendar and move every year. The dates must be checked each year; the BS 2084 dates in the system are provisional until the government publishes its official holiday list.
- **Deployment.** The system runs on a local server only. A deployment guide and an automated production check were prepared, but the system has not been deployed publicly.
- **Features not implemented.** There is no online payment gateway, mobile application or conversational AI assistant.
- **Template origin and AI assistance.** The system was built on an open-source template, and an AI coding assistant was used during development, as stated in the Declaration of Authenticity and in Chapter 3.

1.7 Organization of the Report

This report is organised into seven chapters. Chapter 1 has introduced the background, problem, objectives, scope, significance and limitations of the project. Chapter 2 reviews related work on dynamic pricing, demand forecasting, recommender systems and existing rental platforms, and identifies the research gap. Chapter 3 describes the development methodology, the software requirements specification, the tools and technologies, the feasibility study and the system architecture. Chapter 4 presents the system design through use case, data flow, entity relationship, class, sequence and activity diagrams, together with the user interface design. Chapter 5 explains the implementation of the modules, the application programming interface, the database and the algorithms, including the pricing engine, the forecast pipeline, the machine-learning experiments and the recommendation engine. Chapter 6 reports the test plan, test cases and results of unit, integration and system testing, and the performance evaluation. Chapter 7 summarises the work, draws conclusions and suggests future enhancements. The references and appendices follow.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the literature and existing systems that inform the design of Rento. The review is organised around the three intelligent functions of the system. The first is dynamic pricing, which is studied mainly under the heading of revenue management, together with research on how customers perceive the fairness of variable prices. The second is demand forecasting, which provides the evidence on which demand-based prices can be set and which motivates the machine-learning experiments of this project. The third is recommender systems, which help customers choose a suitable item from a catalogue. The chapter then examines the rental platforms currently operating in Nepal, using the market survey carried out for this project, and compares them with Rento.

Section 2.2 reviews related works in each of these areas and summarises them in Table 2.1 and Table 2.2. Section 2.3 presents the theoretical framework of the study: the rule-based pricing model, the ensemble learning methods and error metrics used in the experiments, and the weighted scoring model and precision measures used for recommendation. Section 2.4 identifies the research gap that Rento addresses, and Section 2.5 summarises the chapter.

2.2 Review of Related Works

2.2.1 Dynamic Pricing and Revenue Management

Revenue management, also called yield management, is the practice of controlling the price and availability of a fixed, perishable capacity in order to maximise revenue. Kimes (1989) identified the conditions under which it is most useful: capacity is relatively fixed, inventory perishes if it is not sold by a given date, demand fluctuates and can be forecast, customers can be segmented, and the marginal cost of a sale is low compared with the cost of adding capacity. A rental car meets all of these conditions. A day on which a car stands idle cannot be sold later, the size of the fleet cannot be changed quickly, and demand rises and falls with weekends, holidays and seasons.

Weatherford and Bodily (1992) proposed a taxonomy of what they called perishable-asset revenue management, which brings together overbooking, capacity allocation and pricing, and noted that the same principles apply to airlines, hotels and car rental. Talluri and van Ryzin (2004) later provided a comprehensive treatment of the theory and practice of the field. They distinguish quantity-based revenue management, in which the firm decides how much capacity to offer at each fixed price, from price-based revenue management, in which the price itself is changed over time in response to demand. Rento follows the second approach, because it changes the daily price of each car rather than the number of cars offered.

The theoretical basis of price-based revenue management was established by Gallego and van Ryzin (1994), who showed how the optimal price of a limited inventory depends on the remaining stock and the time left to sell it when demand is stochastic. Their model shows why prices should normally rise as the remaining capacity shrinks close to the sale date, and why a small number of fixed price levels can perform nearly as well as a continuously changing price. Den Boer (2015) reviewed the later literature on dynamic pricing and learning, in which the seller does not know the demand function in advance and must learn it from observed sales while setting prices. This problem is directly relevant to a new platform such as Rento, which has almost no sales history from which to estimate demand.

Car rental has a long history in this field. Carroll and Grimes (1995) described how Hertz developed its rate and fleet management over several decades, from simple rules towards systems that forecast demand and plan fleet movement between locations. Geraghty and Johnson (1997) reported that the revenue management system introduced at National Car Rental, which combined demand forecasting, pricing and capacity control, made a significant contribution to the company's recovery in the 1990s. Oliveira, Carravilla and Oliveira (2017) reviewed the literature on fleet and revenue management in car rental. They noted that car rental differs from airline and hotel revenue management because rental durations vary, cars can be moved between locations and the fleet can be adjusted over the medium term, and they proposed an integrated framework that links pricing with fleet decisions. These studies show that demand-based pricing is established practice among large car rental companies, but they concern firms with extensive booking histories and dedicated analytics teams, which a small Nepali operator does not have.

Online marketplaces have more recently applied machine learning to pricing. Ye et al. (2018) described the model used by Airbnb to suggest nightly prices to its hosts. The model estimates the probability that a listing will be booked on each night at different prices and combines this with a customised regression that suggests a price for every future date. The approach depends on a very large volume of booking data, and the suggested price is shown to the host, who can accept or override it. Rento adopts a similar division of responsibility: the system suggests and explains prices, while the administrator can apply or override them.

Dynamic pricing also raises questions of fairness. Kahneman, Knetsch and Thaler (1986) found that customers regard price increases as unfair when they appear to exploit a rise in demand rather than to reflect a rise in cost. Xia, Monroe and Cox (2004) developed a framework in which perceived fairness depends on comparisons with past prices, with prices paid by other customers and with prices charged by competitors, and on whether the reasons for a price difference are understood. Haws and Bearden (2006) showed experimentally that customers judge price differences between buyers more harshly than differences over time, and that satisfaction falls when a customer learns that another customer paid less. These findings suggest that variable prices are better accepted when they are limited, predictable and explained. Rento therefore caps every adjustment, shows the price of each future day in advance and gives a plain reason, such as a weekend or a Dashain travel day, for every change.

2.2.2 Demand Forecasting

Pricing according to demand requires an estimate of future demand. Hyndman and Athanasopoulos (2021) present the main forecasting methods for time series, from simple benchmarks to exponential smoothing and regression models. They stress that every forecasting method should be compared with simple benchmarks. One such benchmark is the seasonal naïve method, which predicts that a value will equal the value observed at the same point of the previous season, for example the same hour one week earlier. A model that cannot outperform such a benchmark adds little value. The experiments in this project follow this advice by comparing every model with a seasonal or median baseline.

Tree-based ensemble methods are widely used for demand prediction from tabular data. Breiman (2001) introduced the random forest, which averages the predictions of

many decision trees, each trained on a bootstrap sample of the data and a random subset of the features, and showed that it reduces the variance of single trees while remaining robust to noise. Friedman (2001) developed gradient boosting, which builds an additive model by fitting each new tree to the errors of the model built so far. Chen and Guestrin (2016) described XGBoost, a scalable implementation of gradient boosting that became popular in data science competitions. Grinsztajn, Oyallon and Varoquaux (2022) compared tree-based models with deep neural networks on a range of tabular datasets and found that tree-based models still performed better on typical medium-sized tabular data, partly because they are less sensitive to uninformative features. This supports the choice of random forest and gradient boosting as the main candidates for Rento's learned pricer.

Makridakis, Spiliotis and Assimakopoulos (2018) compared statistical and machine-learning forecasting methods on a large set of monthly time series and found that the machine-learning methods were often less accurate than simple statistical methods, while also being more costly to compute. They argued that machine-learning methods should be evaluated carefully against statistical benchmarks before being adopted. Their findings are consistent with the experience of this project, in which the more complex models gained only modestly over the seasonal baseline on hourly demand, as reported in Chapter 5.

The choice of evaluation method also matters. Bergmeir and Benítez (2012) studied cross-validation for time series and noted that standard random cross-validation can give optimistic results when observations are dependent in time, whereas a blocked or time-ordered split preserves the order of the data. The experiments in this project therefore use a chronological train and test split for the demand data. For error measurement, Hyndman and Koehler (2006) reviewed the common measures of forecast accuracy and discussed their weaknesses, including the instability of percentage errors when actual values are close to zero. Willmott and Matsuura (2005) argued that the mean absolute error is a more natural measure of average error than the root mean squared error, whereas Chai and Draxler (2014) argued that the root mean squared error is more appropriate when errors are expected to follow a normal distribution and large errors should be penalised more heavily. Because the two measures answer different questions, this project reports both, together with the coefficient of determination.

2.2.3 Recommender Systems

Recommender systems suggest items that are likely to be useful to a particular user. Ricci, Rokach and Shapira (2015) describe the main families of recommender systems: collaborative filtering, which recommends items that similar users have liked; content-based filtering, which recommends items whose attributes match the user's preferences; knowledge-based systems, which reason explicitly about how item attributes meet the user's stated requirements; and hybrid systems that combine these approaches. Burke (2002) surveyed hybrid recommender systems and noted that knowledge-based methods are especially suitable when items are purchased rarely and each purchase depends on specific requirements, as in the choice of a car or a house, because there is little rating history to learn from.

Collaborative filtering has been very successful where large amounts of interaction data are available. Koren, Bell and Volinsky (2009) showed how matrix factorisation models, which represent users and items by latent factors learned from ratings, achieved leading accuracy in the Netflix Prize competition. However, collaborative methods suffer from the cold-start problem: they cannot recommend well for new users or new items that have no ratings. Schein, Popescul, Ungar and Pennock (2002) examined this problem and showed that methods which use item content perform better when interaction data are missing. A small rental platform with a fleet of eight active cars and a handful of bookings is an extreme cold-start case, so a collaborative approach is not feasible for Rento.

Content-based methods avoid this limitation by matching item attributes with the user's profile. Lops, de Gemmis and Semeraro (2011) reviewed the state of the art in content-based recommendation and identified its advantages: it does not depend on other users, it can recommend new items immediately and its recommendations can be explained by pointing to the matching attributes. Its main weakness is that its quality depends on how well the attributes describe the items. In Rento, the relevant attributes of a car, such as its type, seats, fuel, price and suitability for rough roads, are recorded in the database, and the customer's requirements are taken from a plain-language description of the trip. The approach therefore combines content-based matching with knowledge-based rules about which vehicle types suit city, highway and mountain roads.

Explanation is an important quality of a recommender system. Tintarev and Masthoff (2015) identified the purposes that explanations serve, including transparency, trust, persuasiveness and helping users to make good decisions, and noted that an explanation should reflect the real reasoning of the system. Rento shows a match percentage and a list of reasons, such as "7 seats for your 6 travellers" or "Rugged, off-road capable: built for mountain and rough roads", derived directly from the criteria that contributed to the score.

For evaluation, Herlocker, Konstan, Terveen and Riedl (2004) reviewed the measures used for recommender systems and distinguished accuracy of predicted ratings from the quality of the ranked list shown to users. For a system that shows a short list of items, measures such as precision among the top-ranked items are appropriate, because the user will usually consider only the first few suggestions. Rento's recommendation engine is evaluated in this way in Chapter 6, using precision at three and the rate at which the first suggestion is relevant.

2.2.4 Existing Rental Platforms

A market survey of Nepali car rental websites was carried out for this project on 27 September 2026. Its findings are recorded in the project repository and summarised here.

Sajilo Rental is the largest online marketplace found. It lists vehicles from many owners, all of which are rented with a driver, and offers instant booking with online payment through eSewa, Khalti and cards (Sajilo Rental, n.d.). Its daily prices are nearly flat within each vehicle category, at about Rs 8,000 for hatchbacks, sedans and the Mahindra Scorpio and about Rs 10,000 for vans, and the customer pays for fuel separately. No change of price with the date, the season or demand was observed.

Self Drive Nepal was the only operator found that published a self-drive rate card (Self Drive Nepal, 2025). Its prices vary by vehicle category and by whether the car stays inside the Kathmandu valley, ranging from about Rs 5,000 per day for hatchbacks and sedans to Rs 10,000 for automatic, luxury and electric sport utility vehicles, with fuel, tolls and parking excluded and a large security deposit required.

Rent in Nepal offers electric cars and vans with a driver at fixed daily rates, with fixed prices for popular routes and a 25% surcharge for night travel (Rent In Nepal, n.d.). Nepal Vehicle Hiring publishes all-inclusive price ranges with a driver and states that

prices rise by 10–15% in the October–November peak season (Nepal Vehicle Hiring, 2026). Kathmandu Car Services lists fares by destination, with surcharges for hill and mountain roads and discounts for long-term hire (Kathmandu Car Services, 2026). Most of these operators take bookings through enquiry forms, telephone and messaging applications such as WhatsApp and Viber, and few publish fixed prices for tours and packages.

Among global systems, the car rental companies Hertz and National Car Rental used demand-based pricing supported by forecasting systems (Carroll & Grimes, 1995; Geraghty & Johnson, 1997), and Airbnb suggests a price for each night to its hosts from a learned model (Ye et al., 2018). These systems price dynamically but do so with large volumes of data that a new Nepali platform does not have. Finally, the open-source "Car Rental Portal" template on which Rento was built provides vehicle listing, search, booking requests at a fixed price per day, testimonials and an administration panel [INSERT template citation]. It has no dynamic pricing, recommendation, invoice or package features.

Table 2.1 summarises the main related works reviewed in this section, and Table 2.2 compares the features of the surveyed platforms with those of Rento.

Table 2.1 Summary of related works

Work	Area	Approach	Main finding	Relevance to Rento
Kimes (1989)	Revenue management	Conceptual analysis	Yield management suits fixed, perishable capacity with variable, forecastable demand	Car rental meets these conditions
Gallego and van Ryzin (1994)	Dynamic pricing	Stochastic optimisation model	Optimal price depends on remaining stock and time; few price levels are nearly optimal	Justifies bounded, stepwise price changes
Geraghty and Johnson (1997)	Car rental	Industry case study	Forecasting and pricing system raised revenue at National Car Rental	Evidence that demand pricing works in car rental
Oliveira et al. (2017)	Car rental	Literature review and framework	Pricing and fleet decisions should be integrated	Positions Rento's pricing within car rental research

Work	Area	Approach	Main finding	Relevance to Rento
Ye et al. (2018)	Accommodation marketplace	Booking-probability and regression model	Learned daily price suggestions for hosts at scale	Suggest-and-override design; needs large data
Xia et al. (2004); Haws and Bearden (2006)	Price fairness	Framework; experiments	Unexplained or inconsistent prices are seen as unfair	Caps, advance calendar and reasons for each price
Breiman (2001); Friedman (2001)	Machine learning	Random forest; gradient boosting	Tree ensembles reduce error of single models	Models compared in the ML experiments
Grinsztajn et al. (2022)	Machine learning	Benchmark on tabular data	Tree models outperform deep networks on typical tabular data	Supports tree ensembles over the MLP
Makridakis et al. (2018)	Forecasting	Empirical comparison	ML methods often fail to beat statistical benchmarks	Every model is compared with a baseline
Schein et al. (2002)	Recommender systems	Cold-start methods	Content information helps when ratings are missing	Rento has almost no interaction data
Lops et al. (2011)	Recommender systems	Survey of content-based methods	Content-based systems are independent of other users and explainable	Basis of the recommendation engine
Tintarev and Masthoff (2015)	Recommender systems	Survey of explanations	Explanations build transparency and trust	Match percentage and reasons

Table 2.2 Feature comparison of existing platforms and Rento

Feature	Sajilo Rental	Self Drive Nepal	Rent in Nepal	Nepal Vehicle Hiring	Kathmandu Car Services	Base template	Rento
Online booking	Yes, instant	–	Enquiry form, phone, chat apps	Form, chat apps	Yes	Booking request	Booking request
Published daily prices	Yes	Yes	Partly	Price ranges	Price ranges	Yes	Yes, for each day
Self-drive	No	Yes	No	No	No	Not specified	Yes
With driver	Yes	Yes	Yes	Yes	Yes	No	Yes
Prices change with date or demand	Not observed	Not observed	Fixed night surcharge	Informal peak surcharge	Road surcharges, long-hire discount	No	Yes, seven rules, capped
Reason shown for each price	No	No	No	No	No	No	Yes
Suggests cars for a trip	No	No	No	No	No	No	Yes
Tour or transfer packages	Hourly and day hire	–	Yes	Yes	Yes	No	Yes, by enquiry
Online payment	Yes	–	–	No	–	No	No
Stored quote and invoice	–	–	–	–	–	No	Yes

Sources: market survey of 27 September 2026 (see Section 2.2.4 and Appendix [INSERT appendix letter]). A dash (–) means that the feature was not described on the pages reviewed; "chat apps" refers to WhatsApp or Viber; "Not observed" means that the published prices showed no such variation.

2.3 Theoretical Framework

The theoretical framework of the study, shown in Figure 2.1, links the theories reviewed above with the inputs, components and outcomes of Rento. Revenue management and price fairness inform the rule-based smart pricer and its 60-day forecast; demand forecasting informs the offline machine-learning models; and content- and knowledge-based recommendation informs the recommendation engine. The following subsections present the formal models used in each component.

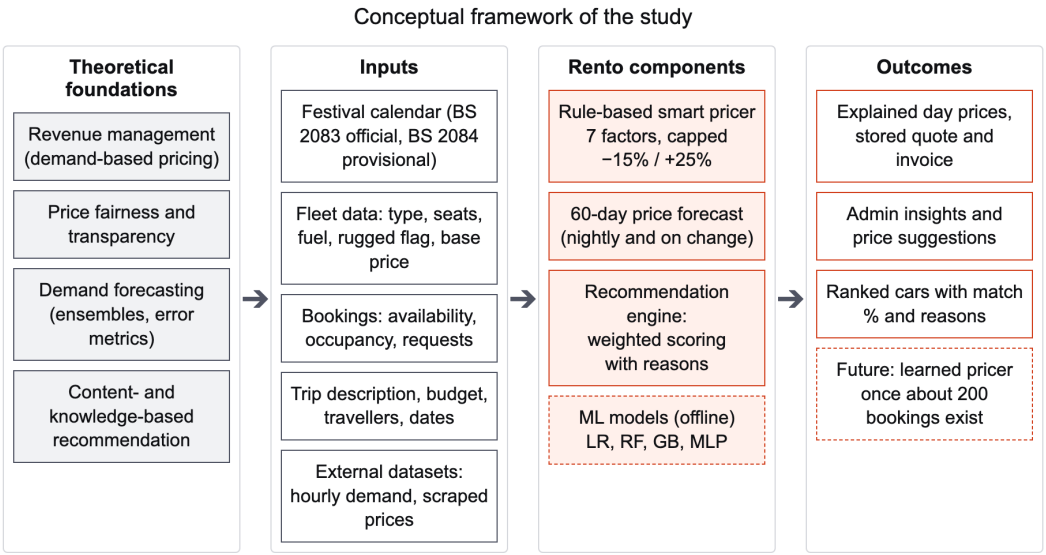


Figure 2.1 Conceptual framework of the study

2.3.1 Rule-Based Dynamic Pricing

A rule-based pricing model sets the price as a base price adjusted by a set of explicit factors, each triggered by a known condition. Unlike a learned model, it needs no historical sales data, and every adjustment can be stated to the customer, which addresses the fairness concerns discussed in Section 2.2.1. Its weakness is that the size of each adjustment is chosen by judgement rather than estimated from data.

In Rento, the price of a car on day d is calculated from its base daily price b and the percentages f of the factors that apply to that day, as shown in Equation 2.1.

$$p_d = R_s\left(b\left(1+\min\left(\max\left(\sum_{i\in A_d} f_i,\ -\delta^-\right),\ \delta^+\right)\right)\right) \tag{2.1}$$

where A is the set of factors active on day d, δ^- and δ^+ are the maximum discount and surcharge, and R rounds the result to the nearest multiple of s rupees. The factors are additive, so a Friday that is also a festival travel day receives the sum of both surcharges. Only the strongest festival factor applies on any day, and the last-minute and early-booking factors are mutually exclusive. With the settings used in this project, the caps are 15% and 25% and prices are rounded to Rs 50. The cap reflects the finding of Gallego and van Ryzin (1994) that a limited range of prices can capture most of the benefit of dynamic pricing, and it keeps the variation predictable for customers. The factors and their values are described in Chapter 5.

2.3.2 Ensemble Learning Methods

A regression model estimates a target value y, such as hourly demand or a daily rental price, from a vector of features x. The ensemble methods used in this project combine many decision trees.

Gradient boosting (Friedman, 2001) builds the model in M stages. It starts from a constant and, at each stage m, fits a regression tree to the negative gradient of the loss function L, called the pseudo-residual, as shown in Equation 2.2.

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F = F_{m-1}} \tag{2.2}$$

The new tree is then added to the model with a learning rate ν between 0 and 1, which shrinks the contribution of each tree and reduces overfitting, as shown in Equation 2.3.

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x) \tag{2.3}$$

For the squared-error loss, the pseudo-residual is simply the difference between the actual value and the current prediction, so each tree learns to correct the errors of the trees before it.

A random forest (Breiman, 2001) instead trains B trees independently, each on a bootstrap sample of the training data with a random subset of features considered at each split, and averages their predictions, as shown in Equation 2.4.

$$\hat{y}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x) \tag{2.4}$$

Averaging many decorrelated trees reduces the variance of the prediction. Boosting mainly reduces bias, whereas a random forest mainly reduces variance; which works better depends on the size and noise of the dataset, so both are evaluated in this project.

2.3.3 Regression Evaluation Metrics

For a test set of n observations with actual values y and predictions $\hat{y}$, the mean absolute error (MAE) measures the average size of the errors in the units of the target, as shown in Equation 2.5.

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.5}$$

The root mean squared error (RMSE) squares the errors before averaging, so it gives more weight to large errors, as shown in Equation 2.6.

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2.6}$$

The coefficient of determination (R^2) compares the squared error of the model with that of a model that always predicts the mean of the actual values, as shown in Equation 2.7.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{2.7}$$

An R^2 of 1 indicates perfect prediction and an R^2 of 0 indicates no improvement on the mean; a negative value means that the model is worse than predicting the mean. Lower MAE and RMSE and higher R^2 indicate a better model. Because error on a single split may depend on how the data happened to be divided, the experiments also use cross-validation, with folds that respect time order for demand data (Bergmeir & Benítez, 2012) and folds grouped by car model for price data, so that the models are tested on cars they have not seen.

2.3.4 Content-Based Recommendation Scoring

A content- and knowledge-based recommender can be expressed as a weighted scoring model. For a trip query q, a set of criteria $K(q)$ applies, such as the number of seats, the budget, the type of road and the preferred fuel. Each criterion k has a weight w and gives a car c a partial score s between 0 and 1, which measures how well the car meets that criterion. The match score of the car is the share of the applicable weights that it earns, as shown in Equation 2.8.

$$S(c, q) = \frac{\sum_{k \in K(q)} w_k \, s_k(c, q)}{\sum_{k \in K(q)} w_k} \tag{2.8}$$

Because only criteria that apply to the trip are counted, a car scores 100% only when it fully meets every requirement the customer has expressed, and the score of a vague

query is not diluted by criteria that the customer did not mention. The partial scores also provide the reasons shown to the customer, in line with the principle that an explanation should reflect the real reasoning of the system (Tintarev & Masthoff, 2015). The criteria and weights used by Rento are presented in Chapter 5.

The quality of the ranked list is measured by precision at k, the proportion of the top k recommended cars that belong to the set G of cars judged relevant for the trip (Herlocker et al., 2004). When fewer than k cars are relevant, even a perfect ranking cannot reach a precision of 1, so a normalised precision is also used, which divides by the smaller of k and the number of relevant cars, as shown in Equation 2.9.

$$P@k = \frac{|R_k \cap G|}{k}, \qquad nP@k = \frac{|R_k \cap G|}{\min(k, |G|)} \tag{2.9}$$

where R is the set of the top k cars returned. A related measure, the hit rate at one, records whether the first recommended car is relevant.

2.4 Research Gap

The review shows that demand-based pricing is well established in the international car rental, airline and hotel industries (Geraghty & Johnson, 1997; Talluri & van Ryzin, 2004) and that online platforms such as Airbnb use learned models to suggest prices (Ye et al., 2018). These approaches, however, rely on large amounts of booking data and dedicated systems. The research on dynamic pricing and learning (den Boer, 2015) and on cold-start recommendation (Schein et al., 2002) shows that a new platform with little data needs a different approach, but it gives little practical guidance on how a small operator in a developing market can introduce demand-based pricing from the first day.

In the Nepali rental market, the survey found no platform that changes prices systematically by date or demand. Prices are either flat by vehicle category or subject to informal surcharges, such as a peak-season increase stated only as a range, and none of the platforms reviewed explains the price of a particular day or recommends vehicles for a described trip. Festival demand around Dashain and Tihar, which drives much of the variation in rental demand in Nepal, is not reflected in any published price. Research on price fairness (Xia et al., 2004) suggests that variable prices would need to be transparent and bounded to be accepted in such a market.

The gap addressed by this project is therefore the absence of a rental platform for Nepal that combines transparent, bounded, day-by-day dynamic pricing based on the Nepali festival calendar and booking conditions with explainable, trip-based vehicle recommendation, and that is designed to operate without historical data while preparing for a learned pricer once data become available. Rento addresses this gap with a rule-based pricer that works from the first day, a content-based recommendation engine that needs no rating history, and offline machine-learning experiments that establish which models should replace the rules when enough bookings exist.

2.5 Summary

This chapter reviewed the literature on revenue management and dynamic pricing, price fairness, demand forecasting and recommender systems, together with the rental platforms operating in Nepal. Revenue management research establishes that car rental is well suited to demand-based pricing, and fairness research indicates that such prices should be bounded and explained. Forecasting research supports tree-based ensemble models for tabular demand data while warning that they must be compared with simple baselines and evaluated with time-aware methods. Recommender systems research shows that content- and knowledge-based methods are appropriate when interaction data are scarce, and that explanations increase transparency and trust. The survey of Nepali platforms found flat or informally adjusted prices, booking by enquiry, and no price explanation or trip-based recommendation.

The theoretical framework defined the capped, rule-based pricing model, the gradient boosting and random forest methods, the MAE, RMSE and R^2 metrics, and the weighted scoring model and precision measures used for recommendation. The research gap identified is the lack of a transparent, data-independent dynamic pricing and recommendation platform for the Nepali rental market. Chapter 3 describes the methodology used to develop Rento in response to this gap.

CHAPTER 3: METHODOLOGY

3.1 Introduction

This chapter describes how Rento was developed. Section 3.2 explains the development methodology, the starting point of the project, the use of an artificial-intelligence assistant and the project timeline recorded in the version control history. Section 3.3 describes the methods used to collect data and requirements. Section 3.4 presents the software requirements specification, Section 3.5 lists the tools and technologies used, and Section 3.6 reports the feasibility study. Section 3.7 describes the overall architecture of the system, and Section 3.8 summarises the chapter.

3.2 Development Methodology

3.2.1 Choice of Methodology

The project followed a phased, iterative and incremental development model. In incremental development, a system is built as a series of versions, each of which adds functionality to the previous one, and the specification, development and validation activities are interleaved rather than carried out once in strict sequence (Sommerville, 2016). Larman and Basili (2003) traced the long history of iterative and incremental development and noted that it reduces risk by producing working software early and by allowing requirements to be refined through feedback.

This model was chosen for three reasons. First, the project started from existing software rather than from a blank page, so the work consisted of a sequence of improvements to a system that had to keep working after each step. A linear waterfall model, in which all requirements are fixed before any design or coding, would have suited this situation poorly. Second, some requirements could only be settled after earlier work had been completed. For example, the prices used by the pricing engine depended on the market survey, and the festival pricing depended on the festival calendar research. Third, the intelligent components carried the greatest technical uncertainty, so they were built only after the platform had been secured and stabilised, which allowed them to be tested on a reliable base.

The work was divided into eight phases, shown in Figure 3.1 and summarised in Table 3.1. Each phase had a defined goal and followed the same cycle: the phase was

planned and its scope agreed; the database was backed up and a working branch created; the changes were built and committed in small steps; automated and manual tests were run; and a phase report was reviewed by the student before the next phase began. Faults found in later phases were handled as follow-up fixes rather than by reopening earlier phases.

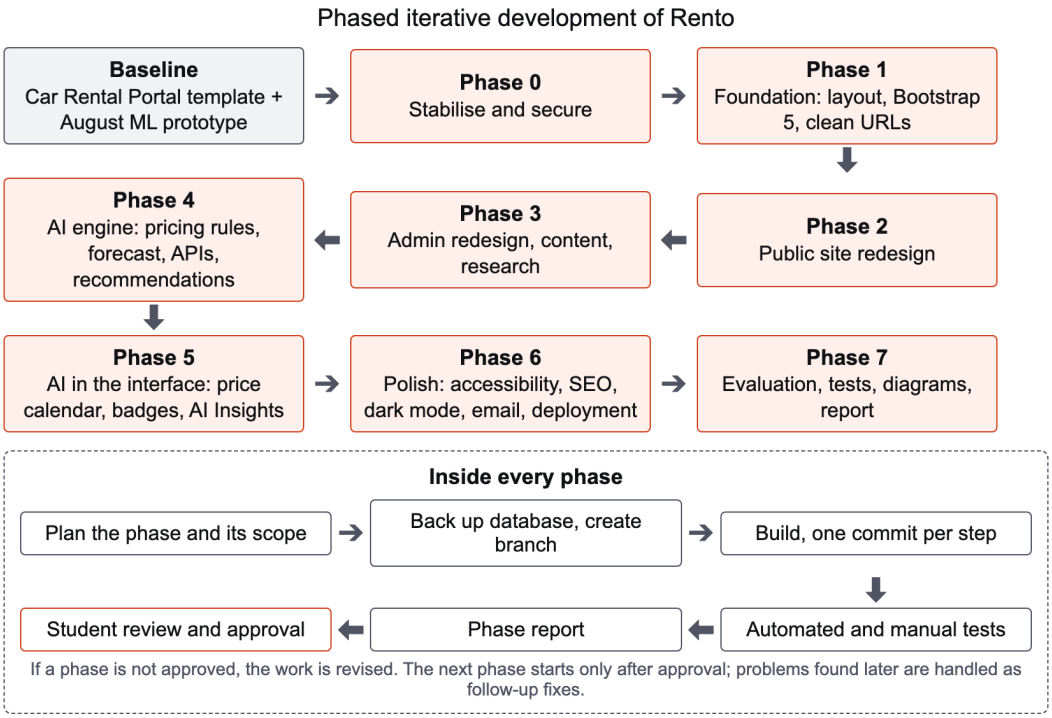


Figure 3.1 Phased iterative development methodology

Table 3.1 Development phases and their main outputs

Phase	Goal	Main outputs
0 Stabilise and secure	Make the inherited system safe and working	Password hashing, CSRF tokens, secure password reset, protected uploads, rate limiting, separate admin and customer sessions, bug fixes
1 Foundation	Prepare a maintainable code base	Shared layout files, Bootstrap 5.3 and Font Awesome 6, prices in rupees, removal of unused code and images, clean URLs
2 Public redesign	Rebuild the customer-facing site	New design system, home page, car listing with filters, car page, Find My Car, account area, WCAG AA contrast
3 Admin redesign and content	Rebuild administration and add content	New admin area, editable site content, active and inactive vehicles, market research data, festival calendar, packages and enquiries
4 AI engine	Build the intelligent back end	Rule-based pricer, 60-day forecast, JSON API, stored booking prices, rebuilt recommendation engine
5 AI in the interface	Show the intelligence to users	Price calendar, card prices and badges, invoice, AI Insights, price overrides, festival travel days
6 Polish	Prepare for real use	Rugged flag, accessibility, search engine optimisation, dark mode, email, legal drafts, deployment guide, production check
7 Evaluation and documentation	Measure and document the system	Machine-learning experiments, test suites, performance and page-weight measurements, diagrams, screenshots, this report

3.2.2 Starting Point of the Project

Rento was not developed from nothing. Its starting point, recorded as the first commit of the project repository ("baseline before refactor", commit 9250954), consisted of two parts.

The first part was an open-source PHP car rental template named "Car Rental Portal" [INSERT template citation]. The template provided a small car rental website with vehicle listing, brand search, booking requests at a fixed price per day, testimonials, contact and newsletter forms, a set of content pages and an administration panel for

vehicles, brands, bookings and users. It used ten database tables, Bootstrap 3 and PHP pages that mixed database queries with HTML.

The second part was an earlier machine-learning prototype developed by the student in August 2026, before the repository was created. According to the file timestamps, the scraper and the scraped Sajilo Rental dataset date from 27 August 2026 and the price model files from 29 August 2026. The prototype comprised a web scraper for public vehicle listings, data preprocessing scripts, a PyTorch neural network for price prediction, a gradient boosting demand model, and PHP pages that attempted to use these models for dynamic pricing and car recommendation. At the start of Phase 0, the dynamic pricing integration always fell back to the base price because the model call failed, and the recommendation results page ended with a fatal error; both faults were corrected in Phase 0 before the components were later rebuilt.

The inherited code also had serious security weaknesses. Passwords were stored as MD5 hashes, which are unsuitable for password storage; no form was protected against cross-site request forgery; and the password reset form allowed anyone who knew an account's email address and mobile number to set a new password. These weaknesses correspond to categories of the OWASP Top 10, notably broken access control, cryptographic failures and identification and authentication failures (OWASP Foundation, 2021), and they were the reason why Phase 0 was dedicated to security.

Table 3.2 compares the baseline with the system at the end of the project. The comparison is based on the code at commit 9250954 and on measurements reported in Chapter 6. It shows that the template served as a starting structure, but that the security layer, the user interfaces, most of the database, and all of the pricing, forecasting, recommendation, testing and documentation work were rebuilt or newly added during the project.

Table 3.2 Comparison of the baseline template and the final system

Aspect	Baseline (template and prototype)	Rento at the end of the project
Front-end libraries	Bootstrap 3.3.7 (public site) and 3.3.6 (admin), Font Awesome 4.4.0, jQuery 1.10.2	Bootstrap 5.3.8, Font Awesome 6.7.2, jQuery 3.7.1, own design tokens, dark mode
Currency	US dollar sign ($)	Nepalese rupees (Rs) throughout
Password storage	MD5 hash	PHP password_hash (bcrypt); old MD5 hashes are upgraded at the next login

Aspect	Baseline (template and prototype)	Rento at the end of the project
Password reset	New password set by anyone who knows the email address and mobile number	Time-limited, single-use reset link, emailed to the account when email is configured
Form protection	No CSRF protection; admin actions by GET links	CSRF token on every POST form; admin actions by POST only
Login protection	No rate limiting; one shared session	Rate-limited login and reset; separate admin and customer sessions
URLs	File names with .php	Clean URLs and readable car URLs; old URLs redirected
Pricing	Fixed price per day; prototype model not working	Seven-factor rule-based pricer, 60-day forecast, price calendar, stored quote and invoice
Recommendation	Prototype page with a fatal error that also overwrote vehicle data	Read-only weighted scoring engine with match percentage, reasons and cautions
Administration	Bootstrap 3 admin theme with basic lists	New admin area with dashboard, Smart Pricing, AI Insights, festivals, packages, content and system check
Database	10 tables	23 tables (13 added by 22 additive migrations)
PHP files (excluding libraries)	56	122
Home page weight (desktop)	40 requests, 4,090 KB	23 requests, 831 KB
Automated tests	None	54 PHPUnit tests and 46 pytest tests

3.2.3 Use of Artificial-Intelligence Assistance

An artificial-intelligence assistant, Claude (Anthropic), was used throughout the development and documentation of the project as a coding and documentation assistant. The assistant was used to inspect the inherited code, propose changes, write and refactor source code, database migrations, automated tests, evaluation scripts and diagrams, run tests and measurements on the local server, and draft and edit parts of this report.

The student directed this work. The student defined the goals and scope of every phase and set rules that the assistant had to follow, including that database changes must be additive only and preceded by a backup, that existing URLs, file names and form field names must be kept unless a change was explicitly approved, that each logical change must be committed separately, that no credential may be changed without the student's permission, and that no numbers, features or references may be invented in the documentation. At the end of each phase the assistant produced a report listing every changed file, the test results and the open issues, and the student

reviewed and tested the result before approving the next phase. The student is responsible for the system and for the content of this report.

The use of the assistant is recorded openly in the version control history: every commit carries a "Co-Authored-By" line naming the assistant, except for one automatic merge commit (8bd13fa) created without the line; the history was not rewritten to add it. The assistant also explains the pace of development shown in Section 3.2.4. This use of AI tools is in line with the policy of Lincoln University College: [INSERT statement on the college's AI policy].

3.2.4 Version Control and Project Timeline

The project was managed with Git. The main branch holds the released state of the project, a dev branch collects completed work, and each larger piece of work in Phase 7 was developed on its own feature branch (for example feature/tests, feature/diagrams and feature/report) and merged into dev when complete. All commits were made locally and pushed to the remote repository by the student. The history has not been rewritten or back-dated.

Table 3.3 lists the activities of the project with their dates, and Figure 3.2 shows the recorded development phases as a Gantt chart generated directly from the commit times. The Git history begins with the baseline commit on 26 September 2026 and, up to the completion of Chapter 2 of this report (commit 9151b7b), contains 102 commits excluding merge commits. The rebuild of the system recorded in the repository therefore took place over about 27 hours on 26 and 27 September 2026. This short period reflects the use of the AI assistant described in Section 3.2.3; it does not include the earlier prototype work, which was not under version control, or the time spent on planning and reviewing between commits. The bars in Figure 3.2 run from the first to the last commit of each phase and show when work was committed rather than the total effort spent.

Table 3.3 Project timeline

Activity	Dates	Evidence
Project proposal and supervisor approval	[INSERT dates]	[INSERT]
Machine-learning prototype: scraping, preprocessing, price and demand models	27–29 August 2026	File timestamps of the prototype files
Phase 0 and Phase 1	26 September 2026	34 commits, 20:24–22:24
Phase 2 to Phase 6	27 September 2026	59 commits, 07:43–20:59
Phase 7: evaluation, tests, diagrams and screenshots	27 September 2026	6 commits, 21:18–22:32
Phase 7: writing of the report and slides	From 27 September 2026 to [INSERT submission date]	Commits on feature/report
Submission and presentation	[INSERT dates]	–

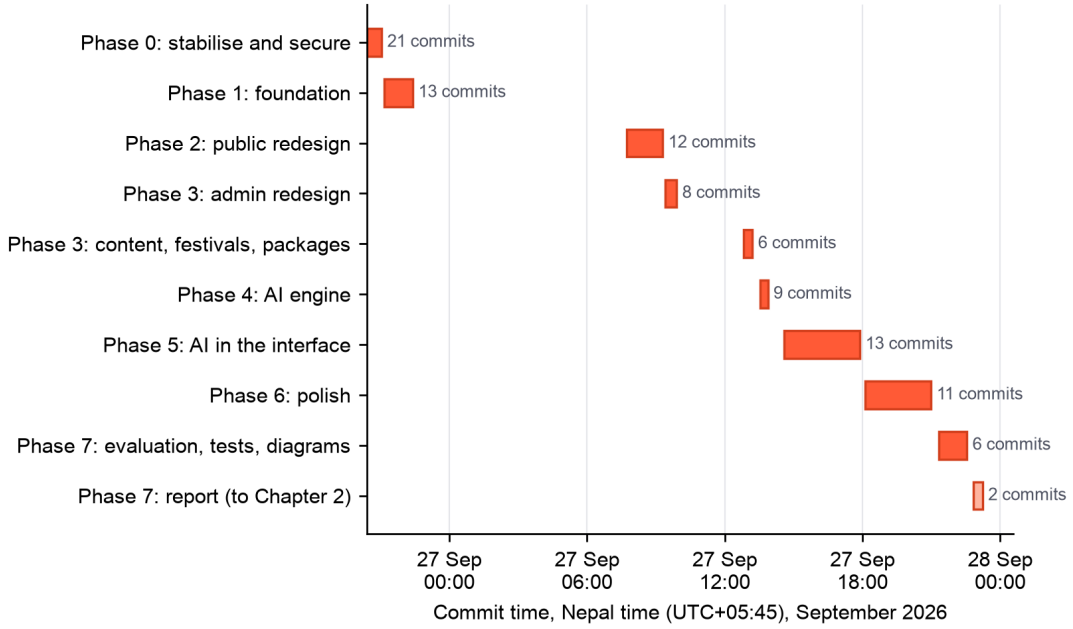


Figure 3.2 Gantt chart of the recorded development phases, generated from the Git commit history

3.3 Data and Requirements Collection

The requirements and data for the project were collected by the following methods.

- **Analysis of the existing system.** The inherited template and prototype were reviewed page by page and file by file to identify faults, security weaknesses and missing features. The findings formed the requirements of Phase 0 and Phase 1.
- **Market survey of existing platforms.** The websites of Nepali rental operators were reviewed on 27 September 2026 to record their services, prices, inclusions and booking methods, as summarised in Section 2.2.4. The survey was used to set the base prices of the active fleet at the published self-drive rates of Self Drive

Nepal (2025), to choose the vehicles and packages offered, and to define the comparison benchmarks shown on the car pages.

- **Festival calendar research.** The dates of the main Nepali festivals and public holidays for BS 2083 were taken from the official list published by the Ministry of Home Affairs (Ministry of Home Affairs, n.d.) and cross-checked with at least one other calendar. Because the official list for BS 2084 had not yet been published, the BS 2084 dates were taken from independent calendars and marked as provisional. Forty festival entries were recorded in the database.
- **Price dataset.** For the price experiment, 1,640 public vehicle listings were collected from the Sajilo Rental website with the scraper developed in the August prototype. The scraper collected only vehicle attributes and listed prices, such as the vehicle model, type, fuel, year and daily price, and waited one second between requests. No personal data were collected.
- **Demand dataset.** Because Rento has almost no booking history, the demand models were trained on an external hourly rental demand dataset, [INSERT demand dataset name and URL] ([INSERT citation]). The dataset contains 18,247 hourly observations and is used only to compare forecasting methods; it is not Rento booking data.
- **Literature and documentation.** Published research, reviewed in Chapter 2, and the documentation of the technologies used informed the design of the pricing, forecasting and recommendation components.

3.4 Software Requirements Specification

The requirements were specified following the general structure recommended in IEEE Std 830-1998 (IEEE, 1998), which distinguishes functional requirements, describing what the system must do, from non-functional requirements, describing the qualities with which it must do it.

3.4.1 Users of the System

The system has three classes of users. Guests can browse the fleet, view prices, use Find My Car, read content pages and send enquiries. Registered customers can additionally request bookings, view their bookings and invoices, and post testimonials. Administrators manage the fleet, bookings, packages, festivals, pricing

settings, content and system settings through a separate administration area with its own login.

3.4.2 Functional Requirements

The functional requirements of the system are listed in Table 3.4.

Table 3.4 Functional requirements

ID	Requirement	Description
FR1	Accounts	Customers can register, log in, log out, update their profile and change their password, and reset a forgotten password through a time-limited link. Administrators have a separate login.
FR2	Car search	Users can browse active cars and filter them by keyword, type, brand, fuel and maximum price, sort them and page through the results.
FR3	Car page	Each car has a page with its images, specifications, description and a price calendar that shows the price of each of the next 60 days with its reasons.
FR4	Price quote	For chosen dates, the system shows the total price, the price of each day, the reasons for each adjustment and whether the car is available.
FR5	Booking request	Logged-in customers can request a booking for valid dates. The server calculates the price and stores it with the booking, together with the breakdown by day.
FR6	My Bookings and invoice	Customers can see the status of their bookings and print an invoice that shows the stored price.
FR7	Find My Car	Users can describe a trip in plain language, with optional dates, budget and number of travellers, and receive a ranked list of suitable cars with a match percentage, reasons and cautions.
FR8	Packages and services	Users can view tour, transfer and other packages and send an enquiry; cars available only with a driver are booked through an enquiry.
FR9	Content and contact	The site provides About, FAQs, Terms and Privacy pages, testimonials, a contact form and a newsletter subscription.
FR10	Fleet and booking management	Administrators can add and edit vehicles and brands, set vehicles active or inactive, mark rugged and driver-only vehicles, manage images, and confirm or cancel bookings.
FR11	Content management	Administrators can edit content pages, draft legal pages and change the text and images of the public site.
FR12	Festival calendar	Administrators can add and edit festivals with their dates, demand level and travel days.
FR13	Smart pricing settings	Administrators can set the percentage of each pricing factor, the maximum discount and surcharge and the rounding, recompute the forecast and view the health of the pricing model.
FR14	AI Insights	Administrators can see price suggestions and a 60-day price chart for each car, apply suggestions and set manual price overrides.
FR15	Price forecast	The system computes and stores the price of every active car for the next 60 days every night and after every relevant change.
FR16	Application programming interface	The system provides JSON endpoints for price quotes, price calendars and recommendations.
FR17	Notifications	When email is configured, the system sends password reset links and confirmations of bookings and enquiries.

ID	Requirement	Description
FR18	Dashboard and system check	Administrators see summary statistics and a check of the production settings.

3.4.3 Non-Functional Requirements

The non-functional requirements are listed in Table 3.5. They were grouped using the product quality characteristics of ISO/IEC 25010 (International Organization for Standardization, 2011), and the method used to verify each requirement is given; the results of the verification are reported in Chapter 6.

Table 3.5 Non-functional requirements

ID	Quality	Requirement	Verification
NFR1	Security	Passwords are hashed with a modern algorithm; every state-changing form is protected against CSRF; all database queries use prepared statements; output is escaped; login and reset attempts are rate-limited; uploads cannot execute scripts; the application uses a least-privilege database user with secrets kept outside the repository.	Unit and HTTP security tests
NFR2	Correctness of prices	Prices are always calculated on the server, stay within the configured caps, are rounded as configured, and the price stored with a booking is the price shown on the invoice.	Pricing checks and integration tests
NFR3	Explainability	Every price adjustment and every recommendation is shown with a human-readable reason; the system never claims that a customer asked for something they did not ask for.	Recommendation scenarios
NFR4	Performance	Pages and API calls respond quickly on the local server, and pages transfer less data than the original template.	Timing and page-weight measurements
NFR5	Usability and accessibility	The interface meets WCAG 2 level AA colour contrast (World Wide Web Consortium, 2023), can be used with the keyboard, and respects the reduced-motion setting.	Automated accessibility checks and manual testing
NFR6	Compatibility	The site works on desktop and mobile screens from 390 pixels wide, in light and dark mode.	Screenshots and manual testing
NFR7	Reliability and data integrity	Database changes are additive and preceded by a backup; the forecast can be recomputed at any time; the site keeps showing base prices if the forecast is unavailable.	Migration review and tests
NFR8	Maintainability	Shared layout and logic files, one commit per change, documentation for developers and administrators, and automated test suites.	Code review
NFR9	Portability	The system runs on a standard Apache, PHP and MySQL stack, with configuration through environment variables and a deployment guide.	Production check
NFR10	Privacy	Only the personal data needed for accounts and bookings are stored, and draft privacy and terms pages are prepared for review.	Review of the database and pages

3.5 Tools and Technologies

Table 3.6 lists the tools and technologies used in the project, with the versions installed in the development environment.

Table 3.6 Tools and technologies

Category	Tool and version	Purpose
Local server	MAMP with Apache 2.4.62, PHP 8.3.14, MySQL 8.0.40	Web server, server-side language and database
Back end	PHP Data Objects (PDO), Composer 2.9.2, PHPMailer 6.12	Database access with prepared statements, dependency management, email
Front end	HTML5, CSS3, JavaScript, Bootstrap 5.3.8, Font Awesome 6.7.2, jQuery 3.7.1	User interface, layout and icons
Admin interface	DataTables 2.3.8, Chart.js 4.5.1, Quill 2.0.3	Tables, charts and the content editor
Intelligence	Python 3.12.14, pandas 3.0.6, NumPy 2.5.3, scikit-learn 1.9.1 (Pedregosa et al., 2011), PyMySQL 1.2.3	Pricing rules, forecast pipeline and machine-learning models
Development only	PyTorch 2.14.0, matplotlib 3.11.2, pytest 9.1.1, PHPUnit 12.5.36	Neural network experiment, charts and automated tests
Documentation	Mermaid, headless Google Chrome 153, Node.js 25.2.1, python-docx 1.2.0, Microsoft Word 16.113	Diagrams, screenshots, measurements and this report
Version control	Git 2.52.0, GitHub	Source control and remote repository ([INSERT GitHub link])
Development assistance	Claude (Anthropic); code editor [INSERT editor used]	Coding and documentation assistant, as described in Section 3.2.3
Hardware	Apple MacBook Air with M3 processor and 16 GB memory, macOS 26.5	Development and testing machine

3.6 Feasibility Study

3.6.1 Technical Feasibility

The project was technically feasible with widely used, well-documented technologies. PHP and MySQL run on almost every shared hosting service, and the Python components need only a standard Python installation with a few libraries. The heaviest computation in normal operation, the 60-day forecast for every active car, is a rule-based calculation that completes quickly on an ordinary laptop, so no special hardware is required. The machine-learning experiments were run offline on the same machine. The main technical risk, the lack of booking data for training a learned pricer, was addressed by designing a rule-based pricer that works without data and by keeping the learned models switched off until enough bookings exist.

3.6.2 Operational Feasibility

The system is designed to be operated by a small business without technical staff. Administrators manage the fleet, festivals, prices and content through web forms, and the forecast is recomputed automatically every night and after every change. Every automatic price is explained and can be overridden, so the operator stays in control of prices. For customers, the booking process uses familiar web forms, and the prices and reasons are shown before a booking is requested. The main operational task that remains manual is checking the festival dates each year when the government publishes the official holiday list.

3.6.3 Economic Feasibility

All software used to build and run the system is free and open-source, including PHP, MySQL, Apache, Python and its libraries, Bootstrap and the other front-end libraries. The development used an existing laptop, so no hardware was purchased. Microsoft Word was used only to produce this report [INSERT licence, e.g. student licence]. Running the system publicly would require web hosting and a domain name, at a cost of [INSERT estimated annual hosting and domain cost]. Because the system can run on ordinary shared hosting, these costs are small compared with the potential benefit of online bookings and demand-based pricing for a rental business.

3.6.4 Schedule Feasibility

The project was completed within the time available for the final year project. The phased approach allowed a working system to exist at the end of every phase, so that the project could have been stopped at any point with a usable result. As explained in Section 3.2.4, the use of an AI assistant shortened the implementation considerably, which left time for evaluation, testing and documentation.

3.6.5 Legal and Ethical Feasibility

The system stores only the personal data needed for accounts and bookings: names, email addresses, telephone numbers, the optional profile fields inherited from the template (date of birth and address) and booking details. Draft Terms and Privacy pages were prepared with reference to the Individual Privacy Act, 2075 (2018) and the Consumer Protection Act, 2075 (2018) of Nepal, and they are kept as drafts until they are reviewed, because they must be checked by a qualified person before

publication. The template on which the system is based is open-source and is credited in this report and in the project documentation. The price data used in the experiment were public listing prices collected at a slow rate, without personal data, and used only for research. Dynamic pricing raises questions of fairness, as discussed in Section 2.2.1; the system addresses them by capping every adjustment, showing prices in advance and explaining every change. The use of an AI assistant is disclosed in the Declaration of Authenticity, in Section 3.2.3 and in the commit history.

3.7 System Architecture

Rento follows a layered architecture, shown in Figure 3.3. The presentation layer consists of the public pages and the administration area, rendered in the browser with Bootstrap and a small amount of JavaScript for the price calendar, the trip search and the colour theme. The application layer runs on Apache and PHP: web server rules provide clean URLs and block access to internal folders, shared PHP modules implement security, pricing, price insights, recommendation, search engine optimisation and email, and a JSON application programming interface serves price quotes, price calendars and recommendations to the pages.

The intelligence layer is written in Python. The live pricer applies the pricing rules to every active car for the next 60 days and stores the results in the database. It is started by PHP after every relevant change and by the operating system scheduler every night. The machine-learning experiments run offline on external datasets and are not part of the live system. The data layer is a MySQL database with 23 tables, accessed by PHP through prepared statements and by Python through PyMySQL, using a database user with only the privileges the application needs.

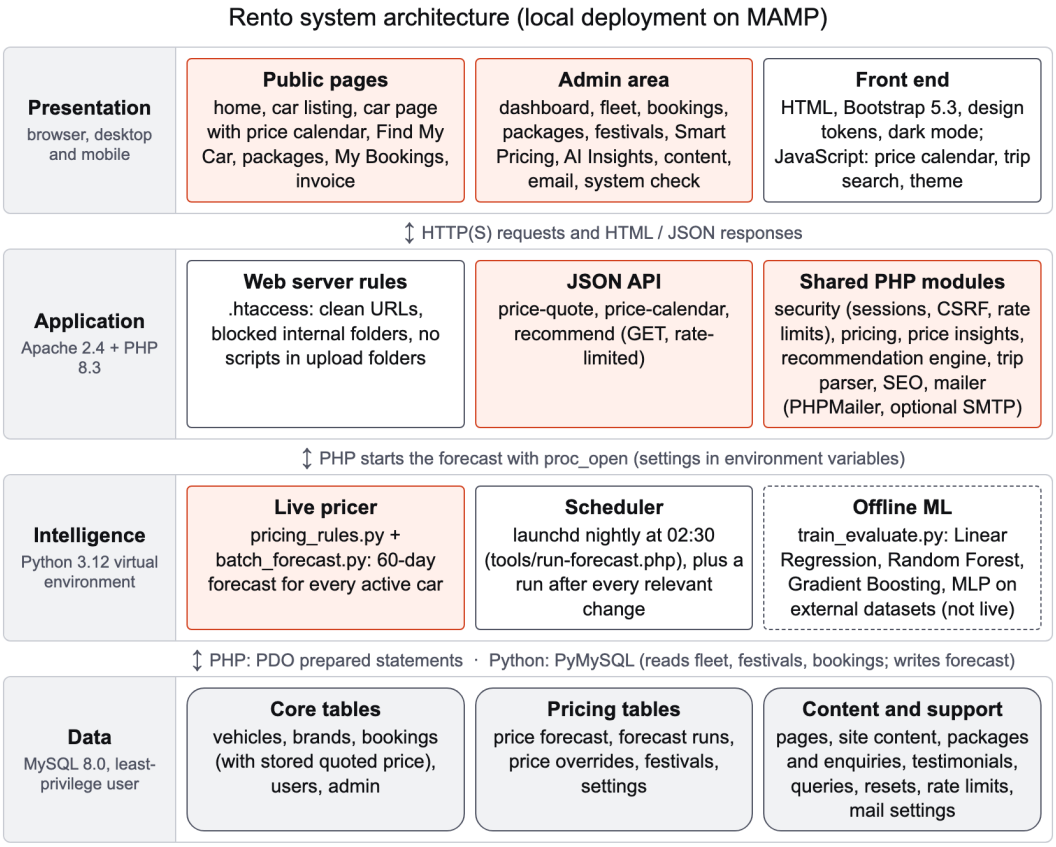


Figure 3.3 System architecture of Rento

This separation keeps the web pages fast, because they read prices that have already been calculated rather than running the pricing model on every request, and it allows the pricing engine to be replaced by a learned model in the future without changing the pages or the API. The detailed design of the system is presented in Chapter 4.

3.8 Summary

This chapter described the methodology used to develop Rento. The system was developed in eight phases following an iterative and incremental model, starting from an open-source car rental template and an earlier machine-learning prototype, both of which were largely rebuilt. An AI assistant was used as a coding and documentation assistant under the direction of the student, and its use is recorded in the commit history, which shows that the recorded rebuild took place over about 27 hours in September 2026. Requirements were collected through analysis of the existing system, a market survey, festival calendar research and external datasets, and were specified as eighteen functional and ten non-functional requirements. The project was found to be technically, operationally, economically, legally and ethically feasible.

Finally, the layered architecture of the system was presented. Chapter 4 describes the design of the system in detail.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter presents the design of Rento. It follows the structured and object-oriented notations commonly used to document information systems: a use case diagram for the functions offered to each user, data flow diagrams for the movement of data between processes and stores, entity relationship diagrams for the database, a class and module diagram for the structure of the code, sequence diagrams for the interactions behind the main functions, activity diagrams for the two most important workflows, and the design of the user interface. The diagrams use the Unified Modeling Language (UML) notation (Object Management Group, 2017) and the Gane and Sarson notation for data flow diagrams (Gane & Sarson, 1979). All diagrams describe the system as it was implemented; the names of tables, files and functions are those used in the source code.

4.2 Use Case Diagram

The use case diagram in Figure 4.1 shows the functions that Rento offers to its actors. A guest can browse and filter cars, view a car page with its price calendar, use Find My Car, register or log in, send an enquiry about a package or a car rented with a driver, and read the content pages. A registered customer can do everything a guest can do, which is shown by the generalisation between the two actors, and can also request bookings and view their bookings and invoices. The administrator manages bookings, the fleet, packages, content, festivals and pricing settings, and uses AI Insights to apply suggested prices or set manual overrides. Two external systems also interact with Rento: the operating system scheduler, which starts the nightly recomputation of the price forecast, and the mail server, which delivers notification emails when email is configured.

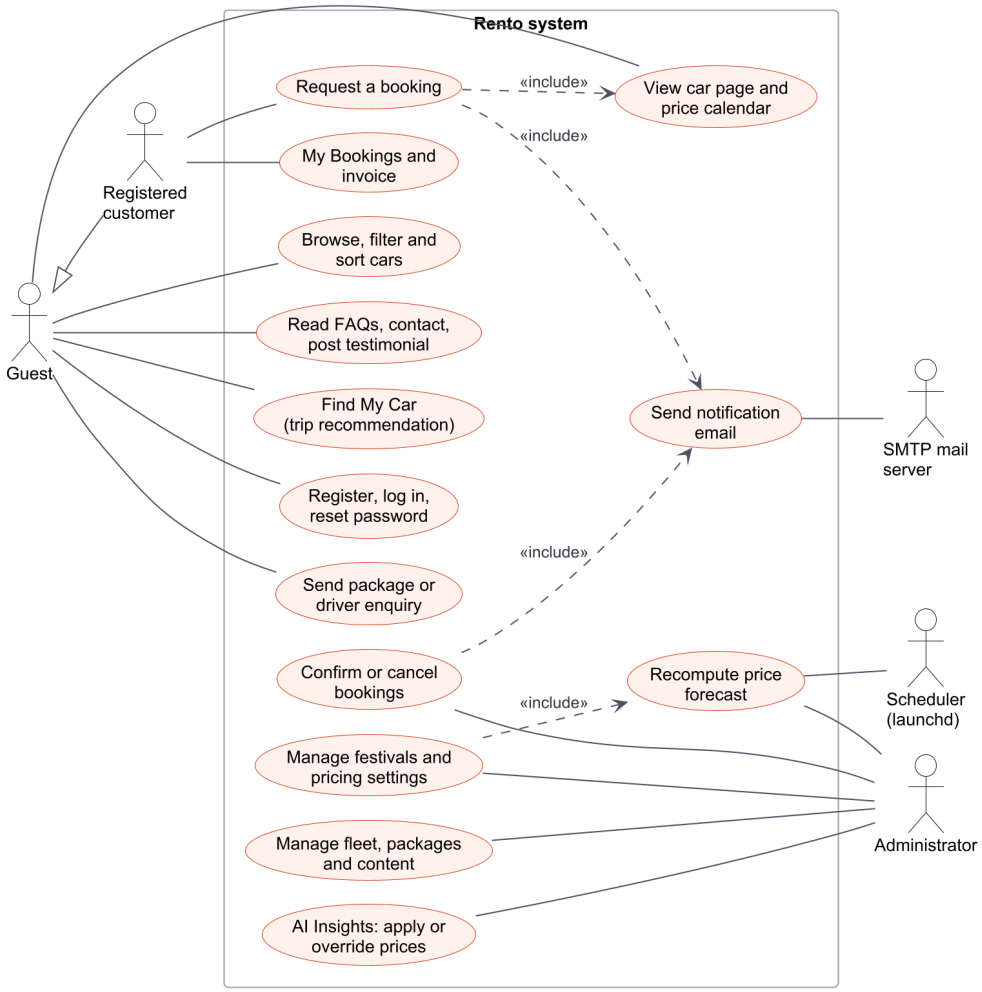


Figure 4.1 Use case diagram of Rento

The «include» relationships show functions that are always performed as part of another function. Requesting a booking always includes the price calendar of the car, because the dates are chosen on it, and a notification email; confirming or cancelling a booking includes a notification email; and changing festivals or pricing settings includes a recomputation of the forecast. Table 4.1 describes the most important use case, requesting a booking, in more detail.

Table 4.1 Use case description: request a booking

Item	Description
Actor	Registered customer
Precondition	The customer is logged in and the car is active and not driver-only
Main flow	1. The customer opens a car page and selects the pickup and return dates on the price calendar. 2. The system shows the price of each day, the reasons for each adjustment and the total. 3. The customer adds an optional message and sends the request. 4. The system checks the form token, the dates and the availability of the car. 5. The system calculates the price on the server from the forecast and stores the booking with the quoted price and the breakdown by day. 6. The system requests a forecast recomputation, sends a confirmation email if email is configured, and shows the booking in My Bookings.
Alternative flows	Invalid form token: the request is rejected. Invalid dates or car already booked: an error is shown and the form is kept. Customer not logged in: the login form is shown first.
Postcondition	A new booking with status "new" and a stored price exists; the administrator can confirm or cancel it

4.3 Data Flow Diagrams

Data flow diagrams describe how data moves through a system without showing the order of events (Gane & Sarson, 1979). In the diagrams that follow, squares are external entities, rounded boxes are processes with their number, and open-ended boxes are data stores with their identifier; stores marked with an asterisk are repeated in the same diagram to avoid crossing lines. Table 4.2 lists the data stores and the tables they contain.

Table 4.2 Data stores used in the data flow diagrams

Store	Tables	Contents
D1	tblusers, admin	Customer and administrator accounts
D2	tblvehicles, tblbrands	Fleet: vehicles with their attributes, base prices and flags, and brands
D3	tblbooking	Bookings with dates, status, quoted price and per-day details
D4	tbl_price_forecast, tbl_forecast_runs, tbl_price_overrides	60-day price forecast, forecast run log and manual price overrides
D5	tblfestivals, tblsettings	Festival calendar and changed pricing settings
D6	tblpackages, tblpackagecategories, tblpackageimages, tblpackageenquiries	Packages, their categories and photos, and enquiries
D7	tblpasswordresets, tblratelimits	Password reset tokens and rate-limit records
D8	tblpages, tblsitecontent, tblmailsettings	Content pages, editable site text and email settings

4.3.1 Context Diagram

The context diagram in Figure 4.2 shows Rento as a single process together with its four external entities. Customers send search criteria, trip descriptions, booking requests, enquiries and account details, and receive car lists, day prices with their reasons, recommendations, booking status and invoices. The administrator sends fleet, festival and pricing data, booking decisions and price overrides, and receives dashboard figures, AI Insights and the health of the pricing model. The scheduler sends the nightly recomputation trigger, and the system sends emails through the mail server.

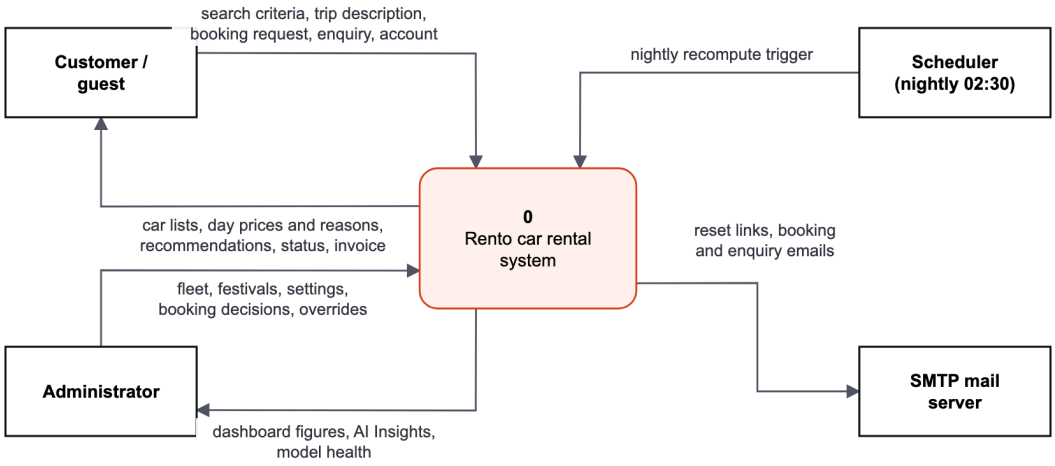


Figure 4.2 Context diagram (level 0 data flow diagram)

4.3.2 Level 1 Data Flow Diagrams

The level 1 diagram divides the system into nine processes. Because a single diagram with all nine processes would be difficult to read, it is shown in two parts. Figure 4.3 shows the processes used by customers: authentication and account management (1.0), browsing and searching (2.0), recommendation (3.0), price quotation (4.0) and booking and invoicing (5.0). The recommendation process reads the fleet, the forecast prices and the customer's rental history, and the quotation process reads the forecast prices and the existing bookings, so that the customer always sees prices and availability taken from the same data that the server uses when the booking is stored.

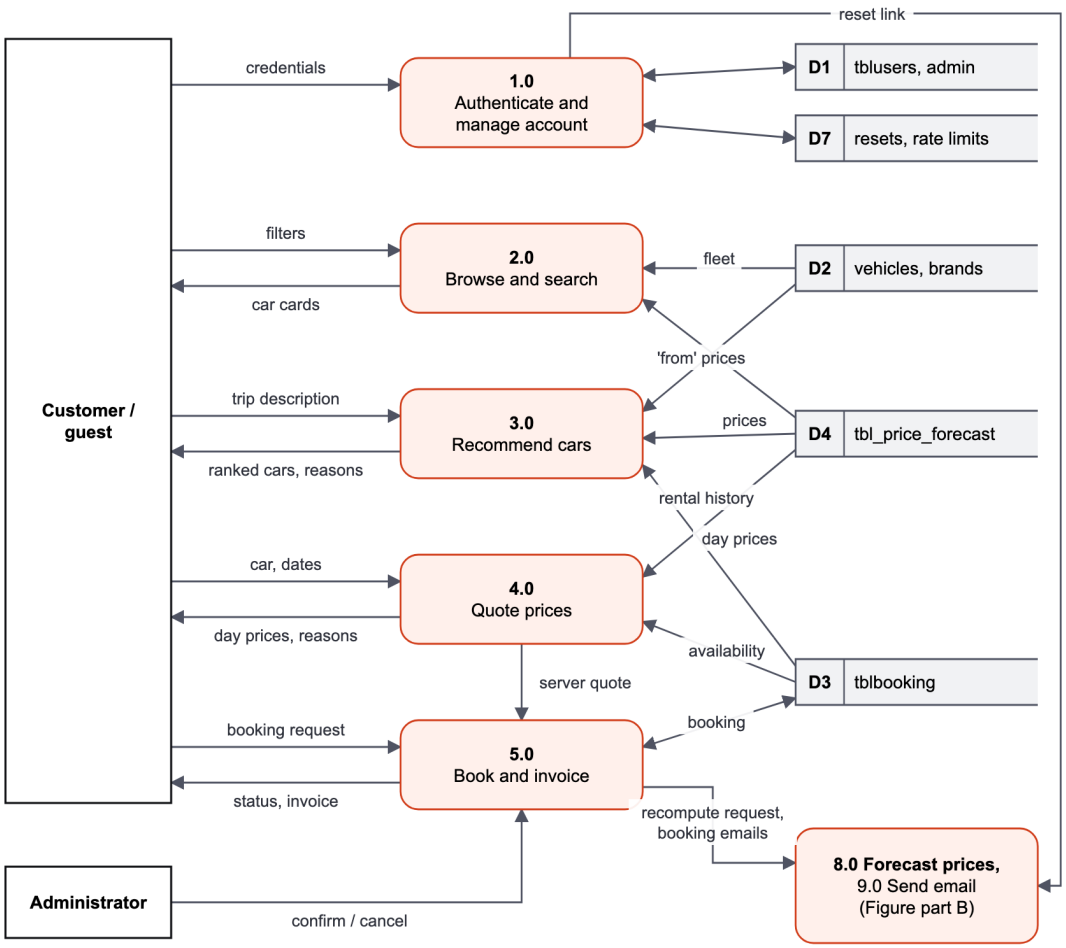


Figure 4.3 Level 1 data flow diagram, part A: customer processes

Figure 4.4 shows the administrative and background processes: packages and enquiries (6.0), administration of the fleet, content, festivals and pricing (7.0), the price forecast (8.0) and email (9.0). The forecast process is triggered by the administrator, by any change to bookings, festivals, settings, base prices or overrides, and by the nightly scheduler. It reads the fleet, the bookings and the festival and pricing settings, and writes the 60-day prices with their reasons. The administrator reads the forecast and the health of the pricing model through process 7.0, in line with the rule that data stores are accessed only through processes.

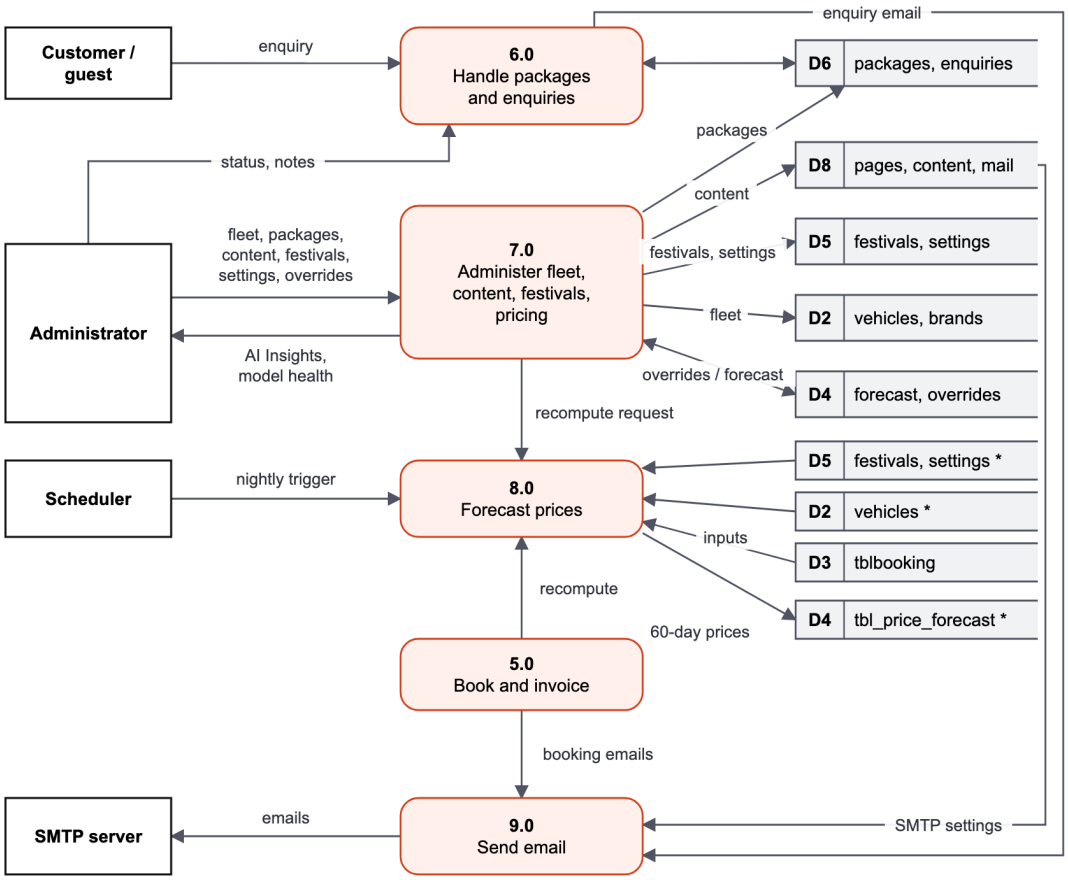


Figure 4.4 Level 1 data flow diagram, part B: administration and background processes

4.3.3 Level 2 Data Flow Diagrams

Figure 4.5 decomposes the booking process (5.0). A request passes through a check of the session and the form token (5.1), a validation of the dates, counted inclusively (5.2), and a check for overlapping bookings of the same car (5.3). The price is then calculated on the server from the forecast (5.4), so that any price sent by the browser is ignored, and the booking is stored with the quoted price and the breakdown by day (5.5). When the administrator confirms or cancels the booking (5.6), the system checks again that it does not clash with a confirmed booking. The invoice (5.7) shows the stored price rather than recalculating it, so the price the customer agreed to does not change later.

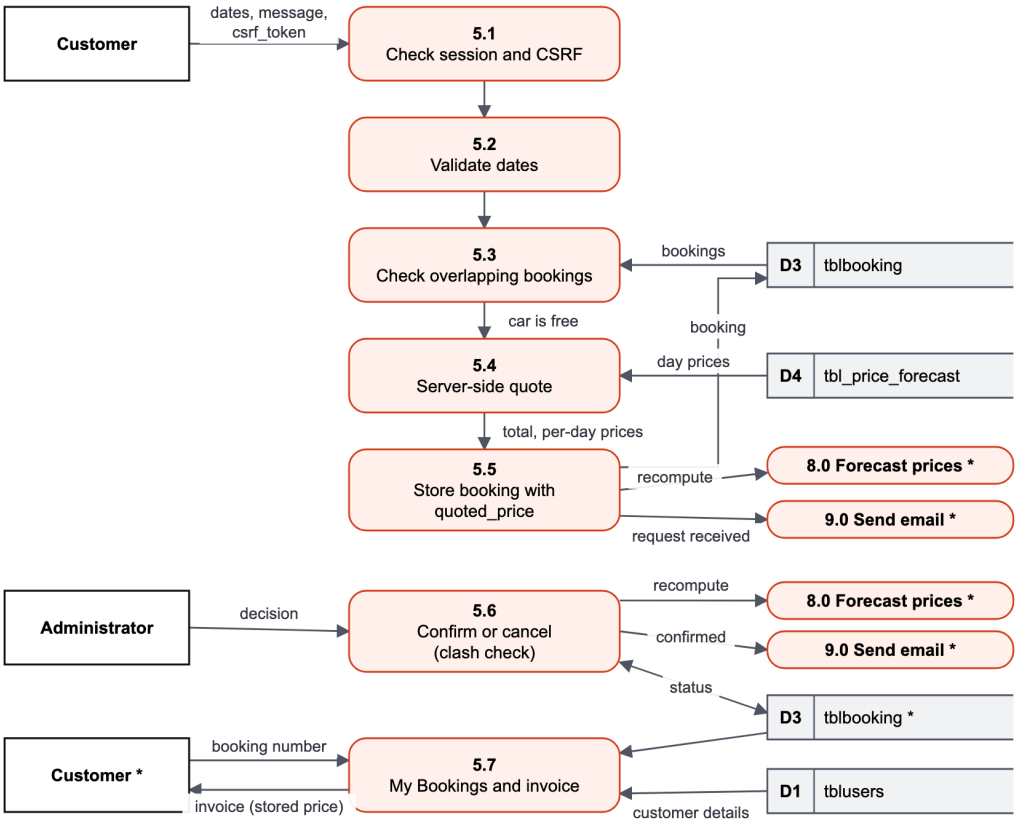


Figure 4.5 Level 2 data flow diagram of process 5.0 (book and invoice)

Figure 4.6 decomposes the forecast process (8.0). A trigger first queues a run (8.1). PHP then starts the Python pricer, passing the database settings and the pricing settings in environment variables rather than on the command line (8.2). The pricer loads the active cars, festivals, bookings and overrides (8.3), prices every car for every day of the next 60 days (8.4) and applies any manual overrides (8.5). It then replaces the whole forecast in a single database transaction, so that the website always reads either the old or the new forecast and never a partial one (8.6), and records the status of the run (8.7).

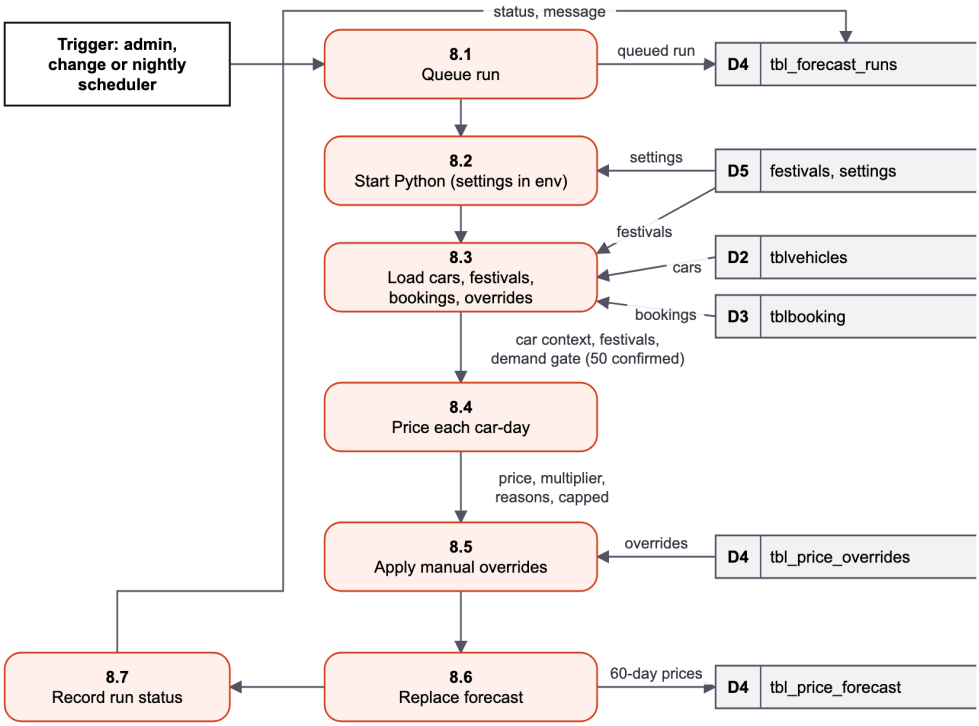


Figure 4.6 Level 2 data flow diagram of process 8.0 (forecast prices)

4.4 Database Design

4.4.1 Entity Relationship Diagrams

The database contains 23 tables with 200 columns. Ten tables come from the original template and 13 were added during the project through 22 additive migrations, which created new tables or added columns without renaming or removing existing ones. Because the diagram of all 23 tables would be too large for one page, the database is shown in two entity relationship diagrams. Figure 4.7 shows the core of the system: users, bookings, vehicles and brands, together with the pricing tables. Figure 4.8 shows accounts, password resets, packages and their enquiries, and lists the seven tables that have no relationships. For readability, each entity shows its keys and its most important columns; the number of further columns is given at the bottom of each entity, and the complete list of columns is provided in the data dictionary in the appendices.

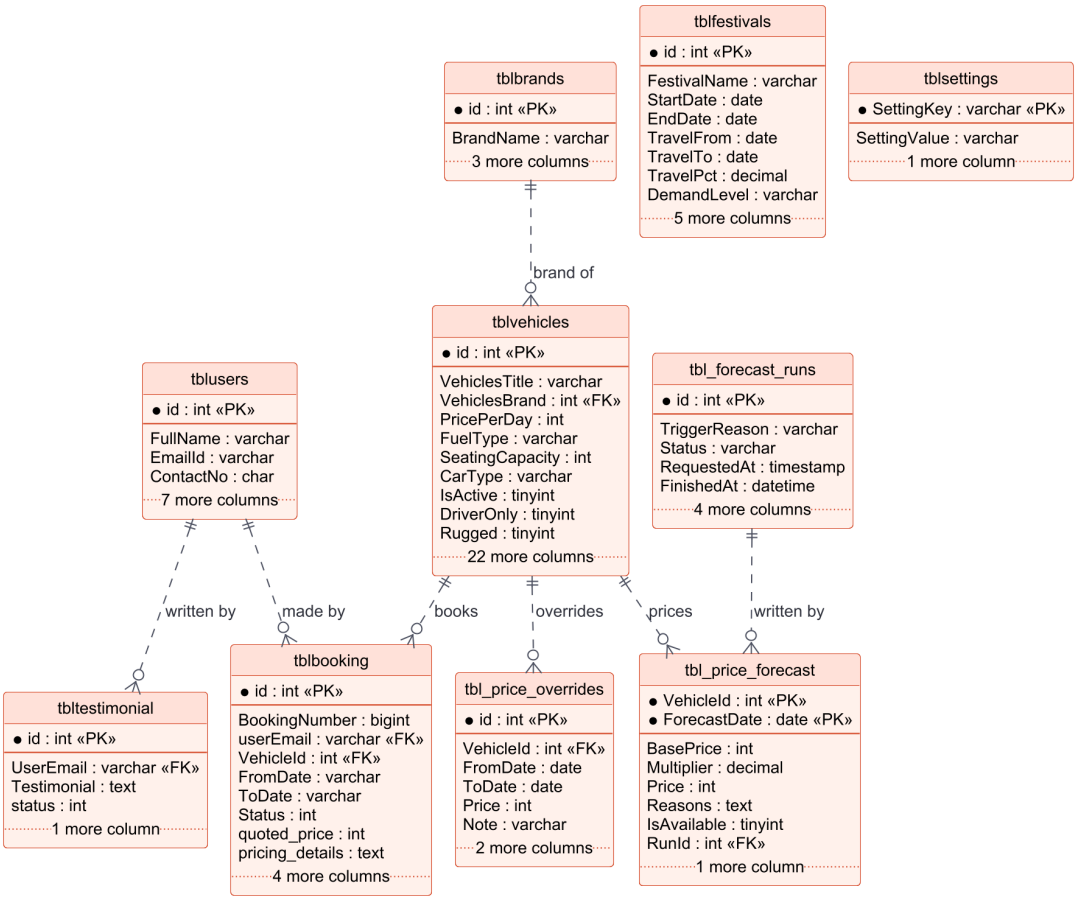


Figure 4.7 Entity relationship diagram, part 1: bookings, fleet and smart pricing

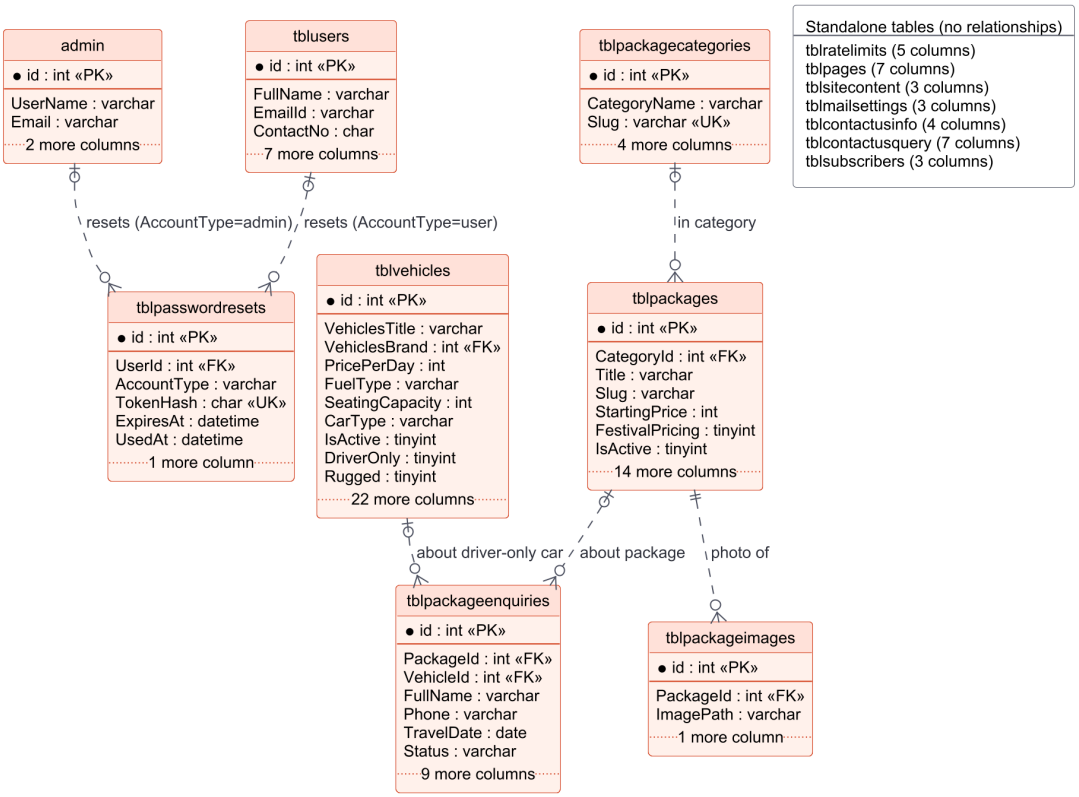


Figure 4.8 Entity relationship diagram, part 2: accounts, security, packages and content

4.4.2 Relationships and Keys

The database defines no foreign key constraints. The template did not use them, and the migrations did not add them, because adding constraints to existing tables with inconsistent legacy data could have failed or deleted data, which the rule of additive-only changes did not allow. The 13 relationships shown in the diagrams are therefore logical relationships: they are maintained by the PHP code, which reads the referenced record before writing a dependent one (for example, a booking is stored only for an existing, active car and a logged-in customer), and every query that uses them is a prepared statement. The diagrams draw these relationships with dashed lines to make this clear. Two relationships are conditional: a password reset refers either to a customer or to an administrator, depending on its AccountType column, and an enquiry refers either to a package or to a car that is rented only with a driver.

Most tables have an auto-increment integer primary key. The forecast table uses a composite primary key of the vehicle and the date, which guarantees that there is exactly one price for each car on each day, and the settings and mail settings tables use their key names as primary keys. Secondary indexes support the most frequent queries, such as the forecast by date, the overrides by vehicle and dates, the festivals

by dates, the packages by active flag and sort order, and the rate-limit records by action, email or IP address and time.

4.4.3 Design of the Main Tables

Table 4.3 shows the booking table. The columns quoted_price, pricing_details and priced_at were added during the project so that each booking stores the price calculated by the server at the moment of the request, together with the price and reasons for each day. The date columns FromDate and ToDate were inherited from the template as text columns; they were kept unchanged to respect the additive-only rule, and the code validates and normalises the dates before storing and comparing them.

Table 4.3 Data dictionary of tblbooking

Column	Type	Null	Description
id	int, auto-increment	No	Primary key
BookingNumber	bigint	Yes	Booking number shown to the customer
userEmail	varchar(100)	Yes	Customer (logical reference to tblusers.EmailId)
VehicleId	int	Yes	Car (logical reference to tblvehicles.id)
FromDate, ToDate	varchar(20)	Yes	Pickup and return dates (inherited text columns, validated in PHP)
message	varchar(255)	Yes	Optional message from the customer
Status	int	Yes	0 or empty: new; 1: confirmed; 2: cancelled
quoted_price	int	Yes	Total price in rupees calculated by the server (added)
pricing_details	text	Yes	Price and reasons for each day, as JSON (added)
priced_at	datetime	Yes	Time at which the price was calculated (added)
PostingDate	timestamp	No	Time of the request (default: current time)
LastUpdationDate	timestamp	Yes	Time of the last change (updated automatically)

Table 4.4 shows the forecast table, which is the central table of the smart pricing engine. It holds one row for each active car and each day of the forecast period; with eight active cars and 60 days, it contained 480 rows at the time of writing. Each row stores the base price used, the multiplier, the final price, the reasons in JSON and whether the car is available on that day, so that the website can show the price

calendar and the "from" price on the car cards without calculating anything when a page is requested.

Table 4.4 Data dictionary of tbl_price_forecast

Column	Type	Null	Description
VehicleId	int	No	Car; part of the composite primary key
ForecastDate	date	No	Day; part of the composite primary key (indexed)
BasePrice	int	No	Base daily price used for the calculation; a forecast for an older base price is ignored
Multiplier	decimal(6,4)	No	1 plus the capped sum of the factor percentages (default 1.0000)
Price	int	No	Final day price in rupees after capping, rounding and any override
Reasons	text	Yes	JSON list of the reasons and their percentages
IsAvailable	tinyint(1)	No	1 if the car is not booked on that day
RunId	int	Yes	Forecast run that wrote the row (logical reference to tbl_forecast_runs.id)
ComputedAt	timestamp	No	Time of calculation

The pricing settings are stored in a key and value table, tblsettings, which contains only the values that the administrator has changed; the default values, such as the maximum discount of 15% and the maximum surcharge of 25%, are defined in the code. This design allows new settings to be introduced without changing the database structure. The festival table stores, for each festival, its dates, its demand level and an optional travel window before the festival with its own percentage, together with the source of the dates and whether they have been verified.

4.5 Class and Module Diagram

Rento is written mainly in procedural PHP, organised in modules: files in the includes and api folders that each group the functions for one concern. Only the recommendation engine is implemented as a PHP class. Figure 4.9 therefore shows the modules as classes with the stereotype «module», together with the one PHP class and the Python modules and data classes of the pricer; the database connection passed to most functions is omitted from the signatures for readability. The dependencies show that the JSON API and the recommendation engine both obtain prices only through the pricing module, which in turn starts the Python batch forecast. The pricing rules themselves are therefore implemented in a single place, the

price_for_day function, and every part of the system that shows a price reads the result from the forecast table.

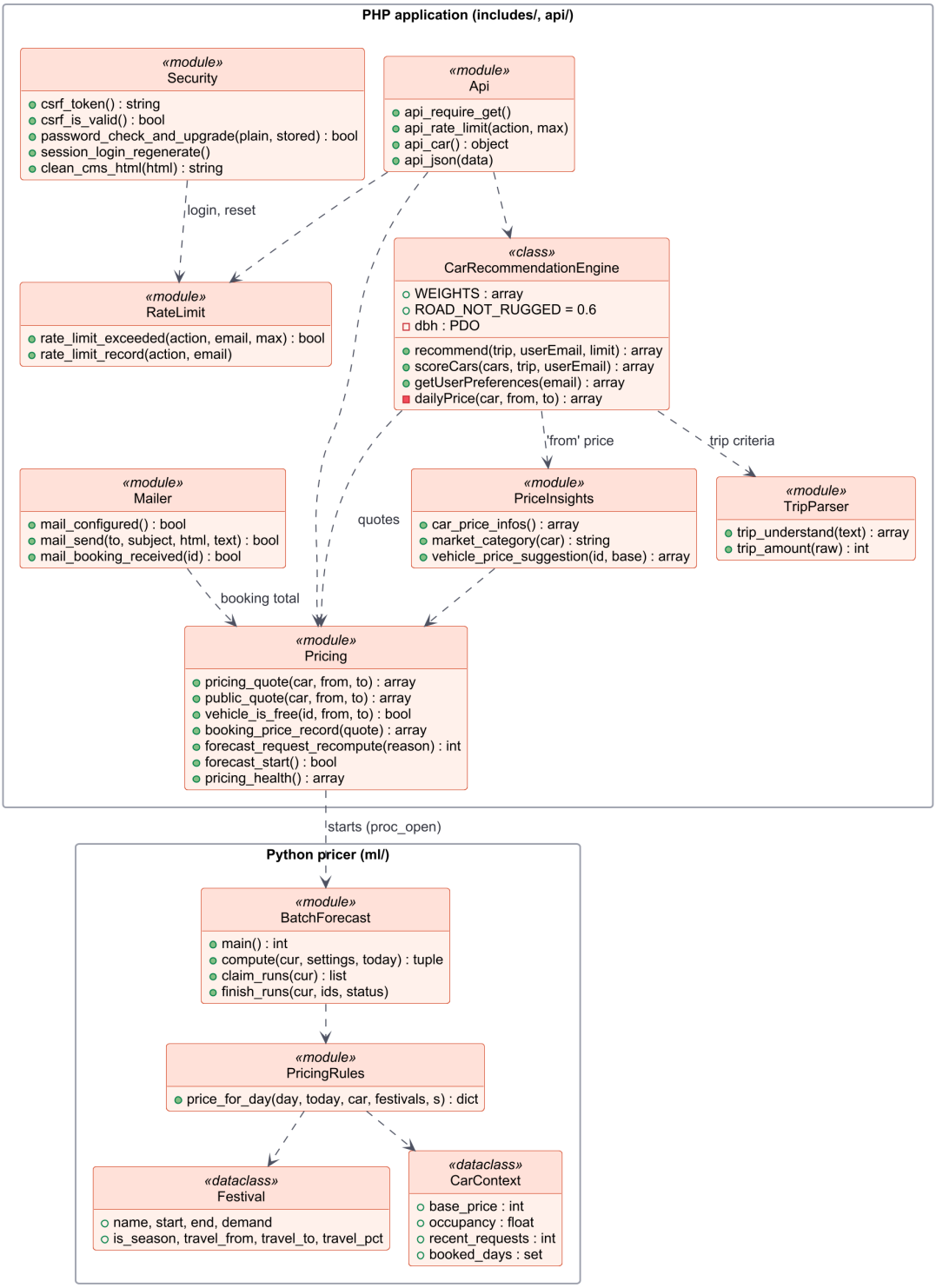


Figure 4.9 Class and module diagram

4.6 Sequence Diagrams

The sequence diagrams show the order of the messages exchanged between the user, the browser, the PHP modules, the Python pricer and the database for five important interactions.

Figure 4.10 shows the customer login. The form token is checked first, then the rate limit for the email address. The password is verified with the modern hash; if the account still has an old MD5 hash from the template, it is verified once and replaced by a new hash. On success, the session identifier is regenerated to prevent session fixation, and on failure the attempt is recorded for rate limiting.

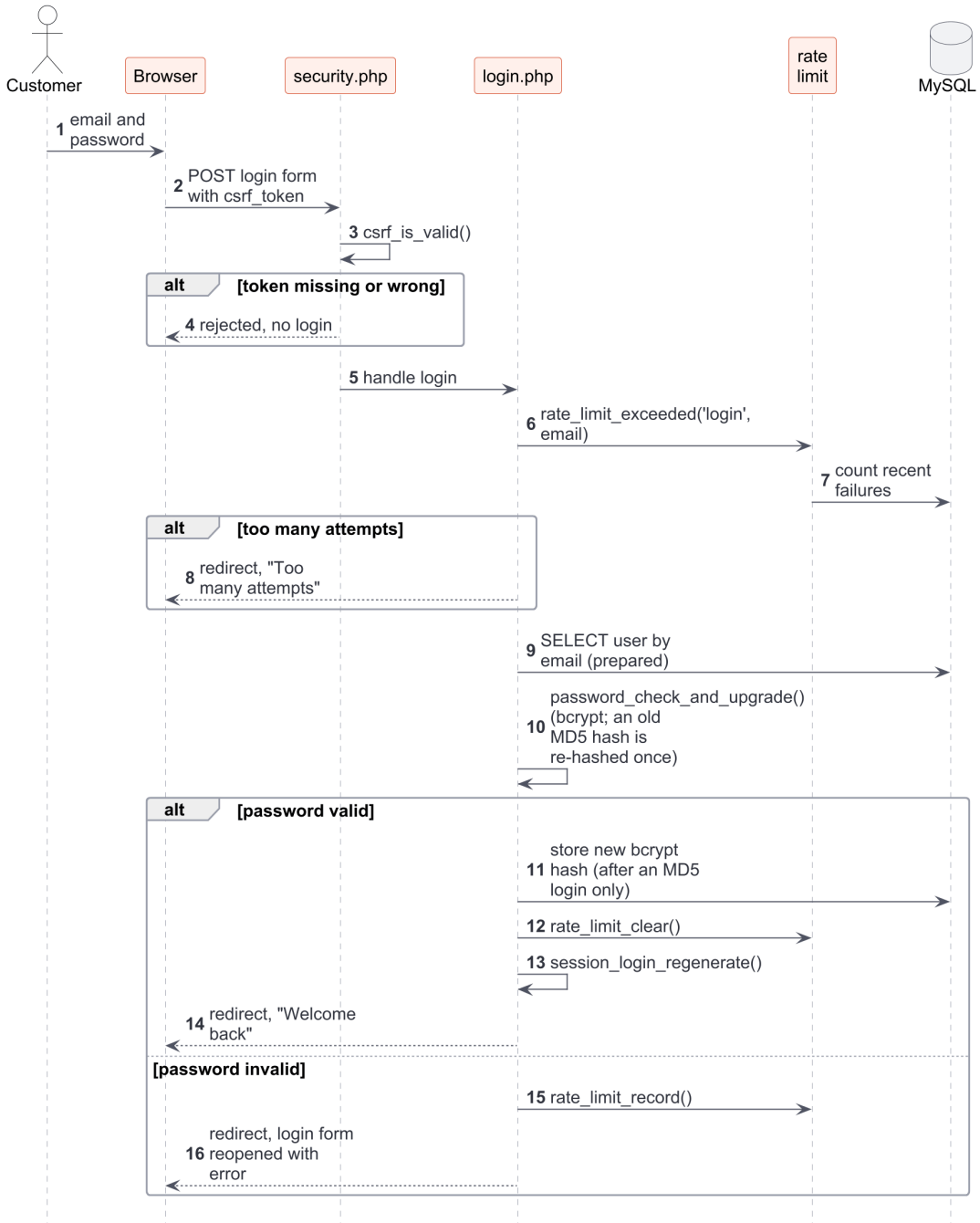


Figure 4.10 Sequence diagram: customer login

Figure 4.11 shows the search and filtering of the car listing. Every filter value is accepted only if it belongs to a list of allowed values, and the query uses bound parameters. The "from" price of each car, which is the lowest price of an available day in the next 30 days, is read from the forecast once per request.

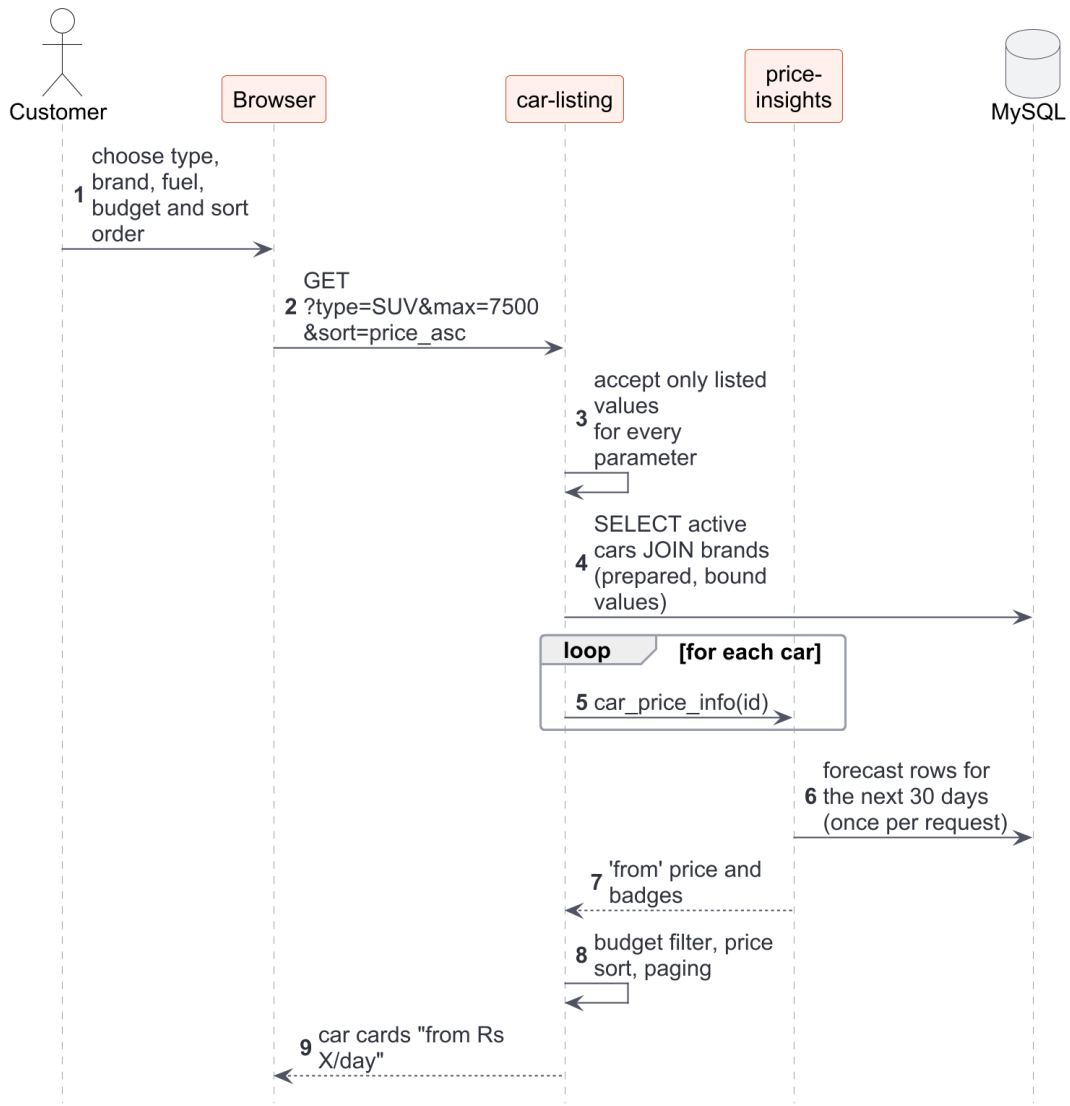


Figure 4.11 Sequence diagram: search and filter the car listing

Figure 4.12 shows the price quotation used by the price calendar. The JSON endpoint accepts only GET requests, is rate-limited to 120 requests in 15 minutes, and validates the car and the dates before asking the pricing module for a quote. The quote sums the day prices from the forecast, merges their reasons, uses the base price for any day beyond the forecast period, and checks whether the car is free.

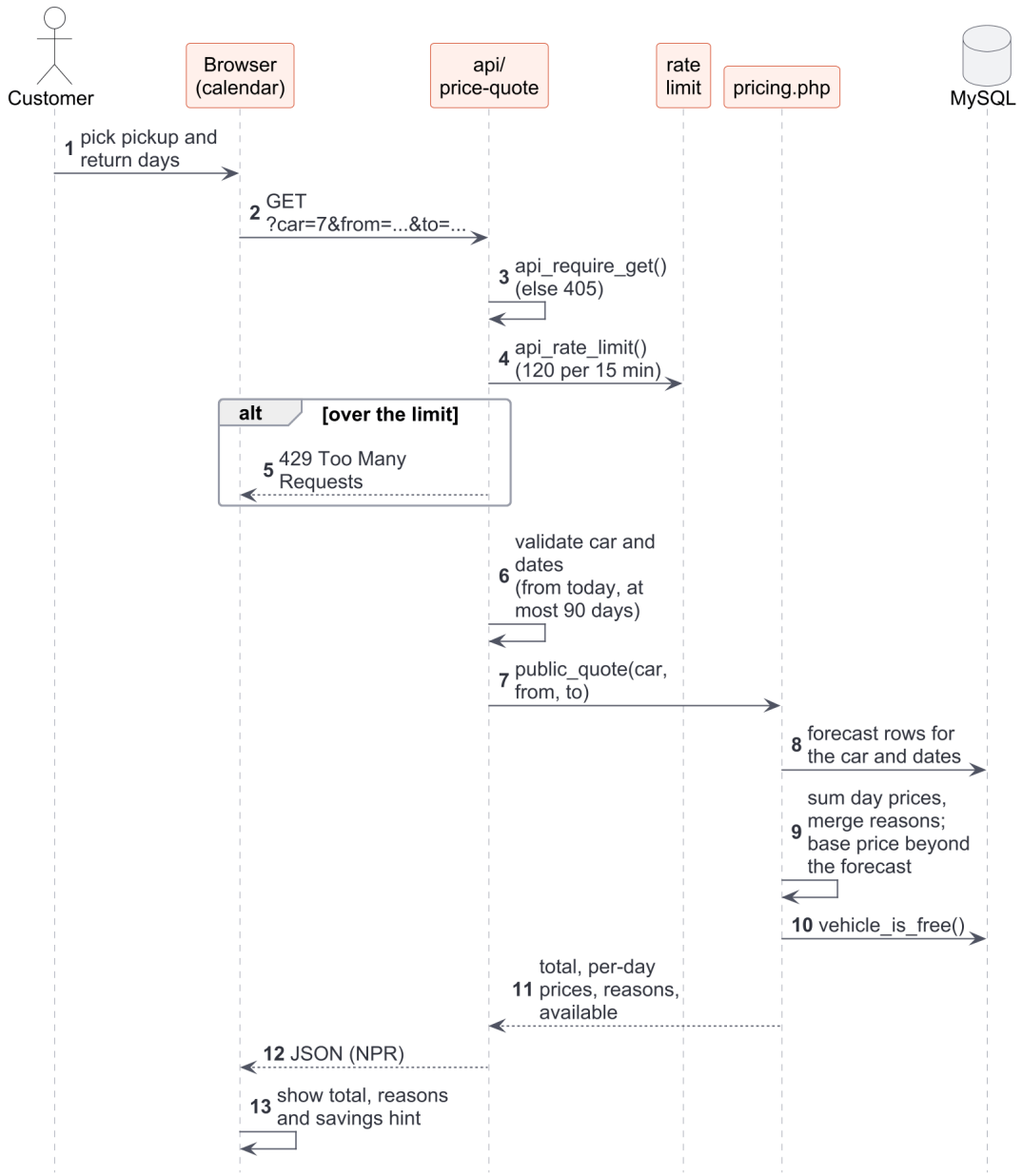


Figure 4.12 Sequence diagram: price quotation through the JSON API

Figure 4.13 shows the recommendation process. The trip parser extracts the vehicle type, road type, number of travellers and budget from the description. The recommendation engine then excludes cars that cannot meet the hard requirements, scores the remaining cars with the weighted model presented in Section 2.3.4, sorts them, and removes any match below 40%.

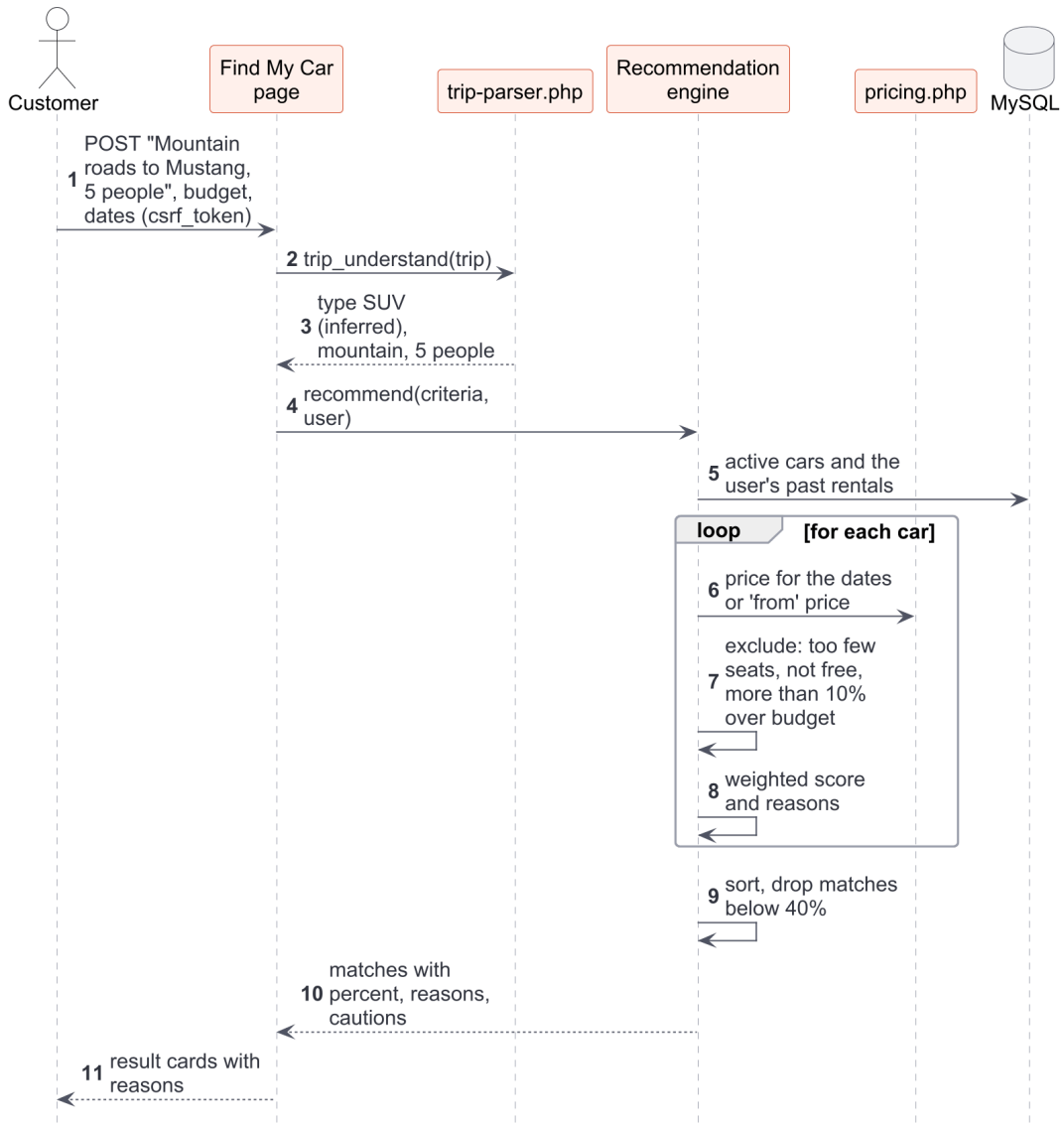


Figure 4.13 Sequence diagram: trip-based recommendation

Figure 4.14 shows the complete booking interaction, from the customer's request, through the server-side quotation, storage of the booking and recomputation of the forecast, to the administrator's confirmation and the customer's invoice.

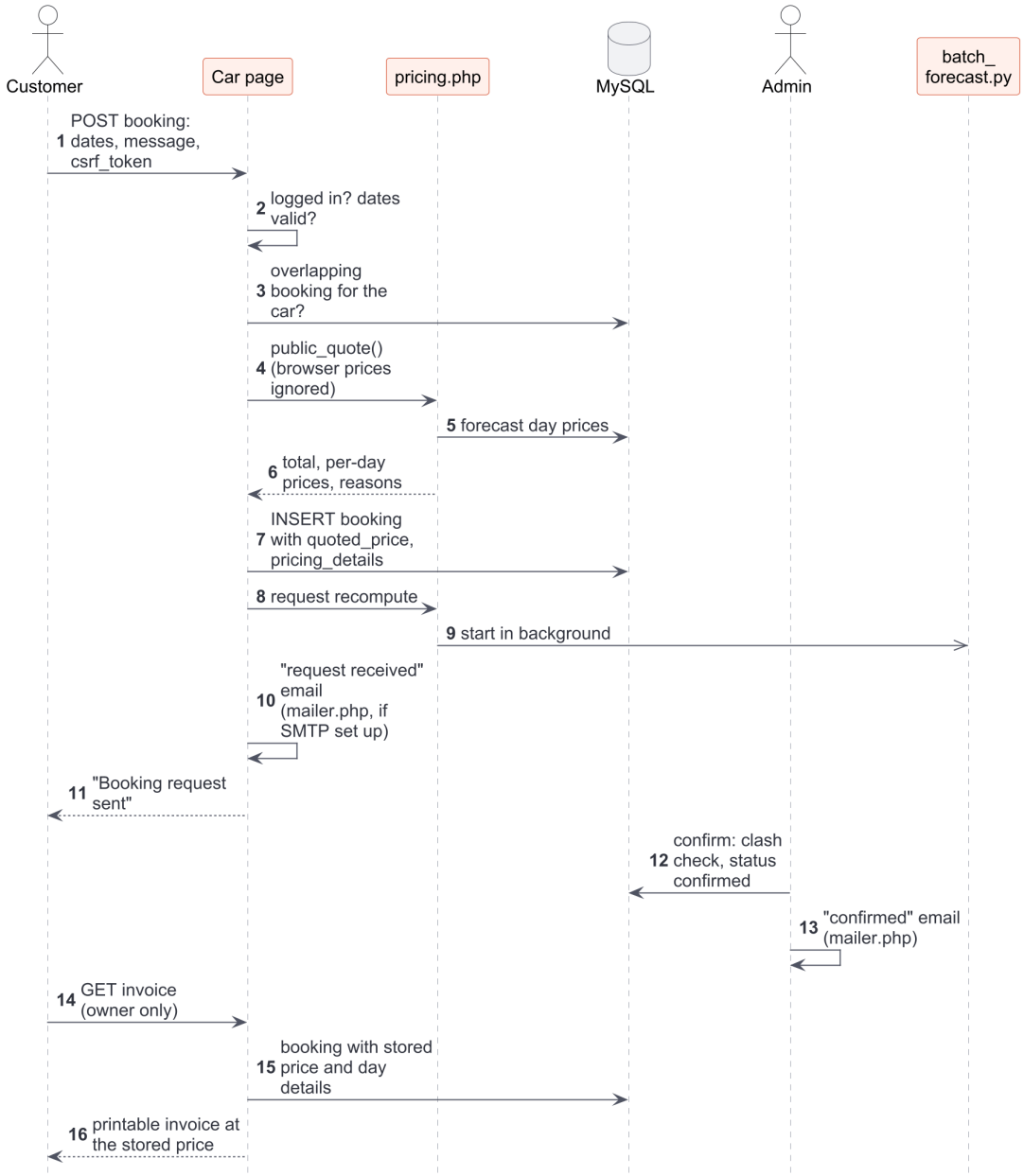


Figure 4.14 Sequence diagram: booking request, confirmation and invoice

4.7 Activity Diagrams

The activity diagram in Figure 4.15 shows the booking workflow across three swimlanes: the customer, the Rento system and the administrator. It makes explicit the two loops in the workflow, in which a customer who is not logged in is asked to log in and a request with invalid dates or for a car that is already booked returns to the date selection, as well as the check for clashes with confirmed bookings when the administrator confirms a booking.

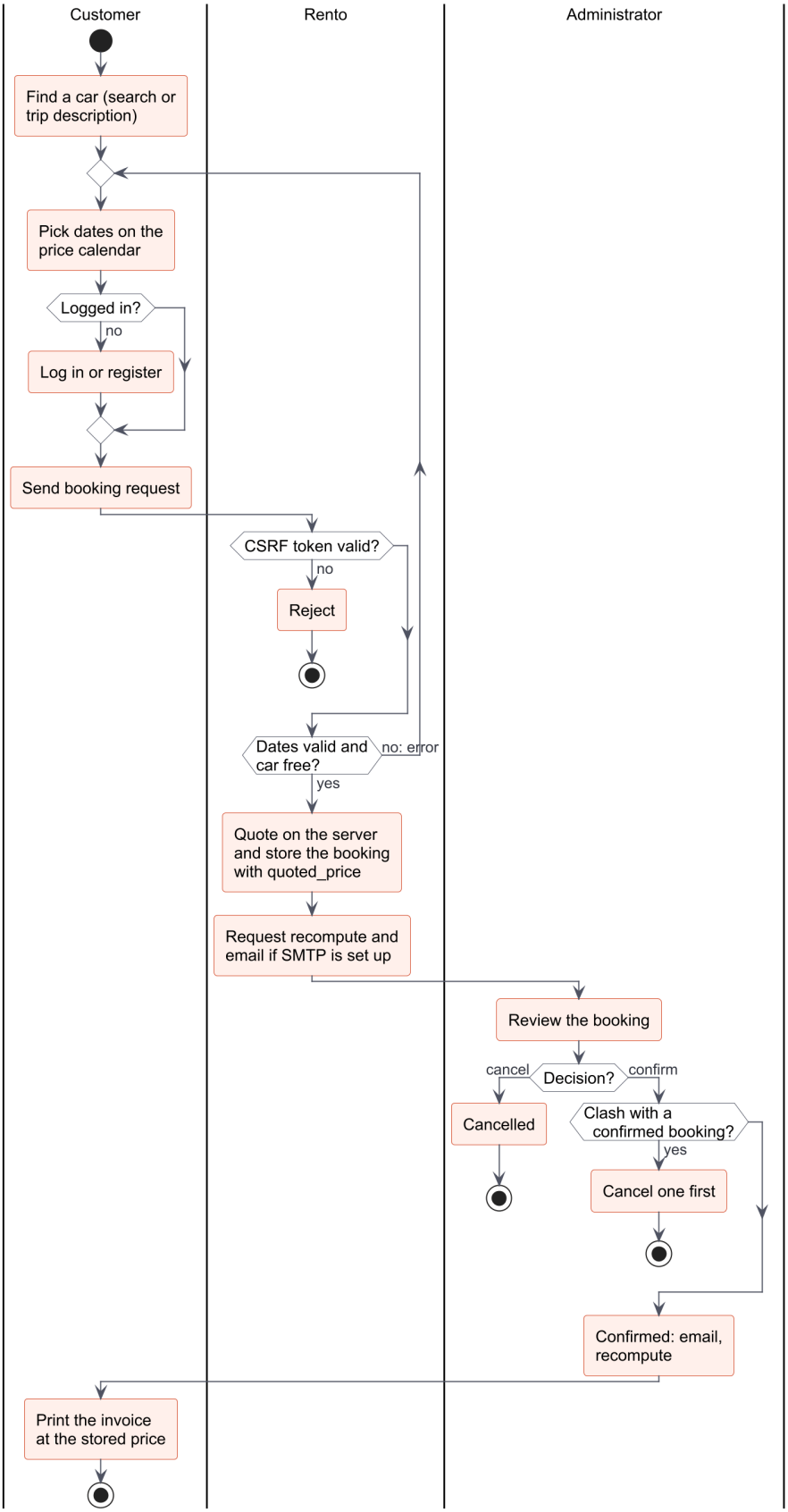


Figure 4.15 Activity diagram: booking workflow

Figure 4.16 shows the workflow of the forecast recomputation. It includes the database lock that prevents two recomputations from running at the same time: a run that finds the lock taken waits up to 300 seconds, and all queued requests are then handled together. It also shows the gate that switches the occupancy and demand factors on only when at least 50 confirmed bookings exist, and the order in which the factors, the caps, the rounding and the manual overrides are applied to each car and day.

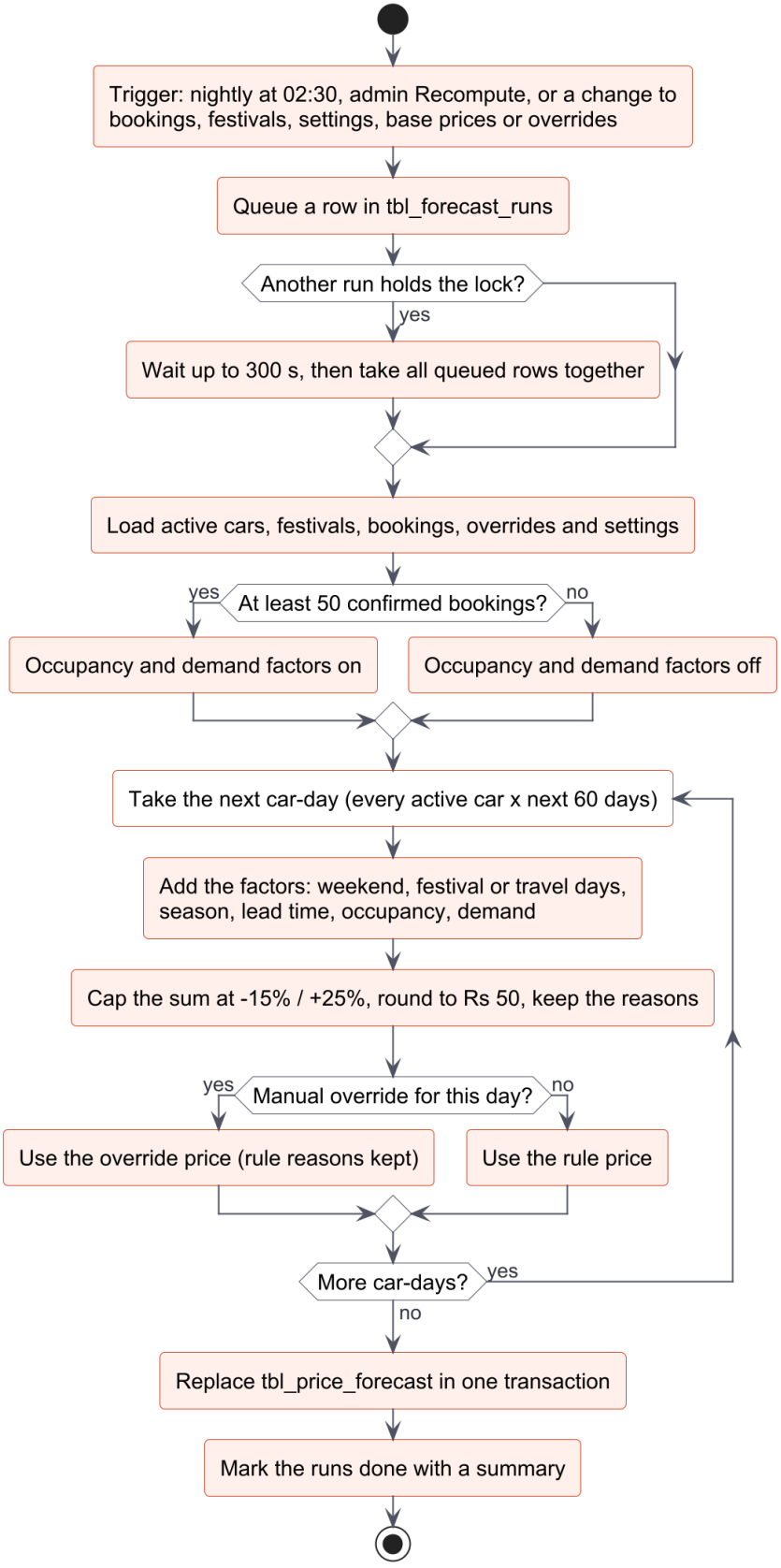


Figure 4.16 Activity diagram: price forecast recomputation

4.8 User Interface Design

4.8.1 Design Principles

The user interface was designed around four principles. First, prices must be clear: every price is shown in Nepalese rupees in the form "Rs 10,500/day", the price calendar shows the price of each day before the customer chooses dates, and every adjusted price is accompanied by its reason. Second, the interface must work on mobile phones, which many Nepali customers use to browse, so every page was designed and tested at widths from 390 pixels, the width of a common phone, up to desktop screens. Third, the interface must be accessible: text and interface elements meet the colour contrast requirements of WCAG 2 level AA (World Wide Web Consortium, 2023), every function can be used with the keyboard, focus is always visible, messages are announced to screen readers, and animations are reduced when the user's system asks for reduced motion. Fourth, the interface should follow familiar conventions, such as a top navigation bar, cards for cars and forms with clear labels, so that customers do not need instructions.

4.8.2 Design System

The visual design is defined by a small design system of CSS custom properties, or design tokens, in a single file, which the Bootstrap 5.3 components also use. Table 4.5 summarises its main elements. The same tokens are redefined for dark mode, which follows the user's system setting by default and can be switched manually.

Table 4.5 Main elements of the design system

Element	Value	Use
Brand colour	Orange #ff5a36	Accents and highlights; with near-black text (6.1:1 contrast)
Button colour	#d23f1c with white text	Buttons (4.7:1 contrast)
Orange text	#c43c1a	Links and orange text on white (5.2:1 contrast)
Dark sections	Ink #101114	Footer, dark sections, headings
Status colours	Green, amber, red and blue, each with a text colour that meets 4.5:1 on its tinted background	Success, warning, error and information messages; cheaper, higher and booked days in the price calendar
Typography	Plus Jakarta Sans for headings, Inter for body text	All text
Corner radius	6, 10, 14 and 20 pixels, and fully rounded pills	Inputs, cards, panels, badges
Layout	Bootstrap 5.3 grid with its standard breakpoints, and own adjustments below 576 and 992 pixels	Responsive page layouts

4.8.3 Page Design

The main pages were designed from the customer's tasks. The home page (Figure 4.17) opens with a photographic carousel and a search form for the car type, brand and budget, followed by the fleet grouped by car type, the steps of booking, the reasons to choose Rento, an invitation to use Find My Car, testimonials and brands; the packages section appears only when at least one package is active. The car page (Figure 4.18) places the photographs and description of the car on the left and a booking panel on the right, with the price calendar at the top of the panel; each day in the calendar shows its price and is coloured when it is cheaper or more expensive than usual or already booked, and selecting a day shows the reasons for its price. On a phone, the booking panel is placed below the photographs and the calendar keeps its full size.

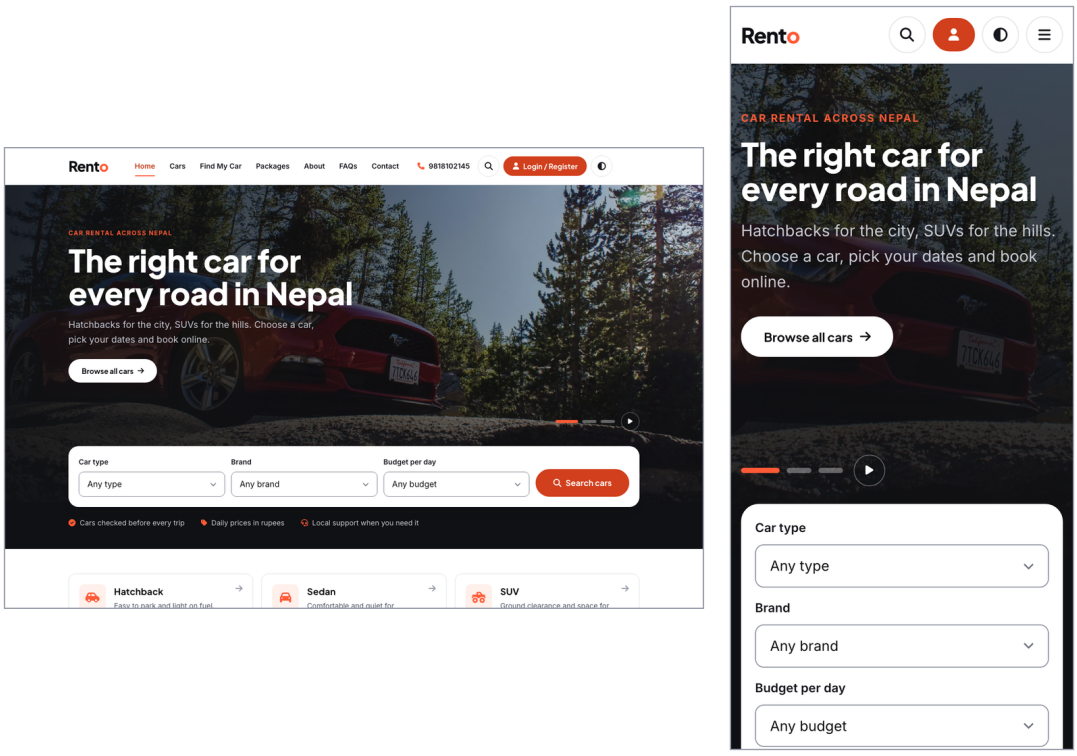


Figure 4.17 Home page on desktop (left) and mobile (right)

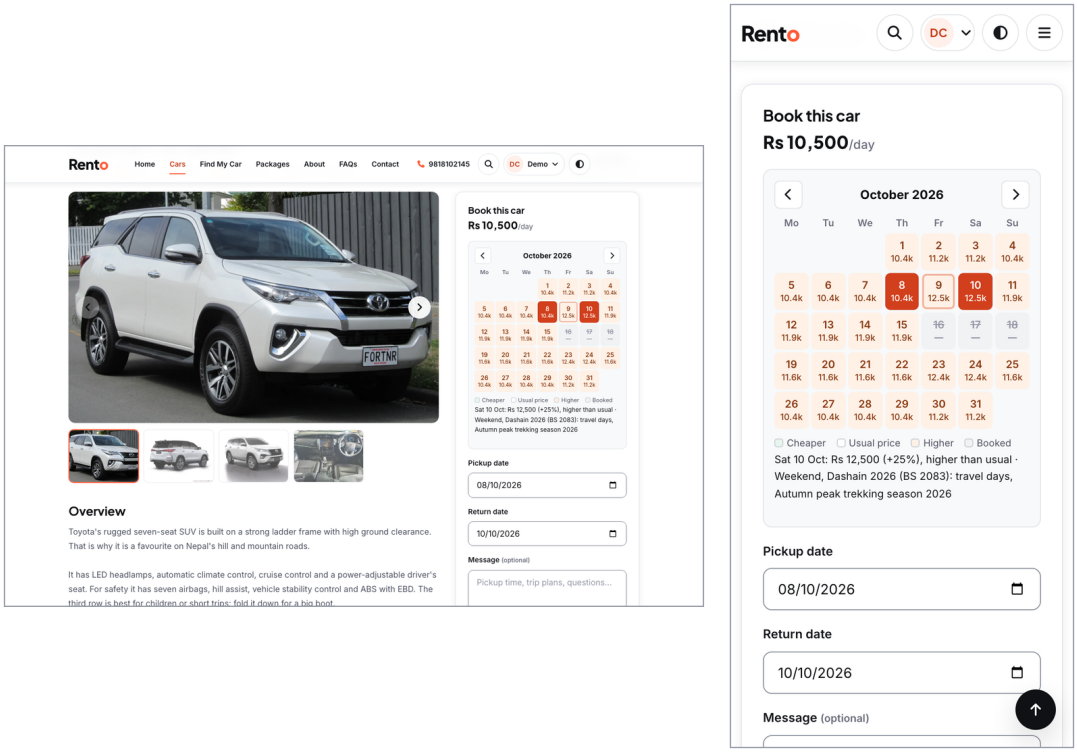


Figure 4.18 Car page with the price calendar on desktop (left) and mobile (right)

Find My Car (Figure 4.19) accepts a trip description in plain language, shows the criteria it has understood as small labels while the customer types, and presents the

recommended cars as cards with their match percentage, reasons and any cautions. Figure 4.20 shows the dark mode of the home page and the car page.

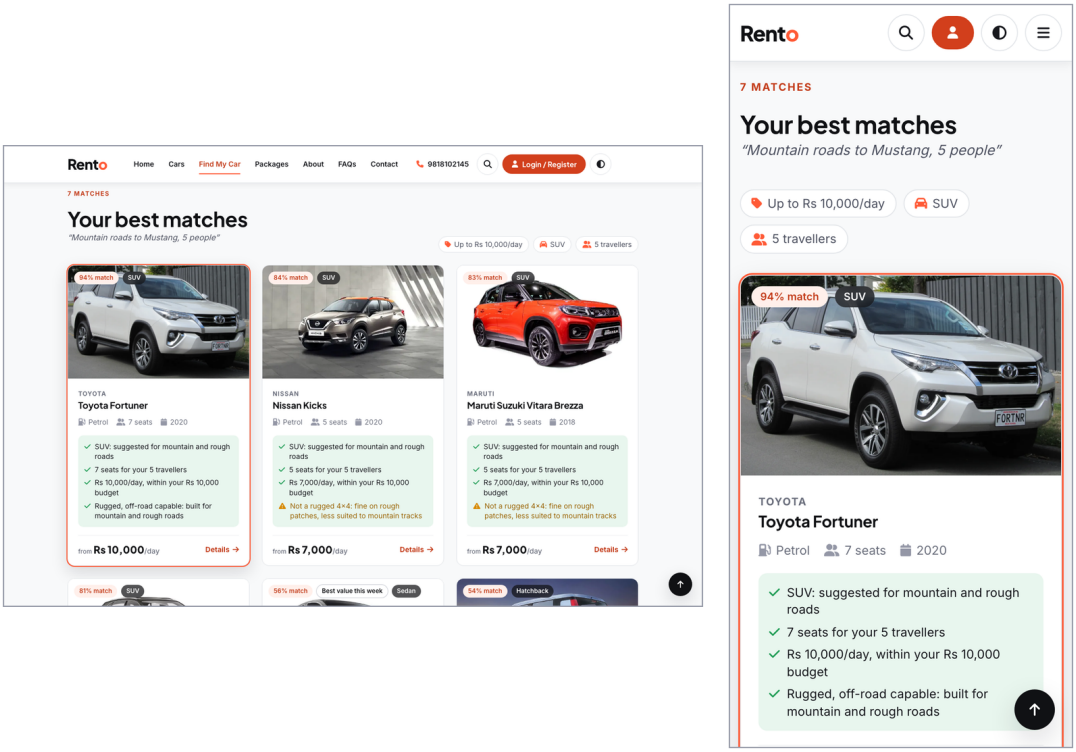


Figure 4.19 Find My Car recommendations on desktop (left) and mobile (right)

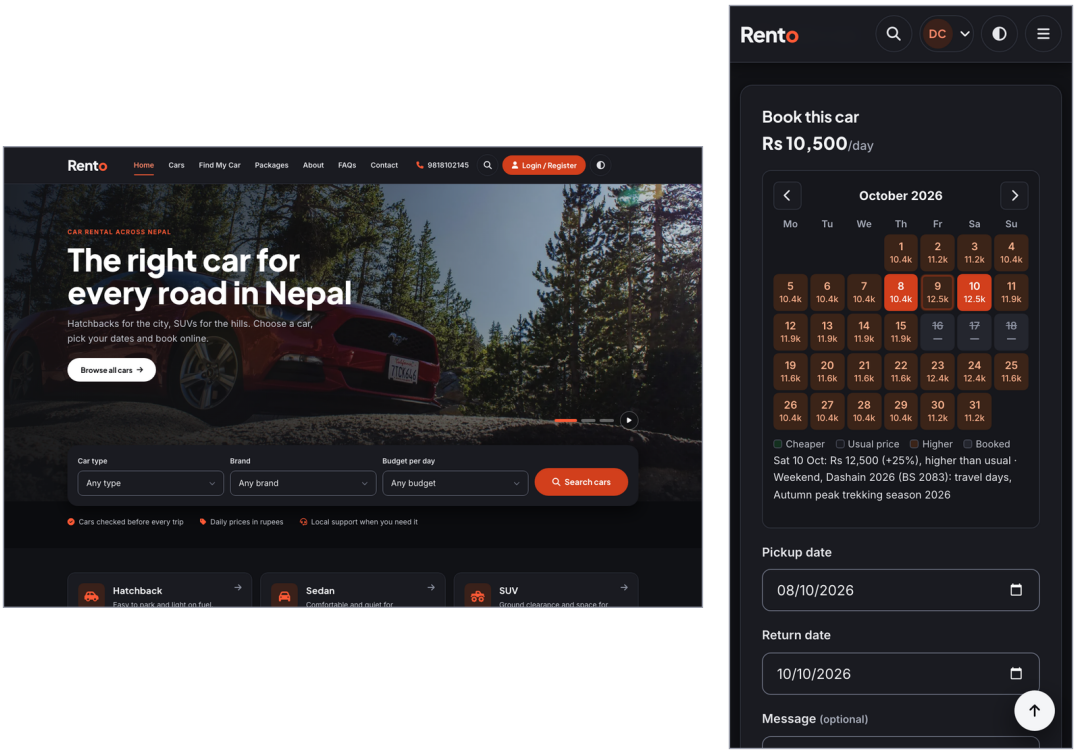


Figure 4.20 Dark mode of the home page (left) and the car page on mobile (right)

The administration area uses the same design system with a sidebar and a top bar. It is designed for efficient management rather than for promotion: tables can be searched and sorted, all actions that change data are buttons in forms protected by tokens, and the pages for smart pricing, AI Insights and festivals explain each setting and show the effect of the current prices. The administration pages are presented with the implementation in Chapter 5.

4.9 Summary

This chapter presented the design of Rento. The use case diagram and its description defined the functions offered to guests, customers and the administrator. The data flow diagrams showed how data moves between the nine processes and eight data stores, and decomposed the booking and forecast processes, which are the core of the system. The entity relationship diagrams and data dictionaries described the 23 tables of the database, including the logical relationships and the tables added for smart pricing. The class and module diagram, sequence diagrams and activity diagrams showed how the code is organised and how the main interactions and workflows take place. Finally, the design principles, the design system and the main pages of the user interface were presented. Chapter 5 describes how this design was implemented.

CHAPTER 5: IMPLEMENTATION

5.1 Introduction

This chapter describes how the design presented in Chapter 4 was implemented. Section 5.2 introduces the development environment and the structure of the code. Section 5.3 describes the implementation of the database. Sections 5.4 and 5.5 present the customer and administration modules, and Section 5.6 the JSON application programming interface. Sections 5.7 to 5.10 explain the intelligent components: the smart pricing engine, the forecast pipeline, the machine-learning experiments and the recommendation engine. Section 5.11 describes the security measures, Section 5.12 the remaining quality features, and Section 5.13 summarises the chapter. The code excerpts in this chapter are taken from the source code, shortened where marked with an ellipsis.

5.2 Development Environment and Code Structure

The system was developed and tested on the local server package MAMP, which provides Apache 2.4.62, PHP 8.3.14 and MySQL 8.0.40, with the Python components in a separate Python 3.12.14 virtual environment. The tools and their versions are listed in Table 3.6. The code is organised in the folders shown in Table 5.1; the public pages remain in the root folder with their original file names, so that existing links continued to work while the pages were rebuilt.

Table 5.1 Structure of the source code

Folder	Contents	Lines
Root folder	Public pages, such as the home page, car listing, car page, Find My Car, packages, My Bookings and invoice	2,788
includes	Shared PHP modules: security, configuration, pricing, price insights, recommendation engine, trip parser, mailer, SEO, layout and components	3,649
api	JSON endpoints for price quotes, price calendars and recommendations, with shared set-up	278
admin	Administration pages and their shared layout, forms and upload handling	4,769
assets	Design tokens and stylesheets (CSS) and scripts for the price calendar, trip search and theme (JavaScript)	6,923 CSS, 643 JS
ml	Python pricer, forecast pipeline and machine-learning experiments	1,508
database	22 additive migrations and their undo scripts	563
tests	PHPUnit and pytest test suites and evaluation scripts	1,035

Line counts exclude third-party libraries (Bootstrap, jQuery, DataTables, Chart.js, Quill, PHPMailer) and minified files.

Configuration values that differ between computers, such as the database user and password and the application key used for encryption, are read from environment variables or from a local configuration file that is excluded from version control. No password is stored in the repository.

5.3 Database Implementation

The database was extended through 22 numbered migration scripts, summarised in Table 5.2. Every migration is additive: it creates a table or adds columns and never renames or removes existing ones, and a backup of the database was made outside the web folder before each migration was applied. Where a migration changed data, such as the market-based prices of migration 017, an undo script was written as well.

Table 5.2 Database migrations

Migrations	Changes
001–004	Password reset tokens and rate-limit records, starter FAQ content, reset tokens for administrators
005–007	Editable site content, active and inactive vehicles, 17 researched vehicles and 5 brands (added as inactive)
008–010	Festival calendar with 40 dated entries, packages with categories, photos and enquiries, driver-only vehicles
011–016	Pricing settings, price forecast and forecast runs, stored booking price, festival travel days, manual price overrides, festival pricing for packages
017–018	Market-based prices, corrected specifications and descriptions of the active cars, FAQ text on smart pricing
019–022	Rugged flag, search engine descriptions, email settings, draft versions of content pages

The application connects to MySQL through PHP Data Objects (PDO) with a dedicated database user, rento, which has only the privileges that the application needs on the rento database; the administrative root account is used only to apply migrations. Every query that contains a value from a request is a prepared statement with bound parameters.

5.4 Customer Modules

5.4.1 Car Listing and Car Page

The car listing accepts the filters keyword, type, brand, fuel, maximum price, sort order and page as query parameters. Each value is accepted only if it belongs to a list of allowed values, and the budget filter and the price sort use the "from" price of each car, which is the lowest price of an available day in the next 30 days taken from the forecast. Each car card also shows badges based on the data, such as "Below market" when the car's 30-day average is below the published self-drive rate for its category.

The car page shows the photographs, specifications and description of the car and a booking panel with the price calendar. The calendar is drawn by a small JavaScript module that requests the prices of a month from the price-calendar endpoint and colours each day according to whether it is cheaper or more expensive than usual or already booked. When the customer selects the dates, the module requests a quote for the rental and shows the total, the reasons and, where applicable, how much the customer would save by moving the rental to cheaper days. The calendar can be operated with the keyboard, and changes of price are announced to screen readers.

5.4.2 Booking, My Bookings and Invoice

A booking request is handled by the car page itself. After checking the form token, the login, the dates and the availability of the car, the server calculates the quote and stores it with the booking, as shown in Figure 5.1. Any price sent by the browser is ignored, so that a customer cannot change the price by modifying the form.

```
$quote = public_quote($dbh, $car, $fromdate, $todate);
[$quotedPrice, $pricingDetails] = booking_price_record($quote);
...
$sql = "INSERT INTO
tblbooking(BookingNumber,userEmail,VehicleId,FromDate,ToDate,message,Status,quote
d_price,pricing_details,priced_at)

VALUES(:bookingno,:useremail,:vhid,:fromdate,:todate,:message,:status,:quoted,:de
tails,NOW())";
...
forecast_request_recompute($dbh, 'new booking #' . $bookingno);
```

Figure 5.1 Code excerpt: the booking is stored with the price calculated on the server (vehical-details.php)

My Bookings lists the customer's bookings with their status and stored price, and the invoice page, which can be opened only by the owner of the booking, shows the stored total and the price of each day with its reasons, in a layout designed for printing. Figure 5.2 shows the invoice of a demonstration booking.

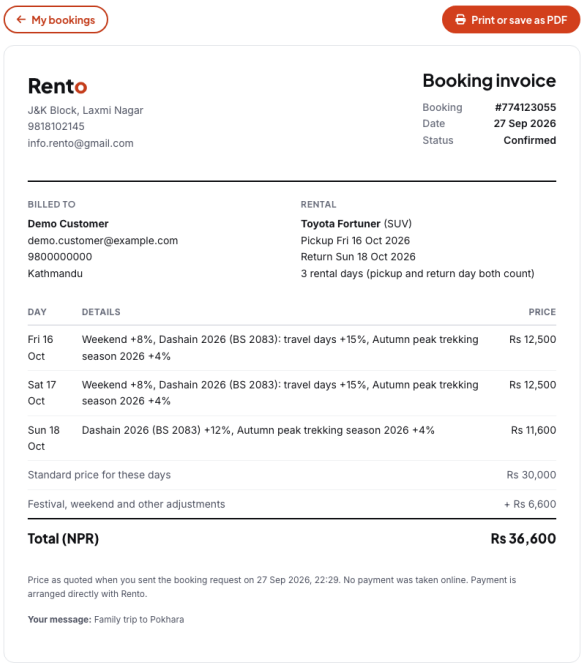


Figure 5.2 Printable invoice showing the stored price and the price of each day

5.4.3 Find My Car and Packages

Find My Car accepts a trip description in plain language with optional dates, budget and number of travellers. The same trip parser exists in JavaScript, which shows the understood criteria as labels while the customer types, and in PHP, which is used by the server; the recommendation engine behind it is described in Section 5.10. The packages pages list active tour and transfer packages with their itinerary, inclusions, exclusions and prices, and send enquiries to the administrator. Cars that are available only with a driver are also booked through an enquiry, because their price depends on the itinerary. At the time of writing, all 16 packages were inactive and the packages section was therefore hidden on the public site.

5.5 Administration Modules

The administration area has its own login and session, separate from the customer session. Its dashboard shows key figures calculated from the database, such as the revenue of confirmed bookings, the active rentals, the pending booking requests, the use of the active fleet and the new users of the month. The bookings pages share one list with a status filter and a detail page from which a booking is confirmed or cancelled; confirmation is refused if the booking would overlap a confirmed booking of the same car. The vehicle form manages all attributes of a car, including the active, driver-only and rugged flags and up to five photographs, and shows a smart price suggestion next to the base price.

The festivals page lists the festival calendar with the dates, demand level, travel window and the source of each entry, and marks the dates that are not yet official. The Smart Pricing page lets the administrator change every pricing setting within safe limits, recompute the forecast, and see the health of the pricing model, as shown in Figure 5.3: which pricer is active, how many confirmed bookings exist compared with the thresholds for the occupancy and demand factors and for the learned model, and how much of the fleet the forecast covers.

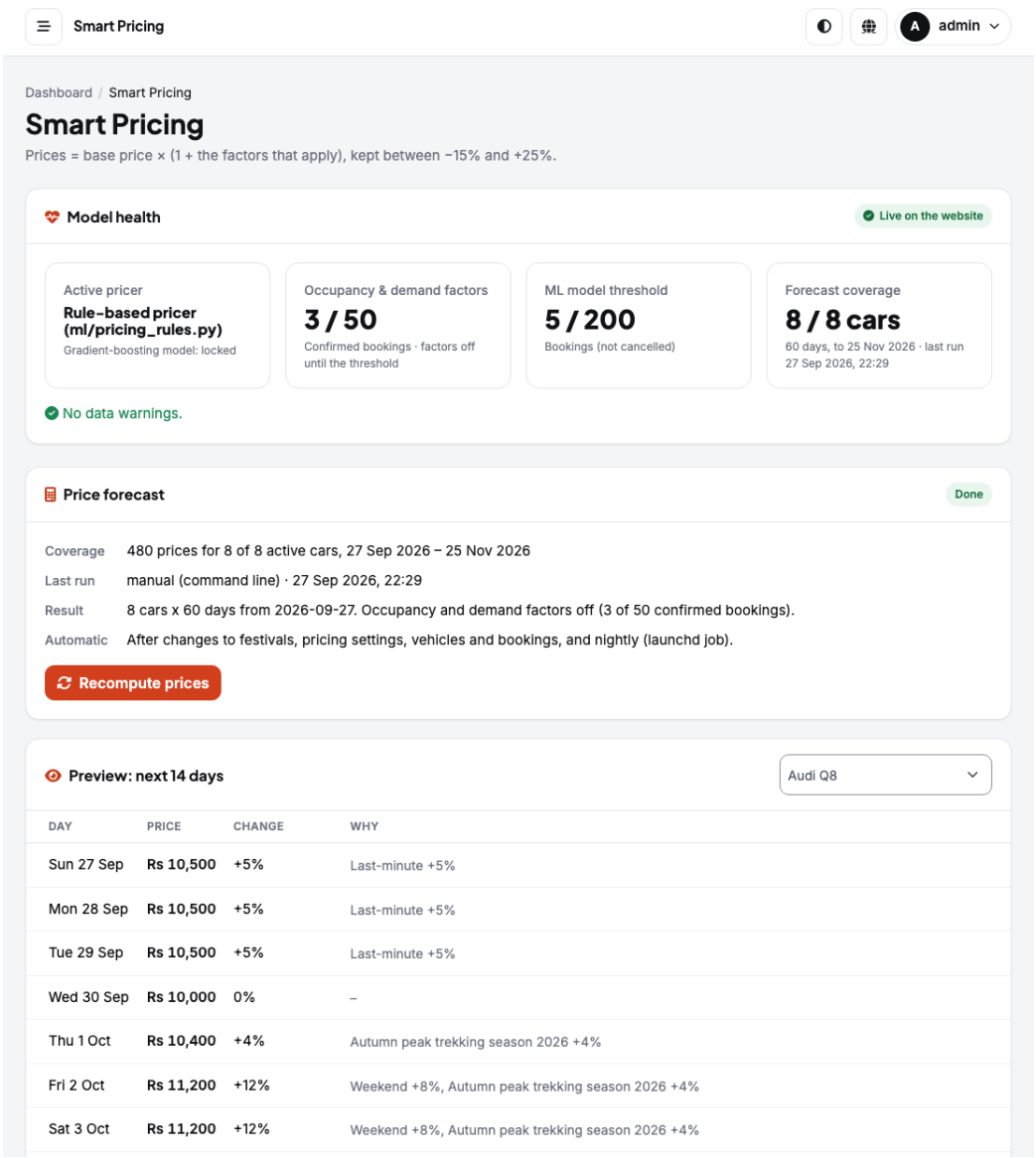


Figure 5.3 Smart Pricing page with the model health and a preview of the next days

AI Insights, shown in Figure 5.4, lists for each car its base price, its average forecast price over 30 and 60 days, its lowest and highest days and the main reasons, together with a suggested base price. The administrator can apply the suggestion or set a manual price for a range of dates; both actions trigger a recomputation. The page states explicitly that the suggestions come from the pricing rules and not from a trained model.

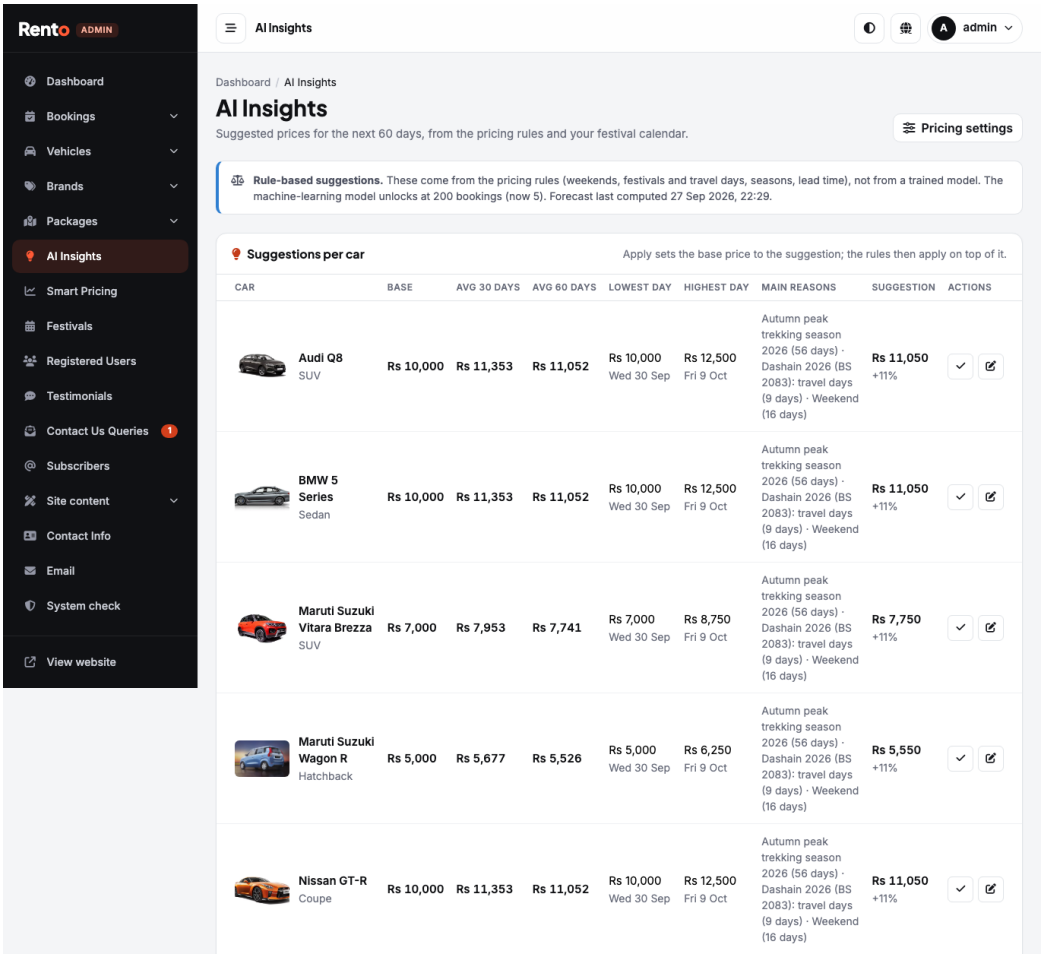


Figure 5.4 AI Insights with the suggested price for each car

> *The screenshots in this report were taken with one temporary demonstration booking in the database, which was deleted afterwards; the booking counts shown in Figures 5.3 and 5.4 therefore include it.*

The remaining administration pages manage brands, packages and their categories and enquiries, registered users, testimonials, contact queries, subscribers, the content pages with a draft workflow, the editable text and images of the public site, the contact information, the email settings and a system check of the production settings.

5.6 JSON Application Programming Interface

The three JSON endpoints in Table 5.3 serve the price calendar, the booking panel and the recommendation feature. They share a set-up file that accepts only GET requests, applies a rate limit per IP address, returns errors as JSON with the correct HTTP status code, and never exposes PHP or database error messages; unexpected errors are written to the server log only. The endpoints do not use the session, so they can be called without logging in, but they are read-only.

Table 5.3 JSON API endpoints

Endpoint	Parameters	Response
api/price-quote.php	car, from, to (dates, from today, at most 90 days)	Total, price and reasons of each day, availability; rate limit 120 requests in 15 minutes
api/price-calendar.php	car, month (YYYY-MM)	Price and reasons of each remaining day of the month, with days beyond the forecast marked as estimates; 120 requests in 15 minutes
api/recommend.php	trip, and optionally budget, type, seats, fuel, from, to	Ranked self-drive cars with match percentage, reasons and cautions, and matching driver-only cars; 60 requests in 15 minutes

5.7 Smart Pricing Engine

The smart pricing engine implements the capped rule-based model of Equation 2.1. The factors, their default values and the conditions under which they apply are given in Table 5.4. All values are settings that the administrator can change on the Smart Pricing page.

Table 5.4 Pricing factors and their default values

Factor	Default	Condition
Weekend	+8%	The day is a weekend day: Friday or Saturday in Nepal
Festival, high demand	+12%	The day falls within a festival marked as high demand, such as Dashain, Tihar or Chhath
Festival, medium demand	+5%	The day falls within a festival marked as medium demand, such as a single-day holiday
Festival travel days	+15%	The day falls within the travel window before a festival; each festival can set its own percentage
Travel season	+4%	The day falls within a season, such as the autumn trekking season
Last minute	+5%	The day is at most 2 days ahead
Early booking	−4%	The day is at least 45 days ahead
High occupancy	+6%	At least 60% of the car's next 30 days are booked (only after 50 confirmed bookings)
Low occupancy	−3%	At most 10% of the car's next 30 days are booked (only after 50 confirmed bookings)
Recent demand	+5%	At least 3 booking requests for the car in the last 30 days (only after 50 confirmed bookings)
Limits	−15% / +25%	The sum of the factors is capped at these values; prices are rounded to Rs 50

Only the strongest festival factor applies on a day, so that a travel day, a festival and an overlapping holiday are not added together; the travel season is counted separately. The occupancy and demand factors are switched off until 50 confirmed bookings exist, because with almost no bookings every car would otherwise appear to be in a quiet period and would be discounted every day. Figure 5.5 shows the core of the function that prices one car for one day.

```
def price_for_day(day, today, car, festivals, s):
    reasons = []
    weekend_days = {int(d) for d in str(s["pricing_weekend_days"]).split(",")
if str(d).strip()}
    if day.isoweekday() in weekend_days and s["pricing_weekend_pct"]:
        reasons.append({"label": "Weekend", "pct":
_pct(s["pricing_weekend_pct"])})
    # The strongest festival that day; a travel day uses the travel surcharge
instead
    ...
    days_ahead = (day - today).days
    if days_ahead <= s["pricing_lastminute_days"] and
s["pricing_lastminute_pct"]:
        reasons.append({"label": "Last-minute", "pct":
_pct(s["pricing_lastminute_pct"])})
    elif days_ahead >= s["pricing_early_days"] and s["pricing_early_pct"]:
        reasons.append({"label": "Early booking", "pct":
_pct(s["pricing_early_pct"])})
    ...
    total = sum(r["pct"] for r in reasons)
    low, high = -float(s["pricing_max_discount"]),
float(s["pricing_max_surcharge"])
    capped = total < low or total > high
    total = max(low, min(high, total))
    multiplier = round(1 + total / 100, 4)
    step = max(1, int(s["pricing_round_to"]))
    price = int(round(car.base_price * multiplier / step) * step)
```

Figure 5.5 Code excerpt: pricing one car for one day (ml/pricing_rules.py)

The effect of the rules is illustrated with the Toyota Fortuner, whose base price is Rs 10,000 per day, in the forecast computed on 27 September 2026. On an ordinary weekday such as Wednesday 30 September no factor applies and the price is Rs 10,000. On Friday 2 October the weekend (+8%) and the autumn trekking season (+4%) apply, giving Rs 11,200. On Friday 9 October, a travel day before Dashain, the weekend (+8%), the travel days (+15%) and the season (+4%) add up to +27%, which is capped at +25%, giving Rs 12,500. Figure 5.6 shows the whole 60-day forecast of the car, with the Dashain, Tihar and Chhath periods highlighted.

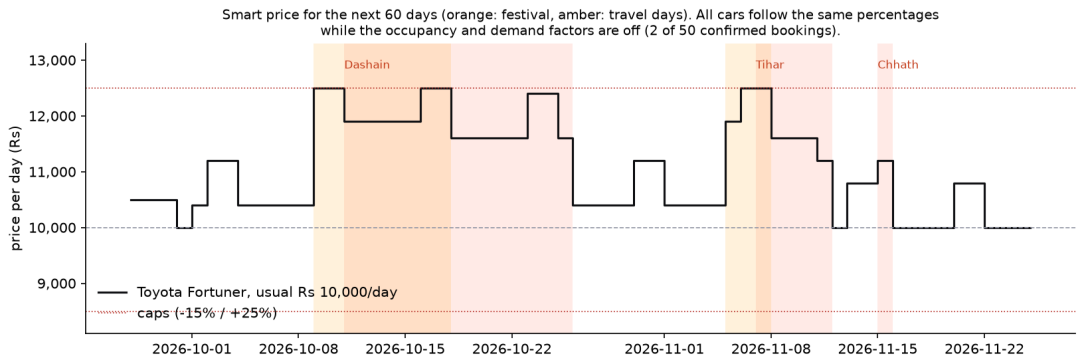


Figure 5.6 Smart price of the Toyota Fortuner for the next 60 days (forecast of 27 September 2026)

5.8 Forecast Pipeline

The forecast pipeline computes the price of every active car for each of the next 60 days and stores the results in the forecast table, so that pages and API calls only read prepared prices. A recomputation is requested whenever something that affects prices changes: a booking is created, confirmed or cancelled; a festival, a pricing setting or a vehicle is saved, added, activated, deactivated or deleted; or a suggested or manual price is applied or removed. In addition, a launchd job on the server starts the pipeline every night at 02:30, so that the lead-time factors move forward each day.

Each request inserts a row in the forecast runs table and starts the Python batch forecast in the background through PHP's proc_open function. The database and pricing settings are passed in environment variables rather than as command-line arguments, where they could be seen by other users of the server. The batch forecast first takes a named MySQL lock, so that only one recomputation runs at a time; a second one waits up to 300 seconds and, if the first is still running, leaves its request to be picked up by it. The pipeline then marks all queued runs as running, computes the forecast and replaces the table in one transaction, as shown in Figure 5.7, so that the website sees either the complete old forecast or the complete new one.

```
cur.execute("SELECT GET_LOCK(%s, %s) AS got", [LOCK_NAME, LOCK_WAIT_SECONDS])
if not cur.fetchone()["got"]:
    print("Another recompute is still running; this one will be picked up by
it.", file=sys.stderr)
    return 0
run_ids = claim_runs(cur)
...
rows, cars = compute(cur, settings, today)
# Replace the whole forecast in one transaction: readers see the old or the new
one.
cur.execute("DELETE FROM tbl_price_forecast")
cur.executemany("INSERT INTO tbl_price_forecast (VehicleId, ForecastDate,
BasePrice, Multiplier, Price, Reasons, IsAvailable, RunId)"
                " VALUES (%s, %s, %s, %s, %s, %s, %s, " + str(int(run_ids[-1]))
+ ")", rows)
...
finish_runs(cur, run_ids, "done", cars, len(rows), ...)
conn.commit()
```

Figure 5.7 Code excerpt: replacing the forecast in one transaction (ml/batch_forecast.py)

With the eight active cars, a run writes 480 rows. If the forecast is missing for a day, for example beyond the 60-day period, the pricing module uses the base price for that day and writes a note to the server log, so that the site continues to work even if the Python component is unavailable. A forecast row computed for an older base price is ignored for the same reason.

5.9 Machine-Learning Experiments

5.9.1 Purpose and Data

Because Rento has only four real bookings, two of them confirmed, the live site prices with the rules described in Section 5.7, and the learned pricer is switched off until about 200 bookings exist. The machine-learning experiments were carried out offline to determine which kind of model should replace or complement the rules at that point. They were implemented in a single script, ml/train_evaluate.py, which uses a fixed random seed of 42 so that the results can be reproduced, and which saves all results and charts for this report. Two external datasets were used; neither contains Rento bookings.

The first dataset is an hourly vehicle rental demand dataset, [INSERT demand dataset name and URL] ([INSERT citation]). The August prototype had prepared it into 18,247 labelled hourly rows from 18 August 2018 to 28 February 2021, with calendar features and features for Nepali festivals. Only features that are known in advance were used, such as the hour, the day of the week, the month, the day of the year, whether the day is a weekend or a festival, and cyclical encodings of the hour, weekday and month, because Rento prices up to 60 days ahead. The second dataset

consists of 1,640 public listings collected from the Sajilo Rental website in August 2026, covering 192 vehicle models of 32 manufacturers, with the listed prices, the distance limits and the estimated fuel cost per kilometre.

5.9.2 Models and Evaluation Method

Four models were compared with a simple baseline in each experiment, as listed in Table 5.5. The baselines follow the advice of Hyndman and Athanasopoulos (2021) to compare every method with a simple benchmark: the mean demand for the same weekday and hour, the mean daily demand for the same weekday, and the median price.

Table 5.5 Models used in the experiments

Model	Configuration
Baseline	Demand: mean of the training data for the same weekday and hour (hourly) or weekday (daily). Price: median of the training prices
Linear Regression	scikit-learn, with standardised numeric features and one-hot encoded categories
Random Forest	300 or 400 trees, minimum 2 or 3 samples per leaf
Gradient Boosting	300 or 400 trees, learning rate 0.03 or 0.05, maximum depth 3 or 4, subsample 0.9 for demand
Multilayer perceptron	The PyTorch network trained in the August prototype, loaded as saved and not retrained (price experiment only)

The demand data were split chronologically, with the last 20% of the timeline as the test set, and the models were also evaluated with 5-fold time-series cross-validation on the training period (Bergmeir & Benítez, 2012). The price data were split with the same 70/15/15 split and random state as the prototype's neural network, so that all models were scored on the same 246 test listings. Because a random split places listings of the same vehicle model in both the training and the test set, the price models were also evaluated with 5-fold cross-validation grouped by vehicle model, which measures how well prices are predicted for models that were never seen in training. All models were compared with the MAE, RMSE and R^2 metrics defined in Section 2.3.3.

5.9.3 Results: Demand

Table 5.6 shows the results for hourly demand. The mean demand was 74.0 rentals per hour. Gradient Boosting achieved the best results on the test set, but it improved on the seasonal baseline only slightly, from an R^2 of 0.228 to 0.285, and Linear

Regression performed worse than the baseline. In the time-series cross-validation, Linear Regression was very unstable, with a mean R^2 of −7.1, while the tree ensembles were stable but weak, with mean R^2 values of 0.124 for Random Forest and 0.070 for Gradient Boosting. The most important features were the day of the year and the hour.

Table 5.6 Results of Experiment 1: hourly demand (test set, 3,650 hours)

Model	MAE	RMSE	R^2
Seasonal baseline	30.95	39.60	0.228
Linear Regression	31.39	40.22	0.203
Random Forest	30.57	38.48	0.271
Gradient Boosting	30.46	38.09	0.285

Because Rento sets prices per day, the same demand was also summed per day, using the 654 days on which at least 20 of the 24 hours were recorded; the last 131 days formed the test set. Table 5.7 shows that the daily task was easier to predict: Random Forest explained 49% of the variance of daily demand, compared with 11% for the weekday baseline.

Table 5.7 Results of Experiment 1b: daily demand (test set, 131 days)

Model	MAE	RMSE	R^2
Weekday baseline	267.45	430.65	0.114
Linear Regression	307.72	407.01	0.209
Random Forest	251.25	327.28	0.489
Gradient Boosting	269.61	347.29	0.424

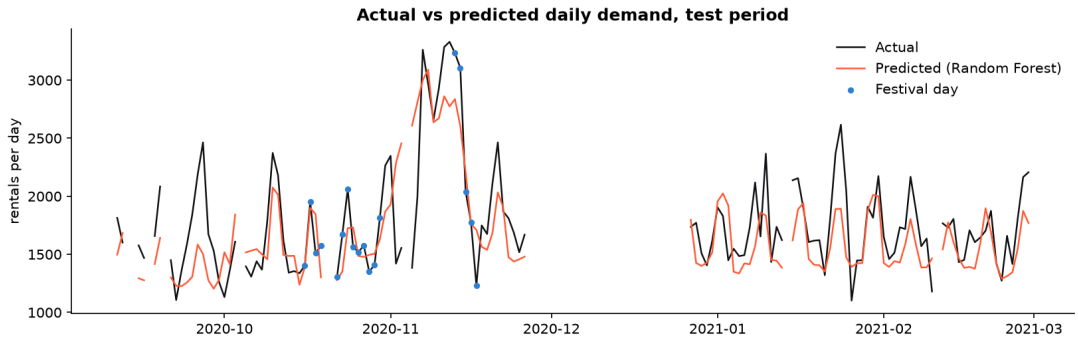


Figure 5.8 Actual and predicted daily demand on the test period

5.9.4 Results: Price

Table 5.8 shows the results for the listed day price. The target has only seven distinct values, between Rs 7,360 and Rs 12,800, because the marketplace prices are nearly

flat by category, as observed in Section 2.2.4. On the shared test set, all learned models performed far better than the median baseline, and Random Forest was the most accurate, with an MAE of Rs 100.60 and an R^2 of 0.981; the prototype's neural network reached an R^2 of 0.963. The grouped cross-validation, however, gave a much less optimistic picture. For vehicle models not seen in training, Gradient Boosting reached a mean MAE of Rs 527 and a mean R^2 of 0.461, Random Forest an R^2 of 0.246, and Linear Regression performed worse than a constant prediction. The most important feature was the estimated fuel cost per kilometre, which reflects the size of the vehicle.

Table 5.8 Results of Experiment 2: day price (test set, 246 listings)

Model	MAE (Rs)	RMSE (Rs)	R^2
Median baseline	1,412.68	2,075.44	−0.312
Linear Regression	226.82	561.80	0.904
Random Forest	100.60	251.31	0.981
Gradient Boosting	198.59	313.95	0.970
Multilayer perceptron (prototype)	134.07	347.03	0.963

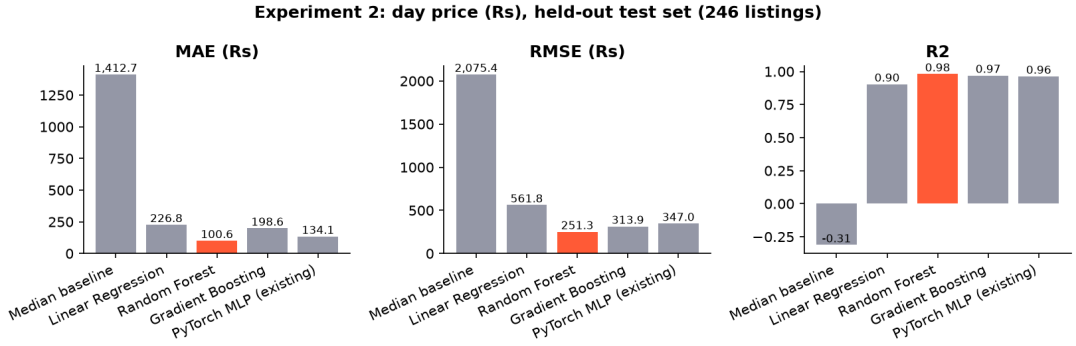


Figure 5.9 Comparison of the price models on the test set

5.9.5 Discussion

The experiments support three conclusions for Rento. First, tree ensembles were the most accurate models in all three tasks, which agrees with the findings of Grinsztajn et al. (2022) for tabular data; the neural network did not outperform them. Second, the gain over simple baselines was small for hourly demand and moderate for daily demand, which is consistent with the observation of Makridakis et al. (2018) that machine-learning methods often gain little over statistical benchmarks, and which supports the decision to rely on transparent rules until Rento's own data become available. Third, the high accuracy of the price models on a random split is largely

due to the same vehicle models appearing in both sets and to the few distinct price levels; for new vehicle models the error is much larger. A learned pricer for Rento will therefore need to be trained on Rento's own bookings, and it should be evaluated with time-ordered and grouped validation rather than with a random split. The pricing settings already contain a switch for a learned model and a threshold of 200 bookings, and the model health panel shows the progress towards it; the model remains locked until then.

5.10 Recommendation Engine

5.10.1 Trip Parser

The trip parser reads the trip description, which is converted to lower case, and extracts the criteria listed in Table 5.9 with keyword rules and regular expressions. A vehicle type that the customer names explicitly, such as "SUV" or "jeep", counts as requested; otherwise a type is inferred from the words that describe the trip and marked as a suggestion. The rules are kept identical in the JavaScript and PHP versions of the parser.

Table 5.9 Criteria extracted by the trip parser

Criterion	Rule (examples)
Road type and inferred car type	Mountain and rough roads, SUV: "mountain", "hill", "off road", "Mustang", "Jomsom", "Manang", "trek". Highway, sedan: "highway", "long drive", "business", "airport", "Pokhara", "Lumbini". City, hatchback: "city", "commute", "Kathmandu", "Bhaktapur", "parking", "solo"
Named car type	"hatchback", "sedan", "SUV" or "jeep", "MUV", "crossover", "coupe" or "sports car", "pickup", "van"
Budget per day	An amount such as "under Rs 8000" or "8k"; "cheap" is read as Rs 5,500 and "luxury" as Rs 15,000
Travellers	A number of people, such as "6 people"
Context	Family ("family", "kids", "parents"), luggage ("luggage", "bags", "airport"), business ("business", "meeting", "wedding"), group ("group", "friends") and comfort ("comfortable", "long drive", "luxury")
Fuel	Petrol, diesel or electric when mentioned

5.10.2 Scoring

The engine first removes the cars that cannot meet the hard requirements: cars with too few seats for the travellers, or fewer than five seats for a family or group trip; cars that are booked on the chosen dates; and cars whose price is more than 10% over the

budget. Each remaining car is then scored with the weighted model of Equation 2.8, using only the criteria that apply to the trip, with the weights in Table 5.10.

Table 5.10 Criteria and weights of the recommendation engine

Criterion	Weight	Partial score
Car type	30 if named, 15 if inferred	1 for the same type, a similarity between 0.2 and 0.8 for related types (for example 0.7 for a crossover when an SUV is wanted)
Seats	15	1 with up to two spare seats, 0.85 with up to four, 0.7 otherwise
Budget	20	1 up to 80% of the budget, falling to 0.9 at the budget, 0.4 when up to 10% over
Road	15, or 25 on mountain roads	Suitability of the car type for city, highway or mountain roads; on mountain roads only cars flagged as rugged score 1 and others at most 0.6
Comfort and luggage	10	Comfort or luggage space of the car type, for family, business, comfort or luggage trips
Fuel	10	1 for the requested fuel, otherwise 0
History	10	0.7 if the customer has rented the same type before and 0.3 for the same brand
Age	5	Newer models score higher, falling to 0 for a car 12 years old

For every criterion that contributes, the engine records a reason in plain language, and for weak points it records a caution, such as "Not a rugged 4x4: fine on rough patches, less suited to mountain tracks". Figure 5.10 shows how the budget and road criteria are scored and explained. The cars are then sorted by score and, for equal scores, by price, and matches below 40% are removed. Cars that are rented only with a driver are listed separately as suggestions to request with a driver. The engine is read-only: it never writes to the database, unlike the prototype, which overwrote the car type of every vehicle each time it was used.

```
if ($budget) {
    $share = $price / $budget;
    $add('budget', $share <= 0.8 ? 1.0 : ($share <= 1 ? 1 - 0.5 * ($share -
0.8) : 0.4));
    ...
    if ($share <= 1) $reasons[] = "$priceText, within your " .
format_price($budget) . ' budget';
    else $cautions[] = "$priceText, slightly over your " .
format_price($budget) . ' budget';
}
if ($road) {
    $suit = self::ROAD_SUITABILITY[$road][$kind] ?? 0.5;
    $rugged = !empty($car->Rugged) && !$sports;
    if ($road === 'mountain') {
        // Rough and mountain roads: being built for them matters more than the
body type.
        $suit = $rugged ? 1.0 : min($suit, self::ROAD_NOT_RUGGED);
    }
    $add($road === 'mountain' ? 'roadMountain' : 'road', $suit);
    if ($road === 'mountain' && $rugged) $reasons[] = 'Rugged, off-road
capable: built for mountain and rough roads';
    ...
}
```

Figure 5.10 Code excerpt: scoring and explaining the budget and road criteria (includes/recommendation_engine.php)

5.11 Security Implementation

Security measures were implemented throughout the system, beginning in Phase 0 with the weaknesses inherited from the template. Table 5.11 relates each measure to the risk it addresses, using the categories of the OWASP Top 10 (OWASP Foundation, 2021). Their effectiveness is tested in Chapter 6.

Table 5.11 Security measures

Risk	Measure
Weak password storage	Passwords hashed with bcrypt through password_hash; MD5 hashes from the template are verified once and replaced at the next login
Account takeover through password reset	Random reset token, stored only as a hash, valid for 30 minutes and for one use, sent by email; the same message is shown whether or not the email is registered
Brute-force login	At most 5 failed attempts per IP address or email address in 15 minutes for login, password reset and email checks
Cross-site request forgery	A random 32-byte token in the session, checked on every POST form with a constant-time comparison; all administrative actions use POST
Session attacks	Strict session mode, cookies only, HttpOnly and SameSite=Lax cookies (Secure over HTTPS), a new session identifier at login, separate sessions for administrators and customers
SQL injection	Prepared statements with bound parameters for every query with user input; filter values checked against allowed lists
Cross-site scripting	All user-controlled output escaped; HTML in content pages cleaned to an allowed set of tags
Price manipulation	Prices always calculated on the server and stored with the booking; browser values ignored
Malicious uploads	Images checked by extension, detected MIME type and image decoding, limited to 5 MB, stored under random names in folders where scripts cannot run
Exposure of internal files	Web server rules that deny access to the includes, logs, machine-learning and test folders and to Composer, PHPUnit and hidden files; database errors hidden from pages
Secrets	Database password and application key outside the repository; the SMTP password stored encrypted with the application key; least-privilege database user

5.12 Other Quality Features

Several further features were implemented in Phase 6 to prepare the system for real use. For accessibility, all pages have a skip link, visible focus rings, correctly ordered headings, labelled form fields and announcements for dynamic messages, and they respect the reduced-motion setting. For search engines, each page has a canonical URL, a description, Open Graph and X card tags and structured data in JSON-LD, and the site provides a sitemap and a robots file. A dark mode follows the system setting and can be switched manually on both the public site and the administration area. Email is sent through PHPMailer over SMTP for password reset links and for booking and enquiry confirmations; it remains switched off until the administrator enters the SMTP settings. Draft Terms and Privacy pages for Nepal were prepared but cannot be published while they contain items marked for confirmation. Finally, a deployment guide and a production check, which reports settings that are unsafe for

production, such as development mode, a site without HTTPS, a missing application key or email that is not set up, were written for moving the system from the local server to real hosting.

5.13 Summary

This chapter described the implementation of Rento. The code is organised into public pages, shared PHP modules, a JSON API, the administration area and the Python pricer, and the database was extended with 13 tables through 22 additive migrations. The customer modules show a price for every day and store the server-calculated price with each booking, and the administration modules give the operator control over the fleet, festivals, prices and content. The smart pricing engine applies seven types of factors within a −15% to +25% cap, and the forecast pipeline keeps 60 days of prices up to date for every car, safely and automatically. The machine-learning experiments showed that tree ensembles are the most suitable models, but that their advantage over simple baselines depends strongly on the task and the validation method, which justifies the rule-based pricer until Rento's own data are available. The recommendation engine combines a keyword-based trip parser with weighted, explained scoring, and the security measures address the main risks of a web application. Chapter 6 describes how the system was tested and evaluated.

CHAPTER 6: TESTING AND EVALUATION

6.1 Introduction

This chapter reports how Rento was tested and how well its intelligent components perform. Section 6.2 presents the test strategy and environment. Sections 6.3 to 6.5 report the unit, integration and system tests. Sections 6.6 and 6.7 evaluate the smart pricing engine and the recommendation engine, Section 6.8 reports the performance measurements, and Section 6.9 the accessibility, compatibility and usability checks. Section 6.10 relates the requirements of Chapter 3 to the evidence that they are met, and Section 6.11 summarises the chapter. The machine-learning experiments, which evaluate models for a future learned pricer, were reported in Section 5.9. All results in this chapter were produced by scripts in the project repository, and the raw results are stored with the report.

6.2 Test Strategy and Environment

Testing was carried out at four levels, summarised in Table 6.1. Unit tests check individual functions in isolation; integration tests send real HTTP requests to the running site and check the database afterwards; system tests exercise complete workflows in a real browser; and evaluation scripts measure the quality of the pricing and recommendation engines and the performance of the site. Tests were run after every phase, and the automated suites were run again for this report on 28 September 2026.

Table 6.1 Test plan

Level	Tools	Scope
Unit	PHPUnit 12.5 (30 tests); pytest 9.1 (46 tests)	Helper functions, pricing and booking totals, security functions, trip parser; pricing rules and forecast computation
Integration	PHPUnit with an HTTP client (24 tests)	JSON API, booking flow with stored price and invoice, protection of internal files, form tokens
System	Headless Google Chrome scripts and manual testing	End-to-end booking and enquiry, link crawl, JavaScript errors, layout overflow, keyboard use, axe accessibility checks
Evaluation	Python and Node.js scripts	480 forecast rows and quotes, 12 recommendation scenarios, response times, page weight

All tests ran on the development machine described in Table 3.6, against the local MAMP server (Apache 2.4.62, PHP 8.3.14, MySQL 8.0.40) with the real Rento

database. Tests that need an account used a temporary user with a random password, which was deleted, together with its bookings, at the end of the test; after the runs for this report, the database again contained its four real bookings and no test users.

6.3 Unit Testing

Table 6.2 summarises the unit tests. All 76 unit tests passed.

Table 6.2 Unit test results

Suite	What is tested	Tests	Result
HelpersTest (PHP)	Rental days counted inclusively (same day, month end, leap day, invalid and empty dates); SQL version of the rule; rupee formatting; clean URLs	11	Passed
PricingTest (PHP)	Booking total uses the stored quoted price; old bookings without a quote; rounding of package surcharges; market categories; readable reasons	5	Passed
SecurityTest (PHP)	bcrypt verification; MD5 hashes accepted once and upgraded; CSRF token must match; scripts removed from content HTML; encryption of secrets	5	Passed
TripParserTest (PHP)	Budget amounts; named and inferred car types; family context and default budget; empty input	9	Passed
test_pricing_properties (Python)	Caps and rounding for every day at several base prices, occupancy levels and request counts; random stacking of factors; travel days never combined with the festival surcharge; strongest festival counted once; season separate; lead-time boundaries at 0, 2, 3, 44 and 45 days; weekend days; demand factors only when switched on; readable reasons	33	Passed
PricingRulesTest (Python)	Plain weekday at base price; weekend; cap; discount floor; festival and season; travel days; demand gate; booked days	8	Passed
test_batch_forecast (Python)	One row per car and day; rows capped, rounded and explained; demand gate; booked days unavailable; manual override wins and keeps the rule reasons	5	Passed

The Python property tests are the strongest evidence for the correctness of the pricer. The test that stacks random combinations of factors confirms that no combination can produce a price outside the −15% to +25% range, and the boundary tests confirm that the last-minute factor applies up to two days ahead and the early-booking factor from 45 days, with no factor in between.

6.4 Integration Testing

The integration tests in Table 6.3 run against the live local site. The booking test logs in as the temporary user, requests a booking, and then checks in the database and on the pages that the stored price equals the sum of the forecast day prices, that the invoice shows the same total, and that the revenue on the administrator's dashboard changes by exactly that amount. The security tests request 14 internal files, among them the configuration files, a migration, the Python pricer, a saved model, the scraped dataset, the log, the decisions document, the Composer files, the Git folder, the PHPUnit configuration and the test bootstrap, and check that each one is refused. All 24 integration tests, with 184 assertions, passed.

Table 6.3 Integration test results

Suite	What is tested	Tests	Result
ApiTest	Quote total equals the sum of the forecast days; bad input rejected with 400; only GET allowed (405); calendar covers the rest of the month; recommendations ranked with reasons; rate limit answers 429	6	Passed
BookingFlowTest	Stored price, invoice and dashboard revenue agree; booking without a form token rejected; invoice visible only to its owner	3	Passed
SecurityHttpTest	14 internal files return 403 or 404; login without a form token does not log in	15	Passed

During the development of these tests, the security test found a real problem: the PHPUnit configuration file and the test bootstrap were served by the web server. Both were blocked with web server rules, the production check was extended to report them, and the test passed afterwards.

6.5 System Testing

System tests were carried out at the end of Phase 6 with scripts that drive a real browser, and manually. Table 6.4 lists the test cases and their results.

Table 6.4 System test cases

ID	Test case	Expected and actual result	Result
TC01	Customer logs in, selects dates on the price calendar and books	The quote of Rs 25,450 equals the sum of the day prices; the booking confirmation shows the same total	Pass
TC02	Customer opens the invoice	The invoice shows the stored total of Rs 25,450 and the price of each day	Pass

ID	Test case	Expected and actual result	Result
TC03	Administrator confirms the booking	The status changes to confirmed for the customer; revenue on the dashboard rises by exactly Rs 25,450	Pass
TC04	Visitor sends a package enquiry	A thank-you message is shown and announced; the administrator sees the enquiry and marks it as contacted	Pass
TC05	Booking and login without a valid form token	The request is rejected and nothing is stored (automated, Table 6.3)	Pass
TC06	Customer opens another customer's invoice	Access refused (automated)	Pass
TC07	Link crawl of 25 public and 228 administration pages	No broken links; the sitemap lists 25 URLs, all returning status 200	Pass
TC08	23 public and 41 administration pages at 1366 and 390 pixels, in light and dark mode	No JavaScript errors, no failed requests and no horizontal overflow	Pass
TC09	PHP error log after the tests	No errors; only the expected notice that dates beyond the forecast use the base price	Pass
TC10	Price calendar and booking with the keyboard only	Every function reachable; arrows move by day and week; focus always visible	Pass
TC11	Recompute prices from the Smart Pricing page	The run queued from the page (27 September 2026, 13:42) completed with 480 rows for 8 cars	Pass
TC12	Find My Car with "Mountain roads to Mustang, 5 people"	The rugged Toyota Fortuner is ranked first with reasons; non-rugged SUVs carry a caution	Pass

After the end-to-end tests, all test data were deleted and the prices recomputed, and the dashboard revenue returned to its previous value of Rs 57,000.

6.6 Evaluation of the Smart Pricing Engine

The pricing evaluation script read the whole forecast of 27 September 2026, 480 rows for 8 cars and 60 days, and recalculated every row independently from the settings, the festivals and the base prices. Its results are shown in Table 6.5. No row was outside the caps, no row was incorrectly rounded, and every stored price agreed with an independent calculation. In 48 rows, the sum of the factors exceeded the +25% cap, with a highest uncapped sum of +27%, and all 48 were capped and marked as such. Because fewer than 50 bookings were confirmed, the occupancy and demand factors were correctly off in every row. In addition, six quotes from the JSON API for two cars and three date ranges, including a Dashain weekend, were compared with the sum of the forecast days, and all six matched.

Table 6.5 Checks of the 60-day forecast (27 September 2026)

Check	Result
Rows (cars × days)	480 (8 × 60)
Rows outside the −15% / +25% limits	0
Rows not rounded to Rs 50	0
Rows whose price differs from an independent calculation	0
Rows in which the factors exceed the cap (all capped and marked)	48
Highest sum of factors before the cap	+27%
Range of multipliers	1.00 to 1.25
API quotes equal to the sum of the forecast days	6 of 6

Table 6.6 shows the breakdown of the price of the Toyota Fortuner on typical days. It confirms that the engine behaves as specified: prices rise at weekends, before and during Dashain and Tihar, and at the last minute, the early-booking discount cancels the season surcharge in November, and a Dashain travel day on a Friday reaches the cap. The price never fell below the base price in this period, because the only discount, for early booking, applied only to days that were also in the trekking season.

Table 6.6 Price of the Toyota Fortuner (base Rs 10,000) on typical days

Case	Date	Factors	Sum	Price (Rs)
Normal day	Wed 30 Sep	None	0%	10,000
Last minute	Sun 27 Sep	Last minute +5%	+5%	10,500
Weekend	Fri 2 Oct	Weekend +8%, season +4%	+12%	11,200
Dashain travel day	Fri 9 Oct	Weekend +8%, travel days +15%, season +4%	+27%, capped at +25%	12,500
Dashain main day	Sun 18 Oct	Dashain +12%, season +4%	+16%	11,600
Tihar travel day	Thu 5 Nov	Travel days +15%, season +4%	+19%	11,900
Tihar main day	Sun 8 Nov	Tihar +12%, season +4%	+16%	11,600
Early booking	Thu 12 Nov	Season +4%, early booking −4%	0%	10,000

This evaluation shows that the engine applies its rules correctly and consistently. It does not show that the chosen percentages are optimal for revenue, which can only be measured with real bookings over a season; this limitation is discussed in Chapter 7.

6.7 Evaluation of the Recommendation Engine

The recommendation engine was evaluated with twelve trip scenarios, written to cover families, mountain roads, city driving, business, large groups, luxury, budgets,

luggage, sports cars and electric cars. For each scenario, the set of relevant cars among the eight active cars was decided in advance from the car attributes, and the evaluation script called the recommendation endpoint and computed the precision at three, the normalised precision at three and the hit rate at one, as defined in Section 2.3.4. It also checked every returned car against the rules of the engine: enough seats, a price within the budget plus 10%, no inactive or driver-only car among the self-drive matches, scores in descending order, no claim that the customer asked for a type that was only inferred, and no claim about a fuel the car does not use. The results are shown in Table 6.7.

Table 6.7 Results of the recommendation scenarios

ID	Trip	First three results (match %)	P@3	nP@3	Hit@1
R01	Family trip to Pokhara under Rs 8,000	Nissan Sunny (97), Nissan Kicks (82), Vitara Brezza (81)	1.00	1.00	Yes
R02	Mountain roads to Mustang, 5 people	Toyota Fortuner (96), Nissan Kicks (79), Vitara Brezza (78)	1.00	1.00	Yes
R03	Daily city commute in Kathmandu, cheap	Wagon R (93), Nissan Sunny (70)	0.67	1.00	Yes
R04	Comfortable sedan for a business meeting	Nissan Sunny (96), BMW 5 Series (94), Nissan Kicks (60)	0.67	1.00	Yes
R05	Trip to Chitwan for 7 people	Toyota Fortuner (96)	0.33	1.00	Yes
R06	Luxury car for a wedding	Nissan Sunny (96), BMW 5 Series (95), Nissan Kicks (79)	0.33	0.33	No
R07	SUV under Rs 7,500	Nissan Kicks (83), Vitara Brezza (82)	0.67	1.00	Yes
R08	Road trip with lots of luggage	Toyota Fortuner (95), Nissan Kicks (77), Vitara Brezza (76)	1.00	1.00	Yes
R09	Sports car for a weekend drive	Nissan GT-R (92)	0.33	1.00	Yes
R10	Off-road trip to a village in the hills for 2	Toyota Fortuner (94), Nissan Kicks (72), Vitara Brezza (70)	0.33	1.00	Yes
R11	Cheap hatchback for two in the city	Wagon R (94), Nissan Sunny (64)	0.33	1.00	Yes
R12	Electric car for the city	Wagon R (71), Nissan Sunny (44), BMW 5 Series (42)	–	–	–
	Mean of 11 scored scenarios		0.61	0.94	0.91

R12 is not scored because the fleet has no electric car; the check confirmed that no result claimed to be electric.

The mean precision at three was 0.61, which is low mainly because several scenarios have only one or two relevant cars in a fleet of eight, so that even a perfect list cannot reach 1. The normalised precision at three, which allows for this, was 0.94, and the

first result was relevant in 10 of the 11 scored scenarios. All 12 scenarios passed the rule checks, and the median response time was 4 milliseconds.

The one clear failure was R06, a luxury car for a wedding. The parser read "luxury" as a budget of Rs 15,000 and as a wish for comfort, and inferred a sedan, but the engine has no criterion for luxury as such. As a result, the Nissan Sunny, an ordinary sedan, scored slightly higher than the BMW 5 Series because it is cheaper, and the Audi Q8 and Nissan GT-R did not appear in the first three. The scenario shows a limitation of keyword-based scoring: a word can carry a meaning, here prestige, that none of the criteria represents. Adding a luxury or segment criterion is a simple improvement for future work. R12 shows the opposite behaviour, which is desirable: when the requested fuel is not available, the engine still suggests suitable city cars with lower match scores and does not claim that they are electric.

6.8 Performance Evaluation

Response times were measured with 30 requests to each page and endpoint, with the client and the server on the same machine. The results in Table 6.8 show median times between 1.9 and 5.2 milliseconds and 95th-percentile times below 9 milliseconds. These times exclude network latency and are therefore lower than a user would experience on a hosted server, but they show that the server-side work is small, which is the result of precomputing the prices: no page or API call runs the pricing rules.

Table 6.8 Server response times (30 requests each, local server)

Request	Median (ms)	95th percentile (ms)	Size (bytes)
API: price quote (3 days)	2.0	4.2	1,268
API: price calendar (1 month)	3.1	5.0	5,088
API: recommendation (trip text)	2.6	3.4	1,761
Home page	5.2	8.9	48,310
Car listing	3.8	5.6	35,910
Car page (Toyota Fortuner)	4.2	6.0	39,235
Find My Car	1.9	4.4	24,492

Page weight was measured in headless Chrome with the cache disabled, after scrolling each page so that all lazily loaded images were fetched, and the same script had been used on the original template pages before the redesign. Table 6.9 and

Figure 6.1 show that the redesigned pages transfer between 33% and 80% less data and make fewer requests. The largest saving, on the home page, came from removing unused libraries and images and serving photographs as WebP in sizes suited to the screen. On a phone-sized screen of 390 pixels, the home page transfers 606 KB and the car page 552 KB.

Table 6.9 Page weight of the original template and of Rento (desktop, 1366 pixels)

Page	Template: requests, KB	Rento: requests, KB	Rento mobile: requests, KB	Change in KB
Home	40, 4,090	23, 831	21, 606	−79.7%
Car listing	38, 2,386	22, 540	22, 540	−77.4%
Car page	35, 1,171	29, 781	26, 552	−33.3%
Find My Car	28, 898	14, 387	14, 387	−56.9%
FAQs	29, 885	13, 389	13, 389	−56.0%
Contact	30, 919	14, 380	14, 380	−58.7%

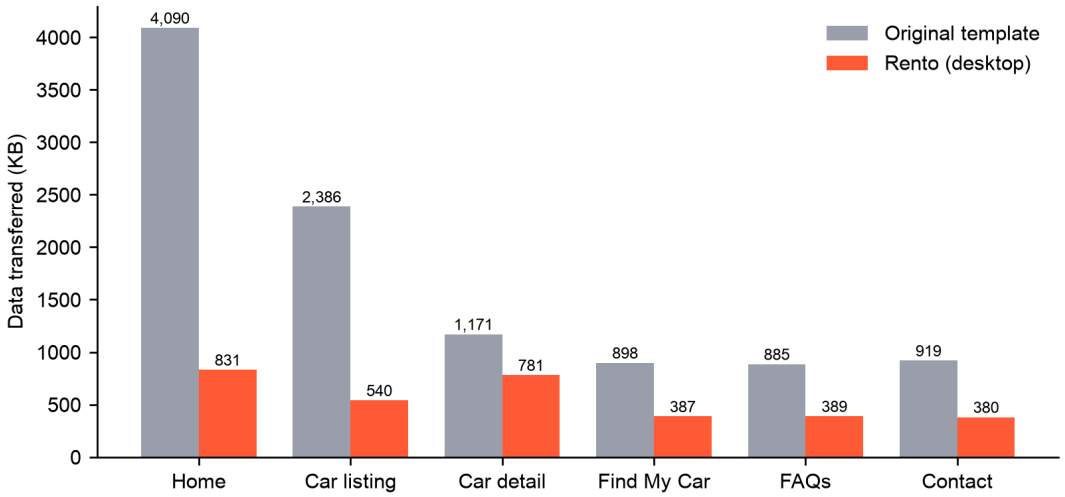


Figure 6.1 Data transferred per page by the original template and by Rento (desktop)

6.9 Accessibility, Compatibility and Usability

Accessibility was tested with the axe-core engine (Deque Systems, n.d.), which checks pages against WCAG rules, together with scripts that operate every page with the keyboard. At the end of Phase 6, axe reported no violations, including colour contrast, on 24 public pages, 42 administration pages and 8 customer pages, in light and dark mode, at widths of 1366 and 390 pixels. The keyboard script pressed the Tab key through every page at both widths and found no focus stop without a name, no invisible focus and no keyboard trap, and 29 keyboard scenarios for the interactive widgets, such as the mobile menu, the login dialog, the slideshow, the FAQ

accordion, the price calendar and Find My Car, all passed. Several problems were found and fixed during this testing, including focus rings with a contrast of only about 1.7:1, a skip link that scrolled without moving the focus, and price calendar days whose "cheaper" or "higher" status was shown only by colour.

Compatibility was tested in Google Chrome at desktop and phone widths in light and dark mode, as reported in test case TC08, and the screenshots in Chapters 4 and 5 show the layouts at both widths. Other browsers, such as Safari, Firefox and Microsoft Edge, and real mobile devices were not tested systematically.

The usability of the system was assessed by the developer through the end-to-end scenarios and the recommendation scenarios. No user acceptance test with customers or rental operators was carried out within the project. [INSERT results of user acceptance testing, if carried out before submission.]

6.10 Requirements Traceability

Table 6.10 relates each requirement of Section 3.4 to the tests that provide evidence that it is met.

Table 6.10 Requirements and test evidence

Requirement	Summary	Evidence
FR1	Accounts and password reset	SecurityTest; SecurityHttpTest (login token); TC01
FR2	Car search and filters	TC07, TC08; sequence of Figure 4.11 tested manually
FR3, FR4	Car page, price calendar and quote	ApiTest (quote, calendar); Table 6.5 (6 of 6 quotes); TC01, TC10
FR5, FR6	Booking with stored price, My Bookings and invoice	BookingFlowTest; TC01–TC03, TC05, TC06
FR7	Find My Car	ApiTest (recommend); TripParserTest; Table 6.7; TC12
FR8, FR9	Packages, enquiries and content	TC04, TC07
FR10–FR14	Administration, festivals, pricing settings, AI Insights	TC03, TC07, TC08, TC11
FR15	60-day forecast	test_batch_forecast; Table 6.5; TC11
FR16	JSON API	ApiTest; Table 6.8
FR17	Notifications	Not tested end to end: email is off until SMTP is configured; encryption of the SMTP password tested in SecurityTest
FR18	Dashboard and system check	BookingFlowTest (revenue); TC03, TC07
NFR1	Security	SecurityTest, SecurityHttpTest, BookingFlowTest; Table 5.11

Requirement	Summary	Evidence
NFR2	Correctness of prices	Property tests; Tables 6.5 and 6.6; BookingFlowTest
NFR3	Explainability	Readable-reason tests; rule checks of Table 6.7
NFR4	Performance	Tables 6.8 and 6.9
NFR5, NFR6	Accessibility and compatibility	Section 6.9; TC08, TC10
NFR7	Reliability and data integrity	Additive migrations with backups; single-transaction forecast; base-price fallback (TC09)
NFR8–NFR10	Maintainability, portability, privacy	Automated suites, documentation and production check; deployment not tested on real hosting

6.11 Summary

This chapter reported the testing and evaluation of Rento. All 76 unit tests and all 24 integration tests passed, and the twelve system test cases passed in a real browser; the tests found and helped to fix real problems, such as exposed test files and accessibility defects. The evaluation of the pricing engine confirmed that all 480 forecast prices respected the caps and rounding and agreed with an independent calculation, and that API quotes equalled the sum of the day prices. The recommendation engine reached a normalised precision at three of 0.94 and a hit rate of 0.91 on eleven scored scenarios, with one clear failure for a luxury request. The server responded in a few milliseconds, the pages transfer up to 80% less data than the original template, and no accessibility violations were found. The main gaps are the lack of user acceptance testing, of testing in other browsers and on real devices, of end-to-end testing of email, and of an evaluation of the revenue effect of the pricing rules. Chapter 7 draws the conclusions of the project.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Introduction

This chapter concludes the report. Section 7.2 summarises the work carried out, and Section 7.3 assesses it against the objectives set in Chapter 1. Section 7.4 states the conclusions of the study, Section 7.5 discusses its limitations, Section 7.6 proposes future enhancements, and Section 7.7 closes with final remarks.

7.2 Summary of the Work

The project set out to address a problem observed in the Nepali car rental market: prices that are flat or adjusted informally regardless of demand, little help for customers in choosing a vehicle for a trip, and bookings arranged mainly by telephone and messaging applications. The literature review showed that demand-based pricing is well established in car rental, airlines and hotels, that variable prices are better accepted when they are bounded and explained, and that content- and knowledge-based recommendation suits situations with little interaction data. No Nepali platform surveyed combined these ideas.

Rento was developed in eight phases from an open-source PHP car rental template and an earlier machine-learning prototype, with the help of an AI coding assistant and under the direction of the student. The template was first secured and stabilised, and its public site, administration area and most of its database were then rebuilt. The intelligent components were added on this base: a rule-based smart pricer with seven types of factors and a −15% to +25% cap; a forecast pipeline that keeps a price with its reasons for every active car for the next 60 days; a price calendar, stored quotes and invoices that make these prices visible and binding; a recommendation engine that interprets a plain-language trip description and ranks cars with explained scores; and offline experiments that compared machine-learning models for demand and price prediction on external data. The system was tested with 100 automated tests, twelve system test cases, an evaluation of all 480 forecast prices and twelve recommendation scenarios, and measurements of performance, page weight and accessibility.

7.3 Achievement of the Objectives

Table 7.1 assesses the project against the seven specific objectives of Section 1.3.2.

Table 7.1 Achievement of the specific objectives

Objective	Status	Evidence
1. Responsive web application with PHP, MySQL and a JSON API, covering listing, booking, invoices, packages and administration	Achieved	Chapters 4 and 5; 18 functional requirements traced to tests in Table 6.10; layouts tested at 1366 and 390 pixels
2. Smart pricing engine with demand factors, configurable limits and a reason for every adjustment	Achieved	Section 5.7; all 480 forecast prices within the caps and equal to an independent calculation (Table 6.5); readable reasons tested
3. Forecast pipeline for 60 days, recomputed nightly and after every relevant change	Achieved	Section 5.8; nightly job and change triggers; single-transaction replacement; test_batch_forecast suite
4. Recommendation engine for a natural-language trip description, with match scores and reasons	Achieved	Section 5.10; normalised precision at three of 0.94 and hit rate of 0.91 on eleven scored scenarios; all rule checks passed
5. Training and evaluation of machine-learning models against baselines	Achieved, on external data	Section 5.9; tree ensembles best in all tasks; learned pricer prepared but locked until about 200 bookings exist
6. Recognised web security practices	Achieved	Table 5.11; security unit and HTTP tests passed; internal files refused
7. Unit, integration and system testing and evaluation of performance, accessibility and responsiveness	Largely achieved	Chapter 6; no user acceptance testing and no testing in browsers other than Chrome

The general objective, to design, develop and evaluate a secure web-based car rental platform for Nepal that prices vehicles dynamically, explains every price and recommends suitable vehicles, has therefore been met, with the qualification that the dynamic pricing is rule-based rather than learned, for the reasons given in Section 7.4.

7.4 Conclusions

Four conclusions can be drawn from the study.

First, demand-based pricing can be introduced by a small rental business from the first day, without historical booking data, if it is based on explicit rules. The Nepali festival calendar, weekends, travel seasons and booking lead time provide enough known signals to set prices that follow the expected pattern of demand, as the 60-day forecast of the Toyota Fortuner shows, with prices rising from Rs 10,000 on an ordinary weekday to the capped Rs 12,500 on a Dashain travel day.

Second, transparency and control can be built into dynamic pricing. Every price in Rento is shown in advance with its reasons, limited by a cap, stored with the booking and reproduced unchanged on the invoice, and every automatic price can be overridden by the operator. These properties respond directly to the findings on price

fairness reviewed in Chapter 2, and the evaluation confirmed that the system keeps them for every car and every day of the forecast.

Third, machine learning is not yet the right tool for Rento's live pricing, but it is the right next step. The experiments showed that tree ensembles are the most accurate of the models compared, but also that their advantage over simple baselines was small for hourly demand and that the apparently excellent price results depend on seeing the same vehicle models in training and testing. With four real bookings, a learned pricer could not be trained or validated on Rento's own data. Keeping the rules live, while preparing the models, the switch and the threshold for a learned pricer, is therefore a sound approach to the cold-start problem.

Fourth, a simple, explained recommendation model can be useful with a small fleet. The weighted scoring engine recommended a relevant car first in ten of eleven scored scenarios and gave reasons and cautions that reflect its actual criteria. Its one clear failure, a request for a luxury car, shows both the limit of keyword-based understanding and how easily the model can be extended.

7.5 Limitations

The study has the following limitations, several of which were introduced in Section 1.6 and confirmed by the testing.

- **No real demand data.** Rento has only four real bookings. The pricing percentages were chosen by judgement and from the market survey, and their effect on revenue and occupancy could not be measured. The occupancy and demand factors and the learned pricer have not yet been used live.
- **External data for the experiments.** The demand dataset is not Nepali booking data, and the price dataset consists of listed prices from one marketplace with only seven distinct price levels. The experimental results show how the models behave but cannot be transferred directly to Rento.
- **Single market benchmark.** Only one operator published self-drive prices, so the base prices and the "below market" comparison rest on a single source.
- **Keyword-based trip understanding.** The trip parser recognises a fixed set of words and place names. It does not understand other phrasings, spelling mistakes or Nepali text, and it has no concept of luxury or prestige.

- **Small evaluation sets.** The recommendation engine was evaluated on twelve scenarios and eight active cars, with relevance judged by the developer.
- **Limited user testing.** No user acceptance test with customers or rental operators was carried out, and the interface was tested only in Google Chrome, without other browsers or real mobile devices.
- **Features not tested or not implemented.** Email notifications were not tested end to end because no SMTP server was configured. There is no online payment, no mobile application and no conversational assistant.
- **Security hardening.** The site does not yet send security headers such as a Content Security Policy or protection against framing, and the database has no foreign key constraints; relationships are maintained by the application code.
- **Local deployment and provisional data.** The system runs only on a local server and has not been deployed or load-tested on real hosting. The festival dates for BS 2084 are provisional until the government publishes its official list, and the festival calendar must be updated every year.
- **Starting point and assistance.** The system was built from an open-source template, and an AI assistant was used for much of the coding and documentation, as described in Section 3.2.

7.6 Future Enhancements

The following enhancements are proposed, in approximate order of priority.

1. **Deployment and user testing.** Deploy the system on a hosting service with HTTPS, following the deployment guide and the production check, configure email, and carry out user acceptance testing with customers and at least one rental operator.
2. **Security headers and constraints.** Add a Content Security Policy and related headers, and, after the legacy data have been cleaned, add foreign key constraints to the main relationships.
3. **Online payment.** Integrate the payment services widely used in Nepal, such as eSewa and Khalti, to take advance payments for bookings, which the market survey identified as a differentiator.

4. **Learned pricing.** Once about 200 bookings exist, train demand and price models on Rento's own data, evaluate them with time-ordered and grouped validation against the rule-based pricer, and use them first to suggest adjustments to the rule percentages and later, if they prove better, to set prices within the same caps and with explanations.
5. **Evaluation of the pricing rules.** Record occupancy and revenue per car and period, and compare periods or cars with and without selected factors, to test whether the rules improve revenue without reducing bookings.
6. **Richer trip understanding.** Add a luxury or segment criterion, recognise more places and phrasings, including Nepali, and consider a language model to interpret trip descriptions, while keeping the explained scoring model for the ranking itself.
7. **Official calendar import.** Import the festival and holiday dates from the official list each year, and warn the administrator when the calendar for the next year is missing.
8. **Fleet and marketplace features.** Support several branches, vehicle owners who list their own cars, and features found in the market survey, such as hourly hire, one-way drop fees and long-term discounts.

7.7 Final Remarks

Rento shows that the ideas of revenue management and recommender systems can be adapted to a small car rental business in Nepal, where data are scarce and customers are used to flat or negotiated prices. By starting with transparent, bounded rules and explained recommendations, and by preparing the data structures, models and switches for learned pricing, the system can be useful from the first booking and become more intelligent as its data grow. The project also provided practical experience of securing and rebuilding an existing system, designing and evaluating intelligent components honestly, and documenting the work, including the use of AI assistance, in a way that others can verify.

REFERENCES

Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences, 191*, 192–213. https://doi.org/10.1016/j.ins.2011.12.028

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction, 12*(4), 331–370. https://doi.org/10.1023/A:1021240730564

Carroll, W. J., & Grimes, R. C. (1995). Evolutionary change in product management: Experiences in the car rental industry. *Interfaces, 25*(5), 84–104. https://doi.org/10.1287/inte.25.5.84

Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature. *Geoscientific Model Development, 7*(3), 1247–1250. https://doi.org/10.5194/gmd-7-1247-2014

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. https://doi.org/10.1145/2939672.2939785

Consumer Protection Act, 2075 (2018). Government of Nepal. [VERIFY official source and URL]

den Boer, A. V. (2015). Dynamic pricing and learning: Historical origins, current research, and new directions. *Surveys in Operations Research and Management Science, 20*(1), 1–18. https://doi.org/10.1016/j.sorms.2015.03.001

Deque Systems. (n.d.). *axe-core: Accessibility engine for automated Web UI testing* [Computer software]. GitHub. Retrieved September 27, 2026, from https://github.com/dequelabs/axe-core

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Gallego, G., & van Ryzin, G. (1994). Optimal dynamic pricing of inventories with stochastic demand over finite horizons. *Management Science, 40*(8), 999–1020. https://doi.org/10.1287/mnsc.40.8.999

Gane, C., & Sarson, T. (1979). *Structured systems analysis: Tools and techniques*. Prentice-Hall.

Geraghty, M. K., & Johnson, E. (1997). Revenue management saves National Car Rental. *Interfaces, 27*(1), 107–127. https://doi.org/10.1287/inte.27.1.107

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35* (Datasets and Benchmarks Track). Curran Associates. [VERIFY pages]

Haws, K. L., & Bearden, W. O. (2006). Dynamic pricing and consumer fairness perceptions. *Journal of Consumer Research, 33*(3), 304–311. https://doi.org/10.1086/508435

Herlocker, J. L., Konstan, J. A., Terveen, L. G., & Riedl, J. T. (2004). Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems, 22*(1), 5–53. https://doi.org/10.1145/963770.963772

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting, 22*(4), 679–688. https://doi.org/10.1016/j.ijforecast.2006.03.001

IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers. https://doi.org/10.1109/IEEESTD.1998.88286

Individual Privacy Act, 2075 (2018). Government of Nepal. [VERIFY official source and URL]

International Organization for Standardization. (2011). *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models* (ISO/IEC 25010:2011). ISO.

Kahneman, D., Knetsch, J. L., & Thaler, R. (1986). Fairness as a constraint on profit seeking: Entitlements in the market. *The American Economic Review, 76*(4), 728–741.

Kathmandu Car Services. (2026, September 26). *Car rental prices in Kathmandu*. Retrieved September 27, 2026, from https://www.kathmanducarservices.com/car-rental-prices-in-kathmandu/

Kimes, S. E. (1989). Yield management: A tool for capacity-constrained service firms. *Journal of Operations Management, 8*(4), 348–363. https://doi.org/10.1016/0272-6963(89)90035-1

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer, 42*(8), 30–37. https://doi.org/10.1109/MC.2009.263

Larman, C., & Basili, V. R. (2003). Iterative and incremental development: A brief history. *Computer, 36*(6), 47–56. https://doi.org/10.1109/MC.2003.1204375

Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 73–105). Springer. https://doi.org/10.1007/978-0-387-85820-3_3

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLoS ONE, 13*(3), e0194889. https://doi.org/10.1371/journal.pone.0194889

Ministry of Home Affairs. (n.d.). *Government and public holidays in 2083*. Government of Nepal. Retrieved September 27, 2026, from https://www.moha.gov.np/page/government-and-public-holidays-in-2083

Nepal Vehicle Hiring. (2026, September 9). *How much does it cost to hire a car in Nepal?* Retrieved September 27, 2026, from https://vehiclehiringnepal.com/how-much-does-it-cost-to-hire-a-car-in-nepal/

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1* (formal/17-12-05). https://www.omg.org/spec/UML/2.5.1/

Oliveira, B. B., Carravilla, M. A., & Oliveira, J. F. (2017). Fleet and revenue management in car rental companies: A literature review and an integrated conceptual framework. *Omega, 71*, 11–26. https://doi.org/10.1016/j.omega.2016.08.011

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Rent In Nepal. (n.d.). *Electric car rental in Nepal* [Web page]. Retrieved September 27, 2026, from https://rentinnepal.com/electric-car-rental

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). *Recommender systems handbook* (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6

Sajilo Rental. (n.d.). *Sajilo Rental: Vehicle rental in Nepal* [Website]. Retrieved September 27, 2026, from https://sajilorental.com/

Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 253–260). Association for Computing Machinery. https://doi.org/10.1145/564376.564421

Self Drive Nepal. (2025, December 15). *Car rental price in Nepal for self drive*. Retrieved September 27, 2026, from https://selfdrivenepal.com/news/car-rental-price-in-nepal-for-self-drive

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Talluri, K. T., & van Ryzin, G. J. (2004). *The theory and practice of revenue management*. Kluwer Academic Publishers. https://doi.org/10.1007/b139000

Tintarev, N., & Masthoff, J. (2015). Explaining recommendations: Design and evaluation. In F. Ricci, L. Rokach, & B. Shapira (Eds.), *Recommender systems handbook* (2nd ed., pp. 353–382). Springer. https://doi.org/10.1007/978-1-4899-7637-6_10

Weatherford, L. R., & Bodily, S. E. (1992). A taxonomy and research overview of perishable-asset revenue management: Yield management, overbooking, and pricing. *Operations Research, 40*(5), 831–844. https://doi.org/10.1287/opre.40.5.831

Willmott, C. J., & Matsuura, K. (2005). Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate Research, 30*(1), 79–82. https://doi.org/10.3354/cr030079

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2* (W3C Recommendation). https://www.w3.org/TR/WCAG22/

Xia, L., Monroe, K. B., & Cox, J. L. (2004). The price is unfair! A conceptual framework of price fairness perceptions. *Journal of Marketing, 68*(4), 1–15. https://doi.org/10.1509/jmkg.68.4.1.42733

Ye, P., Qian, J., Chen, J., Wu, C.-h., Zhou, Y., De Mars, S., Yang, F., & Zhang, L. (2018). Customized regression model for Airbnb dynamic pricing. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 932–940). Association for Computing Machinery. https://doi.org/10.1145/3219819.3219830